

DESIGN AND EVALUATION OF INFRAGUARD AI: A
GOVERNANCE-BY-DESIGN REFERENCE FRAMEWORK FOR
BOUNDED LLM-ASSISTED INCIDENT TRIAGE

BY

ADEBODUN SAMUEL TOMOLA

MATRICULATION NUMBER: 2025/A/MIT/0616

DEPARTMENT OF INFORMATION TECHNOLOGY
SCHOOL OF COMPUTING
MIVA OPEN UNIVERSITY, ABUJA
NIGERIA

A PROFESSIONAL MASTER'S PROJECT SUBMITTED TO THE DEPARTMENT OF
INFORMATION TECHNOLOGY, SCHOOL OF COMPUTING, MIVA OPEN
UNIVERSITY, ABUJA, NIGERIA.

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE
PROFESSIONAL MASTER OF INFORMATION TECHNOLOGY DEGREE

AUGUST 2026

CERTIFICATION

I certify that this project is my original work, except where the work of others is acknowledged through citations and references. The project has not been submitted to this University or another institution for the award of a degree.

Student: Adebodun Samuel Tomola

Matriculation Number: 2025/A/MIT/0616

Signature: _____

Date: _____

APPROVAL

This project has been approved for the Department of Information Technology, School of Computing, Miva Open University, Abuja, Nigeria.

Project Supervisor: Dr. Emmanuel Mkpojiogu

Signature: _____ Date: _____

Head of Department: Dr. Abdullahi Mohammed

Signature: _____ Date: _____

Dean, School of Computing: Dr. Emeka Ogbuju

Signature: _____ Date: _____

External Examiner: Prof. Kingsley Obani

Signature: _____ Date: _____

DEDICATION

This project is dedicated to my family and to the engineers who maintain critical digital services with limited operational resources.

ACKNOWLEDGEMENT

I thank Almighty God for the strength and opportunity to complete this programme and project.

I am grateful to my project supervisor and the staff of the Department of Information Technology for their guidance and criticism throughout the work. Their feedback helped narrow the project to a testable implementation.

I also thank colleagues and practitioners who discussed the operational problems that motivated the system, particularly alert fatigue, slow incident triage, and the cost of maintaining several monitoring tools.

I acknowledge the maintainers of the open-source software used in the implementation, including FastAPI, LangChain, LangGraph, ChromaDB, Prometheus, Grafana Loki, and CrowdSec.

Finally, I thank my family and friends for their patience, encouragement, and support during the programme.

ABSTRACT

Cloud-native systems produce operational evidence faster than small teams can consistently interpret it. Monitoring exposes signals, but an operator must still correlate evidence, judge risk and retain authority over action. This design-science project develops and evaluates InfraGuard AI as an operator-hosted governance-by-design reference framework for bounded LLM-assisted incident triage rather than autonomous diagnosis.

The artefact collects selected Loki logs, Prometheus metrics, HTTP probes and optional Docker and CrowdSec signals through a bounded collect-analyse-decide-notify workflow. Its embedded governance controls include evidence-bounded context, structured verdict validation, provider/model traceability, inspectable runbook provenance, operator-controlled data and action boundaries, and approval before threat response. Selected telemetry still leaves the host when a cloud model is used.

After the 29 August review, the source-scoped suite passed 180 tests at 89% line coverage. The frozen qwen2.5:3b benchmark recorded 40.3% exact agreement with researcher-defined severity labels and 41.7% repeat-stable signatures. Relevant runbooks appeared in the top four for every labelled question, which demonstrates retrievability in the frozen corpus rather than answer correctness or actionable safety.

Two retained practitioner role reports supplied exploratory engineering observations, but absent assessor-by-item score records precluded an independently reproducible quantitative human evaluation. A separate item-level System Usability Scale workbook retained complete convenience-sample responses and produced a descriptive mean of 79.25, suggesting favourable perceived usability only within that group. The contribution is an inspectable governance-by-design reference framework, not production reliability, general diagnostic effectiveness, lower workload, cross-model equivalence or legal compliance.

Keywords: InfraGuard AI; AI governance; governance-by-design; incident triage; observability; retrieval-augmented generation; human oversight; design science; System Usability Scale.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION.....	1
1.1 Background to the Project.....	1
1.2 Statement of the Problem.....	2
1.2.1 Research Gap.....	2
1.2.2 Nature of the Contribution and Novelty Boundary.....	3
1.3 Aim and Objectives of the Project.....	3
1.3.1 Aim.....	3
1.3.2 Research Questions.....	3
1.3.3 Specific Objectives.....	4
1.4 Scope of the Project.....	4
1.4.1 Within Scope.....	4
1.4.2 Out of Scope (Identified as Future Work).....	5
1.5 Significance of the Project.....	5
1.5.1 Significance to Industry.....	5
1.5.2 Significance to Organisations.....	5
1.5.3 Significance to Policy and Society.....	5
1.5.4 Significance to Professional Practice.....	6
1.5.5 Significance to the Master of Information Technology Programme.....	6
CHAPTER TWO: LITERATURE REVIEW AND TECHNOLOGY CONTEXT.....	7
2.1 Conceptual Review.....	7
2.1.1 Cloud-Native Observability and the Three Pillars.....	7
2.1.2 Site Reliability Engineering and Incident Management.....	7
2.1.3 AIOps: Artificial Intelligence for IT Operations.....	7
2.1.4 Large Language Models and Agentic AI.....	7
2.1.5 Retrieval-Augmented Generation.....	8
2.1.6 Statelessness, Alert Deduplication, and Agent Memory.....	8
2.1.7 Human-Automation Decision Support, Alert Fatigue, and Trustworthy AI.....	8
2.1.8 Theoretical Foundations, Domain Concepts, and Design Constructs.....	9
2.2 Review of Existing Systems and Solutions.....	10
2.2.1 Comparative Table of Thirteen Recent and Related Studies.....	10
2.2.2 Synthesis of the Reviewed Studies.....	11
2.2.3 Commercial and Open-Source State of the Art.....	11
2.2.4 Feature-Level Comparison of Related Operator-State Mechanisms.....	12
2.3 Review of Relevant Technologies, Tools, and Frameworks.....	13
2.3.1 Programming Language and Runtime.....	13
2.3.2 Web and API Framework.....	13
2.3.3 LLM Orchestration and Provider.....	13
2.3.4 Observability and Telemetry Sources.....	14
2.3.5 Data Stores.....	14
2.3.6 Security and Deployment.....	14
2.3.7 Relevant Standards and Specifications.....	14
2.4 Gap Analysis.....	15
2.4.1 Grounding and response validation.....	15
2.4.2 Deployment control.....	15
2.4.3 Inspectable entry point.....	15
2.4.4 Recurring-condition acknowledgement.....	15
2.4.5 Operator approval for enforcement.....	16
2.4.6 Organisation-specific runbooks.....	16

2.4.7 Reproducibility.....	16
CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN.....	17
3.1 Project Methodology.....	17
3.1.1 Methodological Phases.....	17
3.1.2 Justification for the Methodology Choice.....	17
3.1.3 Tools Supporting the Methodology.....	18
3.1.4 Research-Question, Objective, and Evidence Alignment.....	18
3.1.5 Design-Science Research Mapping.....	18
3.1.6 Human Evaluation Design and Evidence Separation.....	19
3.2 Requirements Analysis.....	20
3.2.1 Functional Requirements.....	20
3.2.2 Non-Functional Requirements.....	21
3.2.3 Actors and Use Cases.....	22
3.3 System Architecture.....	23
3.3.1 Architectural Style.....	23
3.3.2 Components.....	23
3.3.3 Communication Patterns.....	24
3.3.4 High-Level Architecture.....	24
3.3.5 Deployment Topology.....	25
3.3.6 Heartbeat Verdict Pipeline.....	26
3.4 System Design.....	27
3.4.1 Database Design (Entity-Relationship Model).....	28
3.4.2 Use-Case Model.....	28
3.4.3 Sequence Diagram: One Heartbeat Cycle.....	29
3.4.4 Entity-Relationship Model.....	29
3.4.5 Verdict-Memory Design.....	30
3.4.6 State Machine: LangGraph Reasoning Agent.....	30
3.4.7 Workflow: Runbook RAG Indexing.....	31
3.5 Tools and Technologies Used.....	31
3.5.1 Backend and Orchestration.....	31
3.5.2 Security and Deployment.....	32
3.5.3 Telemetry Sources.....	32
3.5.4 Design Trade-offs and Alternatives.....	33
CHAPTER FOUR: SYSTEM IMPLEMENTATION.....	34
4.1 Development Environment.....	34
4.1.1 Hardware and Operating System.....	34
4.1.2 Source Control and Continuous Integration.....	34
4.1.3 Editor and Tooling.....	34
4.2 Implementation Details.....	34
4.2.1 Repository Layout.....	34
4.2.2 The Reasoning Agent and LangGraph Loop.....	36
4.2.3 The Two LLM Reasoning Modes.....	36
4.2.4 Prompt Construction and Response Validation.....	37
4.2.5 The Verdict-Memory.....	37
4.2.6 Telemetry Collection Tools.....	38
4.2.7 The RAG Runbook Service.....	38
4.2.8 Threat Detection and CrowdSec Response.....	38
4.2.9 The API Service and Middleware.....	38
4.2.10 Persistence and Retention.....	38
4.2.11 The Dashboard.....	39

4.2.12 API Endpoints.....	40
4.3 Security, Performance, and Scalability Considerations.....	41
4.3.1 Authentication and Session Management.....	41
4.3.2 Rate Limiting and Output Hygiene.....	41
4.3.3 Auditability.....	42
4.3.4 Performance and Responsiveness.....	42
4.3.5 Scalability and Degradation.....	42
4.3.6 Deployment Boundary.....	42
4.3.7 Data Governance and Cloud-LLM Boundary.....	43
4.3.8 Security Threats Specific to LLM-Assisted Incident Triage.....	44
4.4 Challenges Encountered.....	45
4.4.1 LLM Output Conformance.....	45
4.4.2 Scope Bounding and False Alarms.....	45
4.4.3 Signature Stability for the Memory.....	45
4.4.4 Security Defects Corrected During Review.....	46
4.4.5 Loki Query Semantics.....	46
CHAPTER FIVE: TESTING, RESULTS AND EVALUATION.....	47
5.1 Testing Strategy.....	47
5.1.1 Unit Tests.....	47
5.1.2 API and Persistence Tests.....	48
5.1.3 Acknowledgement Invariants.....	48
5.2 Automated Test Results.....	48
5.2.1 Coverage by Module.....	48
5.2.2 Representative Test Cases.....	49
5.3 Acknowledgement-Memory Evaluation.....	50
5.4 Limited Researcher-Observed Functional Verification.....	51
5.4.1 Runbook Retrieval Convenience Pilot.....	51
5.4.2 Controlled Local-Model Incident and RAG Benchmark.....	52
5.4.3 Descriptive Exploratory System Usability Scale Assessment.....	57
5.5 Evaluation Against the Objectives.....	58
5.6 Discussion and Threats to Validity.....	58
5.6.1 Supported Findings.....	58
5.6.2 Limitations.....	59
5.6.3 Validity.....	59
CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATIONS.....	60
6.1 Summary of the Project.....	60
6.2 Conclusion.....	60
6.2.1 Answers to the Research Questions.....	61
6.3 Research and Professional Contributions.....	61
6.4 Recommendations for Future Work.....	62
6.4.1 Evaluation.....	62
6.4.2 Test and Security Coverage.....	62
6.4.3 Architecture.....	63
REFERENCES.....	64
APPENDIX A: API ENDPOINT REFERENCE.....	69
A.1 Authentication and Health.....	69
A.2 Status, History, and Configuration.....	69
A.3 Verdict Memory.....	69
A.4 Threats.....	69
A.5 Runbooks (RAG).....	70

APPENDIX B: SAMPLE CODE SNIPPETS.....	71
B.1 Verdict Fingerprint and Signature Derivation.....	71
B.2 Severity Safeguard.....	71
B.3 Known-Conditions Memory Block.....	72
B.4 Deterministic Threat Detection.....	72
B.5 LangGraph Reasoning Loop.....	73
APPENDIX C: DOCKER COMPOSE CONFIGURATION.....	74
APPENDIX D: PRACTITIONER USABILITY INSTRUMENT AND DESCRIPTIVE EXPLORATORY SUS RESULTS.....	75
D.1 Task Script.....	75
D.2 System Usability Scale Questionnaire.....	75
D.3 Executed Descriptive Exploratory SUS Study.....	76
APPENDIX E: CONTROLLED INCIDENT AND RAG EVALUATION PROTOCOL AND OBJECTIVE RESULTS.....	78
E.1 Evaluation Questions and Fixed Conditions.....	78
E.2 Controlled Incident Scenario Set.....	78
E.3 Diagnostic Scoring Rubric and Metrics.....	80
E.4 Labelled RAG Baseline.....	80
E.5 Execution and Reporting Procedure.....	81
E.6 Evidence Status.....	82
E.7 Reproducibility Record.....	82

LIST OF FIGURES

Figure 2.1: Conceptual Framework for Operator-Hosted, LLM-Assisted Incident Triage.....	10
Figure 3.1: High-Level Architecture of InfraGuard AI.....	25
Figure 3.2: Intended Single-VPS Topology for DevPlanner and InfraGuard AI.....	26
Figure 3.3: Heartbeat Pipeline and Remediation Boundary.....	27
Figure 3.4: InfraGuard AI Use-Case Model.....	28
Figure 3.5: Sequence of One Heartbeat Cycle.....	29
Figure 3.6: Verdict and Acknowledgement Data Model.....	30
Figure 3.7: LangGraph Agent State Machine.....	31
Figure 4.1: Repository Layout of the Reference Implementation.....	35
Figure 4.2: Verdict-Memory Fingerprint and Acknowledgement Flow.....	37
Figure 4.3: InfraGuard AI Operations Dashboard.....	39
Figure 4.4: Local Runbook Indexing and Retrieval Process.....	40
Figure 4.5: Cloud-LLM Data Boundary and Local Retention Points.....	44
Figure 5.1: Selected-Module Coverage after Post-Review Verification on 29 August 2026..	49
Figure 5.2: Deterministic 30-Cycle Illustration of Default-View Duplicate Presentation.....	51
Figure 5.3: Proposed Five-Case Practitioner Usability Workflow.....	52

LIST OF TABLES

Table 2.1: Comparative Review of Thirteen Studies in AIOps and LLM-Assisted Operations.	10
Table 2.2: Feature-Level Comparison of Related Operator-State and Memory Mechanisms.	12
Table 2.3: Relevant Standards and Specifications.....	14
Table 3.1: Research-Question, Objective, Method, Evidence, and Current-Answer Alignment.	18
Table 3.2: Design-Science Mapping for InfraGuard AI.....	19
Table 3.3: Functional Requirements of InfraGuard AI.....	20
Table 3.4: Non-Functional Requirements of InfraGuard AI.....	21
Table 3.5: Telemetry Sources and Collection Tools.....	33
Table 3.6: Principal Design Choices, Alternatives, and Trade-offs.....	33
Table 4.1: Repository Module Map and Responsibilities.....	35
Table 4.2: API Endpoint Reference.....	40
Table 4.3: Security and Privacy Controls by Evidence Classification.....	43
Table 5.1: Verified Software-Test Chronology and Evidence Boundaries.....	48
Table 5.2: Representative Automated Test Cases and Results.....	49
Table 5.3: Live Runbook-Retrieval Pilot Configuration.....	51
Table 5.4: Controlled Benchmark Configuration and Provenance.....	53
Table 5.5: Objective Incident-Benchmark Results.....	53
Table 5.6: Severity Confusion Matrix for 72 Incident Responses.....	54
Table 5.7: Objective RAG and No-RAG Benchmark Results.....	54
Table 5.8: Practitioner Role-Report Evidence Status and Permitted Use.....	55
Table 5.9: RAG Evaluation Dimensions, Evidence Status, and Supported Conclusions.....	55
Table 5.10: Practitioner Role-Report Engineering Hypotheses for Future Testing.....	56
Table 5.11: Descriptive Exploratory System Usability Scale Results.....	57
Table 5.12: Objective and Evidence Matrix.....	58
Table 5.13: Provider and Model Interface versus Empirical Evidence Status.....	58
Table D.1: Included Human SUS Participant Scores.....	76
Table D.2: Mean Raw Response by SUS Item.....	77
Table E.1: Gold Severity Policy for the Controlled Benchmark.....	78

Table E.2: Controlled Incident Benchmark Scenarios and Gold Expectations.....	79
Table E.3: Per-Response Diagnostic Scoring Rubric.....	80
Table E.4: Labelled Runbook Question Set for RAG and No-RAG Comparison.....	81
Table E.5: Controlled Benchmark Reproducibility Record.....	82

CHAPTER ONE: INTRODUCTION

1.1 Background to the Project

Organisations increasingly operate containerised services that produce logs, metrics, and service events throughout the day. These signals are useful, but their volume and distribution make incident review difficult for a small operations team. An engineer may need to compare several sources before deciding whether a service is healthy, degraded, or failing.

Observability is the ability to infer a system's internal state from its outputs. Prometheus, Grafana Loki, OpenTelemetry, and related tools provide established ways to collect and inspect telemetry. They support detection, but they do not remove the need for a person to interpret the evidence and choose an appropriate response.

Site reliability engineering treats repetitive operational work as toil and recommends automation where the work is predictable and reviewable (Beyer et al., 2016). Initial incident triage fits this description in part: the inputs are available as telemetry, but a useful judgement still depends on context and experience. The purpose of this project is to assist that first review, not to replace the operator who remains responsible for action.

Large language models can summarise structured and unstructured technical material, and tool-using agents can retrieve additional evidence before producing an answer. These capabilities make LLM-assisted triage worth testing. They also create risks. A model may infer facts that are not present, misread an optional integration as a failure, or produce a response that does not match the expected structure. Prompt constraints, output validation, and human approval can reduce these risks, but they do not prove that every verdict is correct.

Nigerian organisations that operate digital services must also account for data-protection and sector-specific cybersecurity obligations. The Nigeria Data Protection Act 2023 sets duties for responsible processing of personal data. For institutions within its scope, the Central Bank of Nigeria's 2024 cybersecurity framework includes monitoring, reporting, and resilience controls (Central Bank of Nigeria, 2024; Federal Republic of Nigeria, 2023). Local deployment keeps the application records and access controls on the operator's host, but any prompt sent to a cloud LLM provider leaves that host.

LivWell (Integrated Wellness Inc.) provided the operational setting that informed the requirements: several environments, containerised workloads, a small team, and a need for clear incident records. These constraints led to a single-host design, bounded telemetry collection, request auditing, operator-approved threat response, and an acknowledgement feature for recurring conditions. The result is a working reference implementation called InfraGuard AI.

1.2 Statement of the Problem

Five practical issues remain even when monitoring tools are already in place:

1. Triage time. When several telemetry sources report a problem, a responder must compare the evidence before stating the likely cause and next action. This work tests whether a bounded first-pass summary can support that review.
2. Repeated low-risk findings. A known condition may appear on successive heartbeat cycles and occupy the same dashboard space as an unreviewed finding.
3. Ungrounded model output. An LLM can infer a cause that is not supported by the supplied telemetry or treat an optional integration as a fault. Its response therefore needs scope constraints, structured parsing, and operator review.
4. Deployment and data handling. Managed platforms and cloud model APIs differ in cost, hosting model, and the telemetry they receive. A small team needs a design whose data boundary can be inspected and configured.
5. Limited operator memory. A recurring condition may appear on every heartbeat even after an operator has reviewed it. Without an acknowledgement mechanism, the dashboard provides no durable way to distinguish a known low-risk condition from an unreviewed one.

These issues lead to the explicit research questions in Section 1.3.2. They examine the construction of bounded telemetry context, the operator-hosted architecture, the acknowledgement mechanism, the scope of the completed evidence, and the deployment and governance constraints of cloud or local inference.

1.2.1 Research Gap

Knowledge gap. Existing research demonstrates LLM-based incident summarisation, root-cause assistance, tool use and retrieval, but does not establish that a compact local model will classify operational severity reliably in a small-team, single-host setting. The project therefore examines bounded behaviour rather than assuming diagnostic effectiveness.

Design gap. Alert acknowledgement, grouping, deduplication, suppression, incident-management state and agent memory address related operator-state problems. The reviewed sources did not establish the same combination of evidence-bounded context, provider/model traceability, operator-controlled action, a model- and prompt-bound acknowledgement rule, and an independent severity safeguard for recurring LLM-assisted verdicts. This scoped distinction motivates the governance-by-design framework but does not by itself prove universal novelty or superiority.

Engineering gap. Small teams need an inspectable prototype that combines selected telemetry, provider-portable interfaces, local runbooks, structured validation, operator-approved enforcement and reproducible evidence. Deployment control and integration are engineering contributions, not claims that the prototype establishes a new general diagnostic capability.

Empirical evaluation gap. Evidence remains limited on comparative model performance, human diagnostic agreement, task completion, workload, alert fatigue, resource cost and

production effectiveness in this context. The present study addresses feasibility and a frozen single-model baseline while retaining those broader outcomes as future work.

1.2.2 Nature of the Contribution and Novelty Boundary

What is new? InfraGuard AI does not claim invention of observability, RAG, human oversight, alert acknowledgement or governance-by-design. Its contribution is their scoped operationalisation in one inspectable, operator-hosted incident-triage artefact with explicit evidence, data and action boundaries.

Conceptual contribution. A risk-driven governance-by-design model connects imperfect model behaviour to bounded context, visible provenance, validation, retained human authority and continuing oversight.

Architectural contribution. Telemetry collection, evidence construction, model inference, output validation, operator approval and auditable state are separated so their trust boundaries can be reviewed.

Engineering contribution. The single-host reference implementation and versioned evaluation package make the integration and its limitations inspectable without implying production readiness.

Distinctive implemented construct. A condition-and-ruleset acknowledgement fingerprint binds a reviewed condition to the model and prompt version while an independent rule preserves high- and critical-severity visibility.

Empirical contribution. Configuration-specific evidence shows that schema validity, retrieval success and favourable perceived usability do not establish diagnostic reliability. The novelty claim is therefore a scoped integration and operationalisation, not universal uniqueness or superiority.

1.3 Aim and Objectives of the Project

1.3.1 Aim

The aim of this design-science project is to design, implement and verify an operator-hosted, LLM-assisted observability and incident triage artefact that produces structured verdicts from selected telemetry, supports local runbook retrieval and operator-approved threat response, and records acknowledgements for recurring low-risk conditions.

1.3.2 Research Questions

RQ1. How can selected heterogeneous observability telemetry be collected and represented as a bounded, structured and reviewable incident verdict?

RQ2. What operator-hosted architecture supports LLM-assisted analysis with local runbook retrieval, structured validation, human oversight and provider/model traceability?

RQ3. How can prior operator acknowledgement mask an unchanged low-risk finding from the default view without hiding changed or high-risk conditions?

RQ4. What do software verification, the frozen qwen2.5:3b benchmark, the objective RAG retrieval baseline and the descriptive SUS study establish - and fail to establish - about implemented behaviour, model limitations and perceived usability?

RQ5. Which governance, security, data-flow and deployment controls are required for responsible operator-hosted use of cloud or local LLM inference?

1.3.3 Specific Objectives

The project pursued five objectives organised around the research lifecycle:

1. RO1 - Requirements. To derive, document and analyse functional, non-functional, security, data-governance and operational requirements for a bounded operator-hosted incident-triage artefact.
2. RO2 - Design and implementation. To design and implement a requirements-driven, multi-source telemetry and bounded LLM reasoning architecture that produces structured incident triage verdicts.
3. RO3 - RAG and operational assistance. To design and implement a retrieval-augmented runbook mechanism that provides operational guidance conditioned on locally maintained procedures.
4. RO4 - Human oversight and acknowledgement. To implement operator-controlled acknowledgement and threat-response mechanisms that manage recurring low-risk conditions while preserving visibility of high- and critical-severity findings.
5. RO5 - Verification and evaluation boundary. To verify implementation behaviour through automated tests, deterministic scenarios and a controlled benchmark; to describe perceived usability from retained SUS items; and to state the limits created by one evaluated model, purposive researcher-defined labels, provenance-limited practitioner reports and the absence of field evaluation.

1.4 Scope of the Project

1.4.1 Within Scope

- Collection of application/system logs (Loki), infrastructure metrics (Prometheus), endpoint health (HTTP probes), and optional Docker container telemetry, on a configurable heartbeat interval.
- Model analysis through the Gemini Developer API, Anthropic, OpenAI-compatible services, or Ollama, selected through environment configuration.
- Push notification of high- and critical-severity verdicts via the ntfy.sh service.
- Deterministic detection of HTTP/SSH brute-force and port-scan patterns in logs, with operator-approved CrowdSec IP banning (operating in dry-run mode when CrowdSec is not configured).
- A local runbook retrieval path based on Markdown files, ONNX all-MiniLM-L6-v2 embeddings and a ChromaDB vector index.
- A stateful acknowledgement memory with condition-and-ruleset fingerprinting and a configurable time-to-live. A changed signature, prompt version, or model produces a different fingerprint.

- A single-page operations dashboard with session authentication, rate limiting, security headers, request auditing, and output escaping. Production deployment requires HTTPS termination and restricted network access.

1.4.2 Out of Scope (Identified as Future Work)

- Autonomous code remediation, including ingesting security-scanner output such as SARIF, generating code or configuration patches, and opening pull requests. The agent diagnoses and recommends; it does not modify application code.
- A comparative production evaluation of local and hosted models, including capacity planning for self-hosted inference, remains outside the implemented evaluation scope.
- Multi-server fleet operation. The present work targets a single host monitoring its co-located workloads; a distributed control-plane-and-satellite topology is future work.
- Custom training, fine-tuning, or distillation of the underlying LLMs. Models are treated as black-box providers selected through the common interface: hosted models are accessed through their APIs, while Ollama uses the local service.
- Identity and access-management federation with external single-sign-on providers; the system uses cookie-based session authentication only.
- Native mobile or desktop client applications; the dashboard is delivered exclusively as a responsive web application.
- Formal security certification (e.g. SOC 2 or an ISO 27001 audit). The deliverable is a reference implementation and architectural blueprint, not a certified product.

1.5 Significance of the Project

1.5.1 Significance to Industry

Industry significance lies in an inspectable single-host prototype and a reproducible evaluation package that small teams can study without first adopting a large managed platform. The contribution is bounded engineering evidence and documented failure behaviour, not claimed superiority over commercial products.

1.5.2 Significance to Organisations

A small organisation can deploy, test, and modify InfraGuard AI on one host without first adopting a large observability platform. Whether that arrangement is suitable depends on the organisation's scale and controls. Running cost also depends on the LLM provider, heartbeat interval, prompt size, and surrounding infrastructure.

1.5.3 Significance to Policy and Society

The Nigeria Data Protection Act 2023 is relevant to organisations processing personal data; the Central Bank of Nigeria cybersecurity framework applies to institutions within its sector-

specific scope (Central Bank of Nigeria, 2024; Federal Republic of Nigeria, 2023). Operational logs can contain account identifiers, IP addresses, request paths or free text that may relate to an identified or identifiable person. A relevant deployment must therefore define its purpose and lawful basis, minimise prompt content, restrict access, set retention periods, and document applicable processor or cross-border transfer arrangements. InfraGuard AI's controls support these decisions but do not by themselves establish legal or regulatory compliance.

1.5.4 Significance to Professional Practice

Professional-practice significance lies in making governance and evidence boundaries explicit: operators can inspect selected telemetry, provider/model provenance, the structured verdict, acknowledgement rule, threat-response approval and controlled benchmark. The two practitioner role reports and descriptive SUS workbook add exploratory interpretation without converting the framework into a validated diagnostic system.

1.5.5 Significance to the Master of Information Technology Programme

For the Master of Information Technology programme, the work links an operational problem to a software artefact, automated tests, measured coverage, a reproducible benchmark, exploratory qualitative assessment by two human operations practitioners, and a separately retained ten-response human SUS analysis.

CHAPTER TWO: LITERATURE REVIEW AND TECHNOLOGY CONTEXT

2.1 Conceptual Review

The conceptual review connects observability and SRE with AIOps, LLM agents, retrieval-augmented generation, acknowledgement memory, human-automation decision support and socio-technical oversight. The purpose is not to inventory technologies, but to identify the claims, risks and design principles that the artefact can test.

2.1.1 Cloud-Native Observability and the Three Pillars

Observability supplies evidence from which operators infer system state, but telemetry alone does not establish cause. InfraGuard AI therefore selects Prometheus metrics, Grafana Loki logs, HTTP probes and optional Docker telemetry for interpretation; distributed tracing remains outside the implemented scope.

2.1.2 Site Reliability Engineering and Incident Management

Site reliability engineering uses telemetry, service-level objectives, and operational automation to improve reliability. Beyer et al. (2016) use the term toil for repetitive manual work that scales with service growth. Alert review and repeated handling of known conditions are relevant examples. InfraGuard AI addresses a limited part of this work by generating a first-pass verdict and recording operator acknowledgements.

2.1.3 AIOps: Artificial Intelligence for IT Operations

AIOps applies machine learning and analytics to operations data for tasks such as anomaly detection, event correlation, fault diagnosis, and remediation support. Chen et al. (2025) present AIOpsLab as a reproducible environment for deploying cloud systems, injecting faults, exporting telemetry, and evaluating LLM agents across operational tasks. Their results also show that agent capability depends on the task and evaluation setting. InfraGuard AI does not train an anomaly detector or claim autonomous remediation. It asks a general-purpose LLM to interpret a bounded set of collected evidence and returns the result for operator review.

2.1.4 Large Language Models and Agentic AI

Large language models can generate from supplied context and invoke tools that retrieve evidence (Gemini 2.5 Team, 2025; Yao et al., 2023). Reflexion demonstrates verbal feedback across attempts (Shinn et al., 2023), while InfraGuard AI uses a bounded LangGraph workflow and an optional small LangChain telemetry tool set (LangChain, n.d.). Neither

orchestration nor structured output guarantees a correct diagnosis, so operational action remains the operator's responsibility.

2.1.5 Retrieval-Augmented Generation

The retrieval-augmented generation layer uses local Markdown runbooks as its knowledge source (Lewis et al., 2020). Each approved Markdown file is loaded as one retrievable document, embedded locally with the ONNX all-MiniLM-L6-v2 model and stored in ChromaDB. The four most relevant documents are supplied to the selected model. The current implementation does not split files into smaller chunks.

RAG evaluation must separate retrieval relevance, grounding or traceability, answer correctness and actionable safety (Es et al., 2024). InfraGuard AI retains objective top-four retrieval records. The practitioner reports discuss answer quality, but their absent assessor-by-item records mean those dimensions are excluded from the thesis's quantitative findings.

2.1.6 Statelessness, Alert Deduplication, and Agent Memory

A scheduled triage loop can show the same condition on successive cycles even after it has been reviewed. Alert grouping and log-template research show why the identity used for matching matters. A broad key can merge different conditions, while exact raw text often fails to match because timestamps, identifiers, and values change. Drain, for example, derives stable log templates by separating fixed and variable tokens (He et al., 2017). Conventional alert grouping and deduplication merge similar events; suppression rules hide matching alerts; incident acknowledgement records operator ownership; and agent memory can influence later model reasoning. InfraGuard AI instead binds an operator acknowledgement to the condition signature, prompt version, and model. A match changes presentation for an ok or warning verdict but does not prevent the next model call or suppress high- and critical-severity visibility.

The current condition signature can be supplied by the same model whose verdict is being remembered. This creates a circularity risk: an unstable or incorrect signature may cause a recurring condition to be missed, merged with another condition or associated with the wrong acknowledgement. The benchmark exposed this weakness: only 10 of 24 scenarios (41.7%) produced one identical signature across all three runs. Prompt/model binding, expiry and the severity safeguard limit the consequences, but a stronger design would derive primary condition identity deterministically from telemetry before inference and use model text only as supporting description.

2.1.7 Human-Automation Decision Support, Alert Fatigue, and Trustworthy AI

Human-automation research distinguishes appropriate use from misuse, disuse and abuse, and relates the appropriate level of automation to task risk and retained human responsibility (Parasuraman & Riley, 1997; Parasuraman et al., 2000). Trust should be calibrated to demonstrated capability and supported by access to the basis for a recommendation (Lee & See, 2004).

Automation bias and complacency can persist among novice and experienced users when an imperfect aid competes for attention (Parasuraman & Manzey, 2010). Guidance for human-

AI interaction therefore emphasises capability disclosure, correction, user control and visible system change (Amershi et al., 2019), while cognitive forcing can reduce overreliance by requiring active engagement with evidence (Buçinca et al., 2021).

Alarm-reliability experiments show a cry-wolf effect in which response behaviour tracks perceived signal reliability (Bliss et al., 1995). A clinical-monitoring review likewise links excessive nuisance alarms with desensitisation, mistrust and delayed or absent response (Cvach, 2012). The clinical setting differs from IT operations, so this literature motivates careful notification design but does not prove that InfraGuard AI reduces alert fatigue.

Trustworthy AI requires more than users reporting trust: reliance should be warranted by evidence of reliability, safety, transparency and accountability (Jacovi et al., 2021). Human-centred AI similarly combines substantial automation with substantial human control rather than treating them as opposites (Shneiderman, 2020). The NIST Generative AI Profile adds risk identification, measurement, governance and monitoring as continuing lifecycle duties (National Institute of Standards and Technology, 2024).

For InfraGuard AI, these principles map to visible telemetry evidence, structured validation, retained provenance, an operator veto and explicit approval before disruptive action.

Acknowledgement only masks a qualifying recurring verdict from default presentation; inference and storage continue. The measured 40.3% exact agreement with researcher-defined severity labels means reliability was not established for unattended triage, so human review is a control requirement rather than a ceremonial approval step.

2.1.8 Theoretical Foundations, Domain Concepts, and Design Constructs

Primary methodological foundation. Design Science Research (DSR) structures the build-and-evaluate cycle and positions contribution as an evaluated prescriptive design rather than invention by assertion (Gregor & Hevner, 2013; Hevner et al., 2004; Peffers et al., 2007; Venable et al., 2016).

Primary governance foundation. Governance-by-design, informed by organisational AI-governance scholarship, embeds governance choices in architecture, workflows and evidence records. It does not replace organisational judgement or legal accountability (Mäntymäki et al., 2022; Morley et al., 2020; Mulligan & Bamberger, 2018; National Institute of Standards and Technology, 2023, 2024).

Supporting theoretical lenses. Human-automation reliance and trustworthy-AI scholarship supply the appropriate-reliance, transparency and retained-control boundary (Lee & See, 2004; Shneiderman, 2020). These lenses explain why human approval must be substantive when model reliability is limited.

Domain and technical foundations. Observability, SRE, AIOps, incident management and RAG define the problem vocabulary and motivate the telemetry, retrieval and operator-workflow choices; they are not treated as explanatory theories. The design constructs are bounded context, structured verdicts, provider/model provenance, inspectable retrieval, operator-controlled data and action boundaries, ruleset-bound acknowledgement, explicit approval and a high/critical visibility safeguard.

Figure 2.1 summarises the resulting hierarchy and risk-driven design logic. Imperfect model performance requires visible evidence, validation and retained authority; persistent memory requires ruleset binding, expiry and risk override; and cloud/local portability requires explicit egress and provider traceability. The figure is a design model, not an empirically validated explanatory theory.

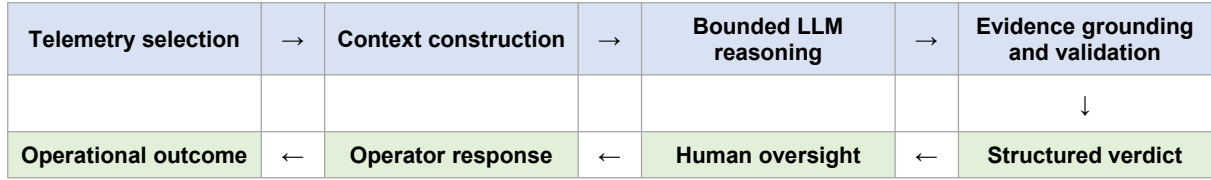


Figure 2.1: Conceptual Framework for Operator-Hosted, LLM-Assisted Incident Triage.

2.2 Review of Existing Systems and Solutions

Thirteen studies were reviewed across site reliability engineering, AIOps, log parsing, retrieval, and LLM-assisted operations. They are compared by method, evidence, limitations, and relevance to InfraGuard AI. Product documentation is considered separately in the discussion of commercial and open-source tools.

2.2.1 Comparative Table of Thirteen Recent and Related Studies

Table 2.1 provides a compact comparison of thirteen studies by method, evidence basis, finding, limit and relevance. The synthesis in Section 2.2.2, rather than the table alone, establishes the known, contested and unresolved claims that informed the design.

Table 2.1: Comparative Review of Thirteen Studies in AIOps and LLM-Assisted Operations.

#	Study	Method	Basis	Principal finding	Limit	Use in this study
1	Beyer et al. (2016)	SRE practitioner synthesis	Operational practice	Defines telemetry-led operations and toil.	Pre-LLM; human interpretation assumed.	Frames repeated triage as bounded operator work.
2	Chen et al. (2025)	Cloud-agent benchmark	AIOpsLab	Agent capability varies by task and setting.	Benchmark, not small-team deployment.	Motivates reproducible controlled evaluation.
3	He et al. (2017)	Streaming log parsing	Drain	Templates stabilise volatile log events.	No reasoning or operator memory.	Motivates deterministic identity before inference.
4	Yao et al. (2023)	Reason-and-act prompting	LLMs plus tools	Tool use can outperform study baselines.	Domain-agnostic; no operations trial.	Supports bounded read-only tool orchestration.
5	Shinn et al. (2023)	Verbal reflection loop	GPT-4 memory	Reflection improved selected multi-step tasks.	Toy code/QA tasks.	Contrasts model reflection with operator acknowledgement.
6	Yuan et al. (2025)	Retrieval-assisted diagnosis	RAIDR	Demonstrates IoT incident reporting with retrieval.	Short IoT demonstration.	Supports local runbook retrieval with visible sources.
7	Ahmed et al. (2023)	Prompting/fine-tuning	40,000+ cloud incidents	Reports root-cause and mitigation recommendations.	Proprietary hyperscale context.	Motivates evidence-bounded advisory output.
8	Jin et al. (2023)	Outage assessment/summarisation	18 cloud systems	Reports empirical and owner evaluation.	Proprietary data and deployment.	Supports structured records, not generalisable claims.
9	Chen et al. (2024)	LLM-assisted RCA	RCACopilot	Reports root-cause accuracy up to 0.766.	Hyperscale diagnostic services.	Motivates selected-source collection and inspectability.
10	Roy et al. (2024)	Agent RCA study	Tool-augmented LLMs	Grounding and tool choices affect results.	Microsoft data/services.	Supports human control and explicit tool boundaries.

#	Study	Method	Basis	Principal finding	Limit	Use in this study
11	Xu et al. (2025)	Open RCA benchmark	335 failures; 68+ GB	Best reported setup solved 11.34%.	Offline benchmark.	Supports mandatory review and model-specific evaluation.
12	Hou et al. (2024)	Systematic review	395 studies	Finds uneven methods and evidence practices.	No operations artefact.	Motivates reproducibility and claim calibration.
13	Guo et al. (2024)	Domain model/benchmark	OWL	Domain tuning improves IT-ops QA in the study.	Model-centric; no live telemetry loop.	Supports comparison as future work, not equivalence.

2.2.2 Synthesis of the Reviewed Studies

Ahmed et al. (2023), Jin et al. (2023), Chen et al. (2024), and Roy et al. (2024) evaluate LLMs for incident recommendations, outage summaries, or tool-assisted root-cause analysis, largely with proprietary cloud data. More recent open work addresses reproducible evaluation and exposes the remaining difficulty: AIOpsLab evaluates agents in controlled cloud environments, while OpenRCA reports that its best configuration solved 11.34 percent of 335 benchmark failures (Chen et al., 2025; Xu et al., 2025). RAIDR applies retrieval to incident diagnosis and reporting in an IoT setting (Yuan et al., 2025). InfraGuard AI uses related tool and retrieval patterns over a small telemetry surface, but keeps the operator responsible for action and does not claim diagnostic accuracy from software tests.

Known. The literature supports telemetry-driven investigation, bounded tool use, retrieval of organisational knowledge, structured operator workflows, accountability records and calibrated human oversight (Ahmed et al., 2023; Chen et al., 2025; Raji et al., 2020; Yao et al., 2023). Alert correlation, grouping and incident state can reduce repeated presentation without establishing diagnostic correctness (Salah et al., 2013).

Contested. Output fluency, schema compliance and tool access do not consistently translate into correct root-cause or severity judgements. Reported performance depends on the model, prompt, telemetry, labels and deployment context, so capability claims remain configuration-specific (Chen et al., 2025; Xu et al., 2025).

Unresolved. Stable incident identity, safe handling of prompt-injected telemetry, cross-model comparison, independent practitioner agreement, task performance, workload, end-to-end cost, multi-host scaling and production resilience remain insufficiently evidenced for this setting.

Design implication. InfraGuard AI instantiates a governance-by-design reference framework that combines inspectable adapters, provider/model provenance, local runbook retrieval, explicit human control and a condition-and-ruleset acknowledgement mechanism in a single-host prototype. The evaluation exposes configuration-specific strengths and failures rather than inferring production effectiveness from integration alone.

2.2.3 Commercial and Open-Source State of the Art

Four products provide useful points of comparison. The descriptions below are limited to capabilities stated in their current product documentation.

Datadog Bits AI and Watchdog

Datadog Bits AI supports investigations using Datadog telemetry, runbooks, environment context, feedback, and investigation memories (Datadog, n.d.). It is part of Datadog's managed commercial platform.

New Relic AI

New Relic AI provides natural-language access to telemetry held in the New Relic platform and can return explanations, summaries, charts, and troubleshooting guidance (New Relic, n.d.). It is a managed platform feature rather than an operator-hosted reference implementation.

PagerDuty AIOps

PagerDuty AIOps provides alert grouping, noise reduction, triage, root-cause features, event orchestration, and automation within PagerDuty's managed incident platform (PagerDuty, n.d.). Its event-grouping functions are related to, but not identical with, the acknowledgement memory studied here.

K8sGPT (Open Source)

K8sGPT is an open-source tool that analyses Kubernetes resources and sends selected information to a configured AI backend for explanation (K8sGPT, n.d.). Its primary scope is Kubernetes diagnosis rather than the multi-source heartbeat implemented by InfraGuard AI.

2.2.4 Feature-Level Comparison of Related Operator-State Mechanisms

Table 2.2 compares related operator-state, governance and memory mechanisms at feature level using scholarly and practitioner sources. The matrix establishes a scoped combination-level distinction only: 'not reported' does not mean absent, and the entries do not establish universal uniqueness, superiority or exhaustive coverage of changing commercial features.

Table 2.2: Feature-Level Comparison of Related Operator-State and Memory Mechanisms.

Mechanism	Identity or rule basis	Human and model-version treatment	Evidence-bounded distinction
Alert acknowledgement	Alert or incident record	Explicit operator state; model binding is not inherent	Tracks ownership or review of a recorded alert (Beyer et al., 2016; PagerDuty, n.d.).
Suppression or routing rules	Fields, thresholds, rules or time windows	Configured by an operator or administrator; prompt/model binding is not inherent	Redirects or hides matching events and can conceal matches if rules are too broad.
Alert correlation taxonomy	Rule, similarity, statistical or model-based correlation	Human review varies; LLM prompt/model identity is not part of the taxonomy	Classifies established correlation approaches rather than an acknowledgement standard (Salah et al., 2013).
Event deduplication or grouping	Event keys, similarity, correlation or templates	Usually automatic before human review; no LLM rationale identity is required	Clusters related noise rather than preserving a bounded judgement (He et al., 2017; PagerDuty, n.d.).

Mechanism	Identity or rule basis	Human and model-version treatment	Evidence-bounded distinction
Incident-management or AIOps memory	Incident timeline, ownership, resolution and investigation context	Acknowledgement and investigation memory are platform-dependent	Preserves lifecycle context within managed incident workflows (Datadog, n.d.; PagerDuty, n.d.).
Agent memory or reflection	Prior textual traces or model feedback	Human approval is not inherent; state may depend on model or prompt	Supports iterative reasoning but is not an incident-acknowledgement standard (Shinn et al., 2023).
AI governance and audit frameworks	Lifecycle risks, roles, controls and evidence	Organisational accountability and review are central	Provide cross-cutting governance foundations rather than this incident-triage implementation (Morley et al., 2020; National Institute of Standards and Technology, 2023; Raji et al., 2020).
InfraGuard AI reference framework	Condition signature + prompt version + active model within bounded evidence/action controls	Explicit for ok/warning; high/critical visibility and human approval are independently preserved	Instantiates a scoped governance-by-design combination; universal novelty, superiority, lower workload and lower inference cost are not established.

2.3 Review of Relevant Technologies, Tools, and Frameworks

This section records only the selection rationale for the principal technologies; Chapter Three specifies the research design and Chapter Four documents implementation. Choices were assessed for maintainability, licensing, operator control, provider portability and suitability for a small single-host deployment.

2.3.1 Programming Language and Runtime

Python was selected so FastAPI, LangGraph and LangChain orchestration, HTTPX integrations, ChromaDB, ONNX sentence embeddings, the Docker SDK and provider clients could share one testable runtime.

2.3.2 Web and API Framework

FastAPI was selected for asynchronous integration routes, dependency-based route controls and generated OpenAPI documentation. Co-hosting the dashboard and JSON endpoints preserves one session-authentication boundary and avoids a separate dashboard service.

2.3.3 LLM Orchestration and Provider

Provider dispatch is controlled by LLM_PROVIDER. The source code currently defaults to gemini-3.6-flash through the Gemini Developer API, but this is a configuration fact rather than evaluated model evidence. The provider and model name form part of the acknowledgement fingerprint, so a model change forces a fresh review of an otherwise recurring condition.

The researcher-observed retrieval pilot used DeepSeek V4 Flash through an OpenAI-compatible route, whereas the controlled benchmark used local qwen2.5:3b through Ollama. One interface supports Gemini, Anthropic, OpenAI-compatible and Ollama paths, but equivalent behaviour or performance across providers was not empirically demonstrated.

2.3.4 Observability and Telemetry Sources

Prometheus and Grafana Loki are the primary metric and log sources and are queried through their HTTP APIs. The deployment expects those services to be configured separately; it does not deploy a log shipper or monitoring backend. HTTP probes provide endpoint checks, the Docker Engine API provides optional local-container telemetry, and CrowdSec is available for operator-approved IP bans. Distributed tracing is outside the project scope.

2.3.5 Data Stores

Runbook embeddings are generated locally with the ONNX all-MiniLM-L6-v2 model. The vectors and document metadata are stored in ChromaDB, allowing retrieval without sending the runbook corpus to an embedding API.

2.3.6 Security and Deployment

Application controls include signed timestamped session cookies, rate limiting, response headers, output escaping, and request logging. Docker Compose packages the API and agent, Terraform provisions a Google Compute Engine host, and GitHub Actions runs tests before the deployment job builds and publishes images. A production deployment still requires HTTPS termination, restricted firewall rules, and protected secret handling.

2.3.7 Relevant Standards and Specifications

Table 2.3 lists the standards and specifications referenced in the design.

Table 2.3: Relevant Standards and Specifications.

Standard / Specification	Source	Role in InfraGuard AI
Prometheus Exposition Format	Prometheus Authors (n.d.)	Metric data format consumed by the metrics-collection tool.
Loki HTTP API (LogQL)	Grafana Labs (n.d.)	Log query interface used to retrieve recent log lines over an explicit time window.
OpenAPI 3.1	OpenAPI Initiative (2025)	Automatic description of the control-plane REST surface generated by FastAPI.
System Usability Scale (SUS)	Brooke (1996)	Standardised ten-item instrument used for the completed human survey; anonymised item-level responses and reproducible scoring are retained for ten complete cases.
ISO/IEC 27001:2022	International Organization for Standardization (2022)	Information-security management reference informing audit-trail and access-control design.
Nigeria Data Protection Act 2023	Federal Republic of Nigeria (2023)	Regional legal context for access control, accountability, and responsible processing.
Risk-Based Cybersecurity Framework for DMBs and PSBs	Central Bank of Nigeria (2024)	Sector-specific context for monitoring, reporting, and resilience controls.

Standard / Specification	Source	Role in InfraGuard AI
NIST AI 600-1	National Institute of Standards and Technology (2024)	Generative-AI risk reference for grounding, output validation, human oversight, and post-deployment monitoring.
OWASP Top Ten	OWASP Foundation (2025)	Reference for the dashboard's security-header, session, and threat-handling choices.

2.4 Gap Analysis

The review exposed seven gaps that informed the system design. They are not all gaps in academic knowledge: deployment control, inspectability and reproducibility are primarily engineering gaps, while stable acknowledgement across repeated LLM-assisted verdicts is a focused design-research problem. The central contribution is a governance-by-design reference framework for bounded LLM-assisted incident triage. Its distinctive subordinate construct is the fingerprint-based operator acknowledgement pattern integrated with bounded telemetry reasoning, local evidence retrieval and human-approved enforcement.

2.4.1 Grounding and response validation

An LLM can infer an unsupported cause or return malformed output. The NIST Generative AI Profile identifies confabulation, information security, human oversight, and post-deployment monitoring as risk-management concerns (National Institute of Standards and Technology, 2024). InfraGuard AI supplies selected telemetry, tells the model not to treat absent integrations as faults, and checks the response fields and severity. These controls constrain the interface, but they do not establish diagnostic accuracy.

2.4.2 Deployment control

The commercial platforms reviewed are vendor-managed services. InfraGuard AI keeps the API, dashboard, database and runbook index on the operator's host. This is an engineering and governance distinction rather than, by itself, a new academic capability. A prompt still leaves the host when the operator selects a cloud LLM provider.

2.4.3 Inspectable entry point

A small reference implementation allows the collection and reasoning paths to be inspected without adopting a full observability platform. This does not imply lower operating cost; cost depends on the host, provider, prompt size, and request rate.

2.4.4 Recurring-condition acknowledgement

A scheduled triage loop needs a durable distinction between a reviewed low-risk condition and a new finding. InfraGuard AI stores a fingerprinted acknowledgement and sets a presentation flag on a matching ok or warning verdict so default API/dashboard views may

hide it. The next heartbeat still performs inference and stores its verdict. A change to the signature, prompt version, or model produces a different fingerprint.

2.4.5 Operator approval for enforcement

IP bans can interrupt legitimate traffic. InfraGuard AI therefore applies a CrowdSec ban only after an operator approves the detected threat. Acknowledgement rules also prevent high and critical findings from being hidden.

2.4.6 Organisation-specific runbooks

Operational procedures are maintained as local Markdown files under the runbooks directory. Docker Compose mounts this directory read-only into the API and agent containers, while subdirectories provide category metadata for retrieval.

2.4.7 Reproducibility

Private datasets and managed infrastructure limit direct reproduction of several reviewed systems. InfraGuard AI keeps the collection, matching, persistence, and API logic in one repository so another evaluator can inspect those decisions.

These gaps set the boundary for the implementation. Chapter Three translates them into requirements and design decisions, while Chapter Five limits empirical claims to the evidence actually collected. The governance-by-design reference framework is the principal design-science contribution; the acknowledgement fingerprint is its distinctive implemented construct. The completed human SUS survey supports only descriptive perceived-usability findings; broader claims about reduced workload, population-level usability, independently human-rated diagnostic accuracy, cross-model performance or production effectiveness require the independent, comparative and field evaluations specified in Appendices D and E.

CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN

3.1 Project Methodology

An iterative design-science research method guided the project. It connects problem identification, objectives, artefact design, demonstration, evaluation and communication rather than treating implementation as the research result by itself (Hevner et al., 2004; Peffers et al., 2007). Short cycles allowed requirements, code, tests and documentation to change together, while the evaluation boundary distinguishes verified software behaviour from operational effectiveness that remains unmeasured.

3.1.1 Methodological Phases

The work moved through six overlapping DSR phases. Test results or code review sometimes required an earlier decision to be revisited.

1. Problem identification. Reviewed observability, SRE, AIOps, incident-analysis, RAG, and agent research, then defined the narrow problem of first-pass telemetry interpretation for a small team.
2. Objective definition. Converted the problem into five implementation and evaluation objectives.
3. Design and development. Implemented telemetry adapters, the reasoning workflow, provider abstraction, runbook retrieval, threat handling, persistence, API, dashboard, and acknowledgement memory.
4. Demonstration. Exercised the main workflow paths with mocked upstream services and controlled test fixtures.
5. **Evaluation.** Separated software verification; objective controlled-model and retrieval measures; provenance-limited qualitative practitioner reports; descriptive item-level SUS evidence; and security/data-governance analysis. The practitioner aggregates were excluded from quantitative findings because their item-level records were unavailable.
6. Communication. Updated the report, diagrams, references, and appendices so that they describe the implemented system and measured evidence.

3.1.2 Justification for the Methodology Choice

A strictly sequential method would have been a poor fit because output parsing, acknowledgement matching and middleware order became clearer during implementation. The six-phase DSRM provides a transparent process for revisiting those decisions while

maintaining traceability from problem and objectives to demonstration, evaluation and communication (Peffers et al., 2007).

3.1.3 Tools Supporting the Methodology

The Git repository contains the source code, infrastructure definitions, tests, and report sources. Two GitHub Actions workflows are included: pull requests run the test suite, while pushes to the main branch run tests before the container build and deployment steps.

3.1.4 Research-Question, Objective, and Evidence Alignment

Table 3.1 makes the research chain explicit. A current answer is reported only where the completed evidence supports it; unresolved effectiveness questions remain bounded as future empirical work.

Table 3.1: Research-Question, Objective, Method, Evidence, and Current-Answer Alignment.

RQ	Objective	Method and Evidence	Current Answer and Boundary
RQ1	RO1-RO2	Requirements analysis; architecture and adapter implementation; unit and orchestration tests.	A bounded context and structured-verdict path were implemented and verified with fixtures; live diagnostic accuracy remains unmeasured.
RQ2	RO2-RO4	Architecture analysis; output-schema tests; RAG pilot; review of operator approval boundaries.	The operator-hosted design, local runbooks, validation and approval boundary functioned in the tested environment.
RQ3	RO4	Fingerprint, expiry, mismatch and severity-safeguard tests; 30-cycle display scenario; frozen signature-stability result.	Exact fingerprint matching masks a qualifying repeated low-risk row and preserves high-risk visibility, but only 41.7% of frozen scenarios produced repeat-stable model signatures; dependable condition identity and workload reduction remain unresolved.
RQ4	RO5	Dated software verification; frozen qwen2.5:3b incident and objective RAG retrieval benchmark; descriptive item-level SUS analysis; validity analysis. Practitioner role reports are provenance-limited qualitative material only.	The completed evidence establishes implemented behaviour, configuration-specific model limitations, frozen-corpus retrievability and favourable perceived usability in the convenience group. Diagnostic validation, human-rated RAG comparison, cost, workload, scaling and production effectiveness remain unresolved.
RQ5	RO1, RO4-RO5	Threat model and deployment review; security controls; Nigerian data-protection analysis.	The demonstration boundary and required production controls are identified; no compliance certification is claimed.

3.1.5 Design-Science Research Mapping

The design-science logic used in the project is summarised in Table 3.2. Its evaluation strategy moves from artificial, controlled tests toward limited naturalistic practitioner access, consistent with the distinction in the FEDS framework (Venable et al., 2016). The completed evidence remains weighted toward controlled evaluation and is not proof of production effectiveness.

Table 3.2: Design-Science Mapping for InfraGuard AI.

Design-Science Element	InfraGuard AI Application	Evidence Location
Problem relevance	Incident-triage burden, repeated findings, grounding risk and data-boundary concerns.	Chapter One
Artefact	Operator-hosted InfraGuard AI agent, API, dashboard, persistence and runbook retrieval.	Chapters Three and Four
Design principles	Bounded context, structured output, human approval, local evidence and acknowledgement safeguards.	Chapters Two to Four
Demonstration	Single-host architecture, mocked telemetry fixtures, a researcher-observed convenience runbook-retrieval pilot, and a controlled local-model benchmark.	Chapter Five
Evaluation	Automated tests, measured coverage, acknowledgement scenario, objective incident metrics, objective RAG retrieval, provenance-limited practitioner reports, and a reproducible descriptive SUS study.	Chapter Five
Contribution	Governance-by-design reference framework (primary); condition-and-ruleset acknowledgement fingerprint (distinctive implemented construct); inspectable reference implementation and bounded empirical findings.	Chapters Five and Six
Outstanding evaluation	Authentic assessor-by-item records or a new independent panel; gold-label review; cross-model and production comparison; adversarial model evaluation; full resource cost; instrumented task study.	Appendices D and E

3.1.6 Human Evaluation Design and Evidence Separation

Two human operations practitioners, anonymised as A1 (DevOps lens) and A2 (SRE lens), received separately randomised role packages covering the retained incident and RAG/no-RAG outputs. The unchanged role reports and hashes are retained, while names, identity mapping and email headers remain confidential. The reports are used only to identify qualitative engineering hypotheses. The SRE report records seed 2026082412 in its opening record and summary but 2026081902 in its method table. The preserved correction notice identifies 2026082412 as the authoritative human role-package seed and leaves the hashed report binary unchanged.

The public A/B worksheets preserve 72 assessment identifiers per condition but contain no populated assessor-by-item score cells. Consequently, the report means, rankings, agreement percentages and kappa coefficients cannot be verified or recomputed and are excluded from the thesis's primary evidence, research-question answers and conclusions. The role reports do not provide diagnostic validation or a generalisable practitioner study.

Descriptive exploratory System Usability Scale study.

The distinct human-usability stream used the ten-item System Usability Scale after practitioners accessed the dashboard. SUS is a global perceived-usability measure, not a measure of workload, task success or diagnostic correctness (Bangor et al., 2008; Brooke, 1996). Eleven consented submissions were received; complete-case analysis retained ten and excluded P9 because Question 3 was blank, with no imputation.

Odd-numbered items were scored as response minus 1; even-numbered items were scored as 5 minus response. The ten adjusted contributions were summed and multiplied by 2.5. The exported form's quiz Score column contains zeros and is not a SUS score, so it was excluded. The supporting workbook retains the unchanged source sheet, inclusion decisions, formulas, participant scores, summary statistics, item means and source hashes.

Participant codes P1-P11 are anonymised survey identifiers, not role labels. According to the researcher's provenance confirmation, the survey records are original human-practitioner responses. Identity and contact evidence was supplied separately to the supervisor and is not reproduced in the public document. The analysis is descriptive: the small convenience sample and absence of task-level timing, error, assistance and free-text records prevent population, causal or task-performance claims.

3.2 Requirements Analysis

No formal stakeholder-interview or elicitation sample was retained. Requirements were researcher-derived from the operational problem, reviewed literature and standards, repository constraints, and the supervisor-agreed project boundary. The later benchmark assessors and SUS respondents were evaluation participants, not sources of initial requirements.

Prioritisation used Must for the heartbeat, bounded evidence, structured verdict, persistence, operator control and safety constraints, and Should for optional integrations or controls that did not define the core artefact. Validation combined repository inspection, requirement-to-component tracing and automated tests; it did not constitute external stakeholder validation or operational-field validation.

Selected need-to-evidence traces are:

Bounded telemetry and verdict need → FR-01/FR-02 → agent tools, prompts, schema and orchestration → unit and orchestration tests.

Recurring-condition control → FR-07/FR-08/FR-09 → agent/memory.py, api/store.py and acknowledgement routes → memory, store and API tests.

Local runbook assistance → FR-12/FR-13 → runbook loader, ONNX embeddings and ChromaDB retrieval → runbook and vector-store tests plus the frozen RAG benchmark.

Human-controlled threat response → FR-10/FR-11 and NFR-04-NFR-07 → deterministic detector, CrowdSec adapter and authenticated middleware → threat-response and API tests.

Data and deployment governance → NFR-08/NFR-10/NFR-13/NFR-14 → Compose, Terraform, prompt selection and provider traceability → configuration review and focused tests, not independent security or legal assessment.

3.2.1 Functional Requirements

Table 3.3: Functional Requirements of InfraGuard AI.

ID	Requirement	Description	Priority
FR-01	Telemetry Collection	The system shall collect logs (Loki), metrics (Prometheus), endpoint health (HTTP probes), and optional container state (Docker Engine API) on a configurable heartbeat interval.	Must
FR-02	Structured Verdict Generation	The system shall send collected context to the configured LLM provider and request a verdict containing severity, summary, root cause, recommended action, and signature.	Must
FR-03	Severity Classification	Each verdict shall be classified into one of four severities: ok, warning, high, or critical.	Must
FR-04	Dual Reasoning Modes	Support both direct provider calls and LangChain multi-tool reasoning mode.	Must

ID	Requirement	Description	Priority
FR-05	Notification	The system shall dispatch a push notification via ntfy.sh for verdicts of high or critical severity.	Must
FR-06	Verdict Persistence and Retention	Verdicts shall be persisted to SQLite and automatically pruned after a configurable retention window.	Must
FR-07	Verdict Memory (Acknowledgement)	The artefact shall fingerprint each verdict on its condition signature plus ruleset version and allow an operator to acknowledge a low-risk condition so that a matching row is masked from the default view while its record is retained.	Must
FR-08	Memory Matching	A changed condition signature, prompt version, or model shall produce a different fingerprint; acknowledgements may also expire after a configurable time-to-live.	Must
FR-09	Severity Safeguard	High- and critical-severity verdicts shall never be masked from visibility by an acknowledgement.	Must
FR-10	Threat Detection	The system shall scan recent logs for HTTP/SSH brute-force and port-scan patterns and surface detected threats with their source IPs.	Must
FR-11	Operator-Approved Threat Response	On explicit operator approval, the system shall build and apply a CrowdSec ban for a detected threat, operating in dry-run mode when CrowdSec is unconfigured.	Should
FR-12	Runbook Indexing	Index and retrieve local Markdown runbooks using local embeddings and ChromaDB.	Must
FR-13	Runbook Query (RAG)	The system shall answer operator questions with an advisory response conditioned on retrieved runbook text and return source titles.	Must
FR-14	Operations Dashboard	The system shall provide a web dashboard exposing current status, integration configuration, check history, the threat panel, and the runbook assistant.	Must
FR-15	Integration Status	The system shall report which integrations are configured, exposing booleans only and never secret values.	Should

3.2.2 Non-Functional Requirements

Table 3.4: Non-Functional Requirements of InfraGuard AI.

ID	Quality Attribute	Requirement	Target
NFR-01	Cost Control	The heartbeat interval and prompt size shall be configurable so provider usage can be bounded.	One analysis request per heartbeat
NFR-02	Grounding Controls	The prompt shall include only configured evidence sections and shall instruct the model not to treat absent integrations as failures.	Covered by prompt-construction tests
NFR-03	Responsiveness	Runbook indexing and embeddings shall remain local to the deployment.	No direct blocking call in route coroutine
NFR-04	Authentication	All routes other than /health and /login shall require a valid signed session cookie.	24-hour maximum age
NFR-05	Rate Limiting	The API shall enforce request limits, with a stricter limit on the login route.	60/min; 5/min on login

ID	Quality Attribute	Requirement	Target
NFR-06	Output Hygiene	Server-derived text rendered in the dashboard shall be HTML-escaped and response security headers shall be applied.	Escaping and headers covered by tests
NFR-07	Auditability	Each request shall be logged with timestamp, IP, route, method, user, status, and latency.	Structured local request log
NFR-08	Portability	The API and agent shall deploy through Docker Compose on one Linux host.	Single-host composition
NFR-09	Maintainability	The backend shall have an automated suite with measured line coverage.	Coverage reported, not used as a sole quality measure
NFR-10	Data Minimisation	Only selected telemetry sections shall be included in the prompt sent to the configured LLM provider.	No full observability-store export
NFR-11	Storage Retention	Verdict storage shall be bounded by automatic retention pruning.	Default 30 days, configurable
NFR-12	Signature Stability	Fallback signature derivation shall remove volatile IP, hash, and numeric tokens.	Deterministic tests
NFR-13	LLM Data Boundary	Prompt construction shall include only selected telemetry, and the configured provider and model shall be identifiable for governance and audit purposes.	Documented provider path and minimised prompt context
NFR-14	Production Network Security	A production deployment shall use HTTPS ingress and restrict telemetry, administration and application-origin ports to trusted networks.	Demonstration and production postures explicitly separated

3.2.3 Actors and Use Cases

Three actors appear in the use-case model. The Operator is responsible for the monitored infrastructure; the InfraGuard Agent runs each heartbeat without direct input; and the configured LLM Provider supplies the external model service. Their interactions are shown in Figure 3.4.

- UC-01: View the current status and verdict (Operator → System).
- UC-02: Inspect the root cause and recommended action of a verdict (Operator → System).
- UC-03: Acknowledge a verdict as a known non-issue (Operator → System).
- UC-04: Un-acknowledge a condition so it re-opens (Operator → System).
- UC-05: Review detected threats (Operator → System).
- UC-06: Apply a CrowdSec ban for a detected threat (Operator → System).
- UC-07: Ask the runbook assistant a question (Operator → System).
- UC-08 — Consult a Local Runbook: the operator submits an incident question, the application retrieves relevant whole Markdown documents from the indexed corpus, and the selected model produces an advisory answer conditioned on the supplied text with source titles.

- UC-09: Generate a verdict on a heartbeat, consulting the LLM provider (Agent → LLM Provider).
- UC-10: Dispatch a notification on a high or critical verdict (Agent → Operator).

3.3 System Architecture

InfraGuard AI runs as two processes. The agent process executes the reasoning heartbeat, while the API process serves the dashboard and REST surface. Both share the same SQLite database. Long-running integration calls are moved to worker threads so they do not block the API event loop. Figure 3.1 shows the main components.

3.3.1 Architectural Style

The single-host architecture keeps collection, verdict generation, and memory logic separate from the Loki, Prometheus, HTTP-probe, Docker, and LLM-provider adapters. Each adapter has a small interface, which makes a source easier to omit, replace, or test independently.

3.3.2 Components

1. Agent (Heartbeat). A long-running asyncio process that, on each interval, fetches operator acknowledgements, collects telemetry, invokes the LangGraph reasoning loop, persists the verdict, and dispatches notifications. Implemented in `agent/main.py` and `agent/orchestrator.py`.
2. FastAPI Service. Hosts the dashboard and the REST API (status, alerts, configuration, threats, runbooks, and acknowledgement endpoints) behind authentication, rate-limiting, security-header, and audit middleware. Implemented in `api/main.py`.
3. Reasoning Substrate. A LangGraph state machine implements the collect-analyse-decide-notify loop with two interchangeable analysis modes: a single structured provider call and a LangChain multi-tool agent.
4. LLM Client. A provider layer supporting the Gemini Developer API, Anthropic, OpenAI-compatible endpoints and optional local Ollama models through a common verdict schema.
5. Verdict-Memory. The fingerprinting and acknowledgement subsystem (`agent/memory.py` and the acknowledgements table) records matching low-risk verdicts with a flag used by default API/dashboard views. Later prompts receive only validated canonical condition identifiers for exact reuse; operator notes are excluded, and an identifier is neither an instruction nor evidence of health. The subsystem does not bypass inference or storage.
6. RAG Service. A retrieval pipeline over local Markdown runbooks, local ONNX all-MiniLM-L6-v2 embeddings, and a ChromaDB vector store.

7. Threat Subsystem. A deterministic detector that scans recent logs for attack patterns and an operator-approved CrowdSec adapter.
8. SQLite Store. The persistence layer for verdicts and acknowledgements, with automatic retention pruning.
9. Dashboard. A single self-contained HTML, CSS, and JavaScript page served as a static asset by FastAPI and communicating over JSON.
10. External Services. The selected cloud-model API (Gemini, Anthropic, or an OpenAI-compatible endpoint), ntfy.sh, and the optional CrowdSec Local API. Ollama provides an optional local-model path.

3.3.3 Communication Patterns

During each heartbeat, the agent queries the Loki and Prometheus HTTP APIs, runs the configured HTTP probes, and may read from the Docker Engine API. Verdicts and acknowledgements are stored in the SQLite database shared with the API. The dashboard exchanges JSON with the API and uses a session cookie for authentication. HTTPS termination is left to the deployment environment.

3.3.4 High-Level Architecture

Figure 3.1 presents the high-level architecture. The scheduled agent and FastAPI service share the SQLite verdict store, while the API also uses the local ChromaDB index and a read-only mount containing twenty Markdown runbooks. The agent reads configured telemetry sources and calls the selected provider. It sends ntfy alerts for high or critical verdicts. The API calls the CrowdSec Local API only after an operator approves an IP ban; other remediation remains manual.

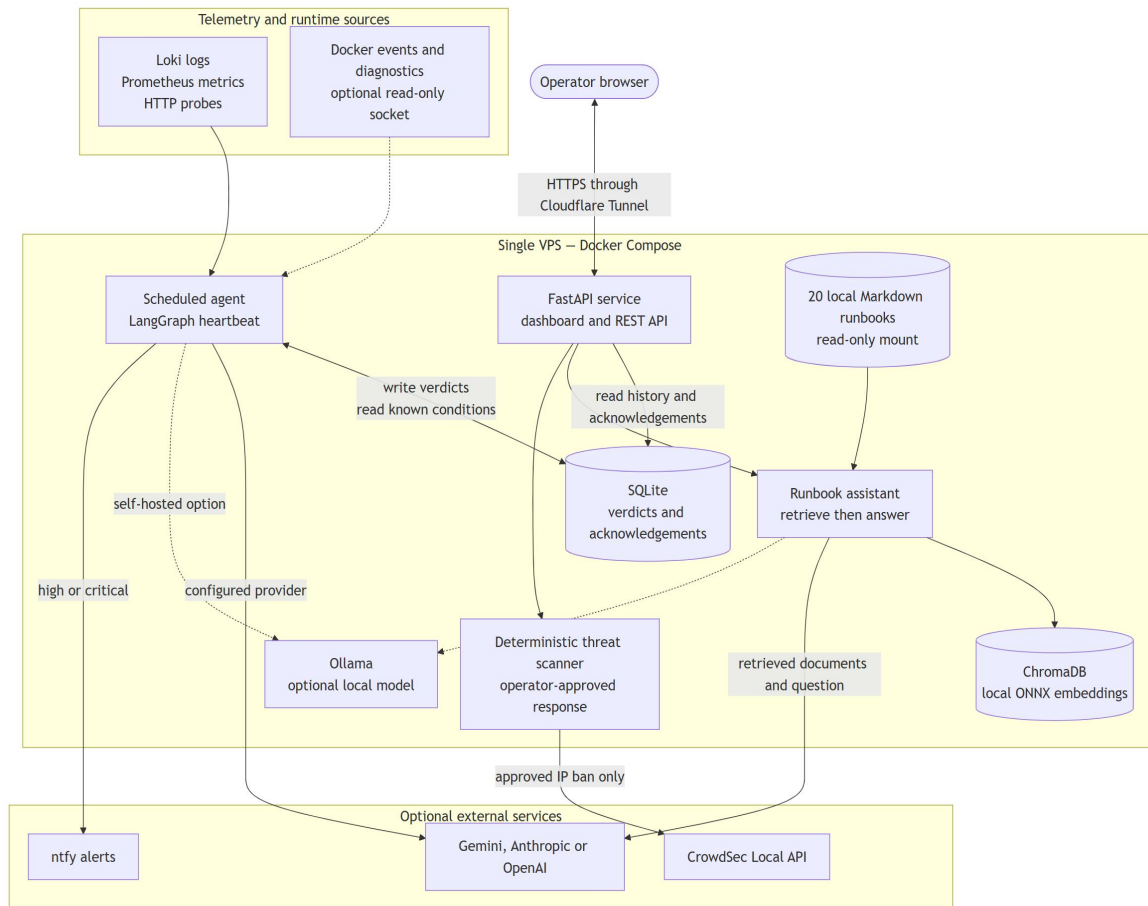


Figure 3.1: High-Level Architecture of InfraGuard AI.

3.3.5 Deployment Topology

Figure 3.2 depicts the intended single-VPS topology. DevPlanner and InfraGuard AI run as separate Docker Compose projects on the same Linux host. Cloudflare Tunnel supplies the proposed public HTTPS entry point, while InfraGuard reaches its API, Loki, Prometheus, and CrowdSec through loopback-bound origins. Runbooks and application state remain on local volumes. Only the selected cloud-model API and ntfy require outbound access; Ollama can be used for local inference. This diagram is an architecture description, not a retained deployment receipt.

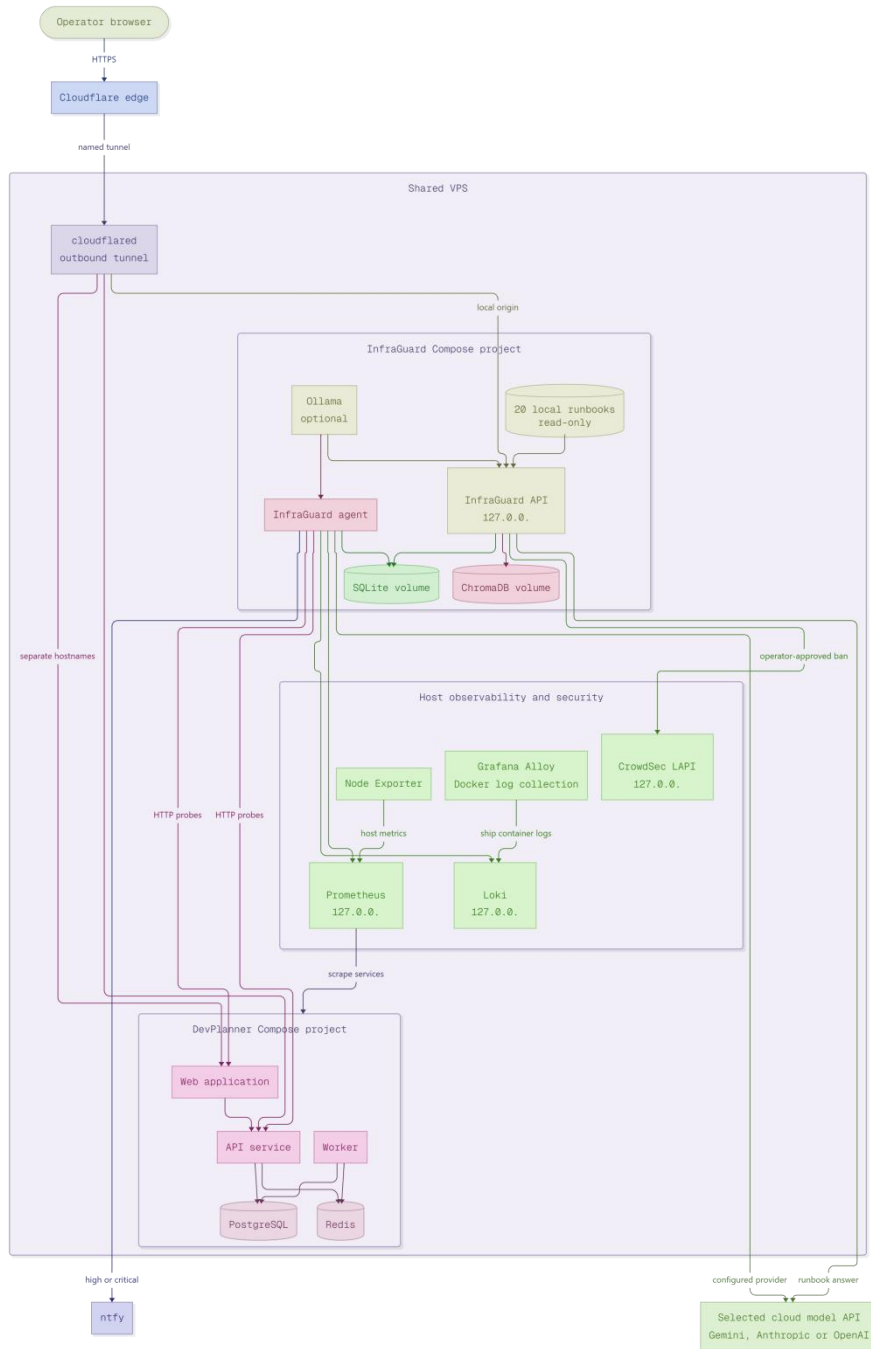


Figure 3.2: Intended Single-VPS Topology for DevPlanner and InfraGuard AI.

3.3.6 Heartbeat Verdict Pipeline

Figure 3.3 traces the heartbeat and remediation boundary. The agent loads active acknowledgements, collects configured telemetry, calls the selected model, validates the structured verdict, computes its fingerprint, and stores it. A high or critical verdict may add Docker diagnostics and trigger ntfy. Separately, deterministic log rules may identify a threat, but an IP ban is submitted to CrowdSec only after operator approval. The LLM does not restart services, delete data, change configuration, or alter the firewall.

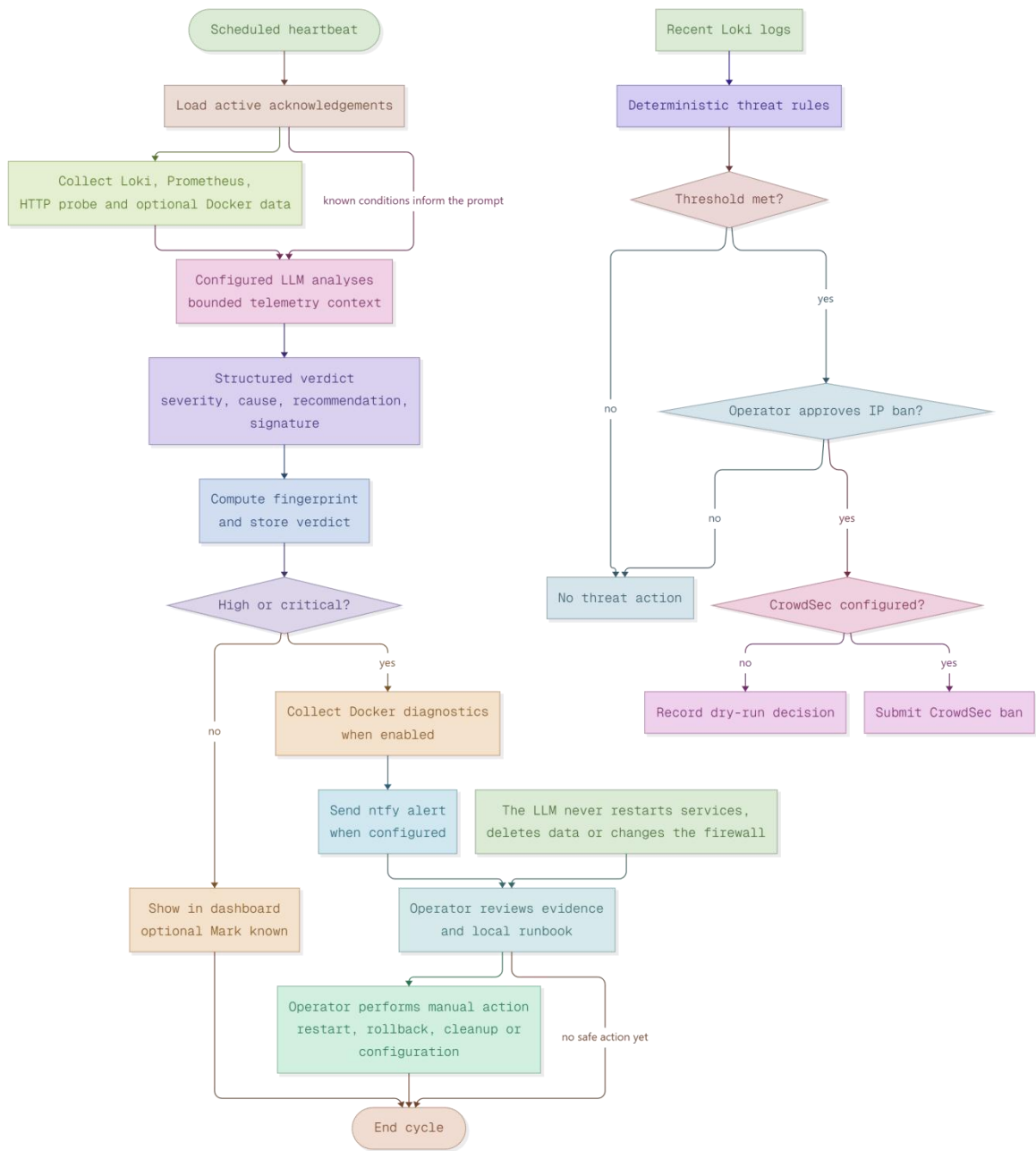


Figure 3.3: Heartbeat Pipeline and Remediation Boundary.

3.4 System Design

The design work focused on five areas: the database schema, the normal heartbeat sequence, acknowledgement matching, the reasoning state machine, and runbook indexing.

3.4.1 Database Design (Entity-Relationship Model)

Operational state is kept in two SQLite tables. ChromaDB holds the runbook embeddings, while API audit events are written to an append-only file. Figure 3.6 shows the entity-relationship model, followed by a field-level description.

Entity: verdicts

Columns: id (PK, AUTOINCREMENT), created_at (TEXT, UTC ISO-8601), severity (TEXT), summary (TEXT), payload (TEXT, JSON of the full verdict and extras), signature (TEXT), fingerprint (TEXT). Each heartbeat appends one row; rows older than the retention window are pruned on insert.

Entity: acknowledgements

Columns: fingerprint (PK, TEXT), signature (TEXT), severity (TEXT), summary (TEXT), note (TEXT), acked_by (TEXT), created_at (TEXT), expires_at (TEXT, nullable). One row per acknowledged condition; an acknowledgement is active while expires_at is null or in the future.

Verdicts and acknowledgements are matched by fingerprint. Several verdict rows may share a fingerprint when the same condition recurs, while each fingerprint can have at most one active acknowledgement. A matching verdict is marked acknowledged; only an ok or warning verdict receives a presentation-masking flag. The stored record remains available, and every later heartbeat still invokes the configured model and stores its verdict.

3.4.2 Use-Case Model

Figure 3.4 shows the use-case model. Through the dashboard, the operator reviews current and historical verdicts, marks a low-risk condition as known, queries or re-indexes local runbooks, reviews detected threats, and may approve a CrowdSec IP ban. The heartbeat scheduler collects telemetry, obtains and stores a structured verdict from the configured model, and sends a high- or critical-severity alert when configured. Service remediation remains an operator action.

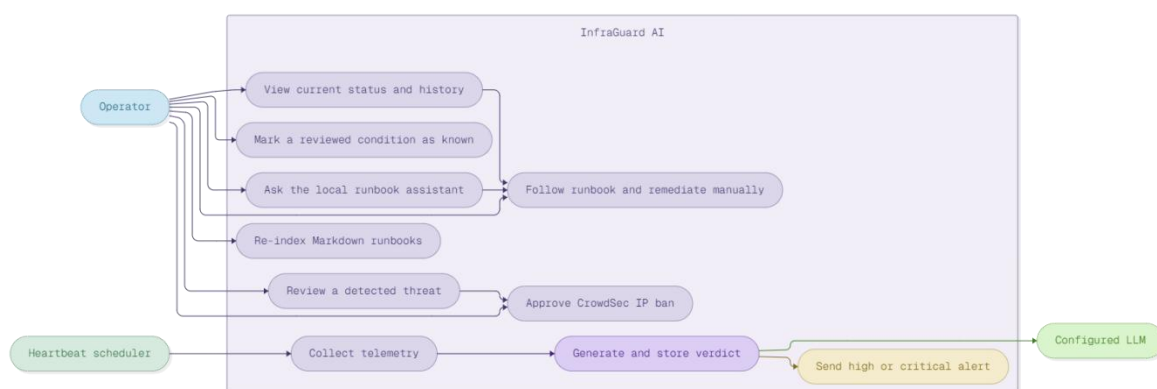


Figure 3.4: *InfraGuard AI Use-Case Model.*

3.4.3 Sequence Diagram: One Heartbeat Cycle

A heartbeat begins by loading active acknowledgements and collecting bounded telemetry from Loki, Prometheus, HTTP probes, and optional Docker sources, as shown in Figure 3.5. Only validated canonical acknowledgement identifiers are returned to the model context for exact signature reuse; notes are excluded and current telemetry alone determines severity. The returned verdict is validated, fingerprinted, and stored. When Docker monitoring is enabled, high or critical results may collect extra diagnostics; those severities can also trigger an ntfy alert. No service-changing command is executed by the heartbeat.

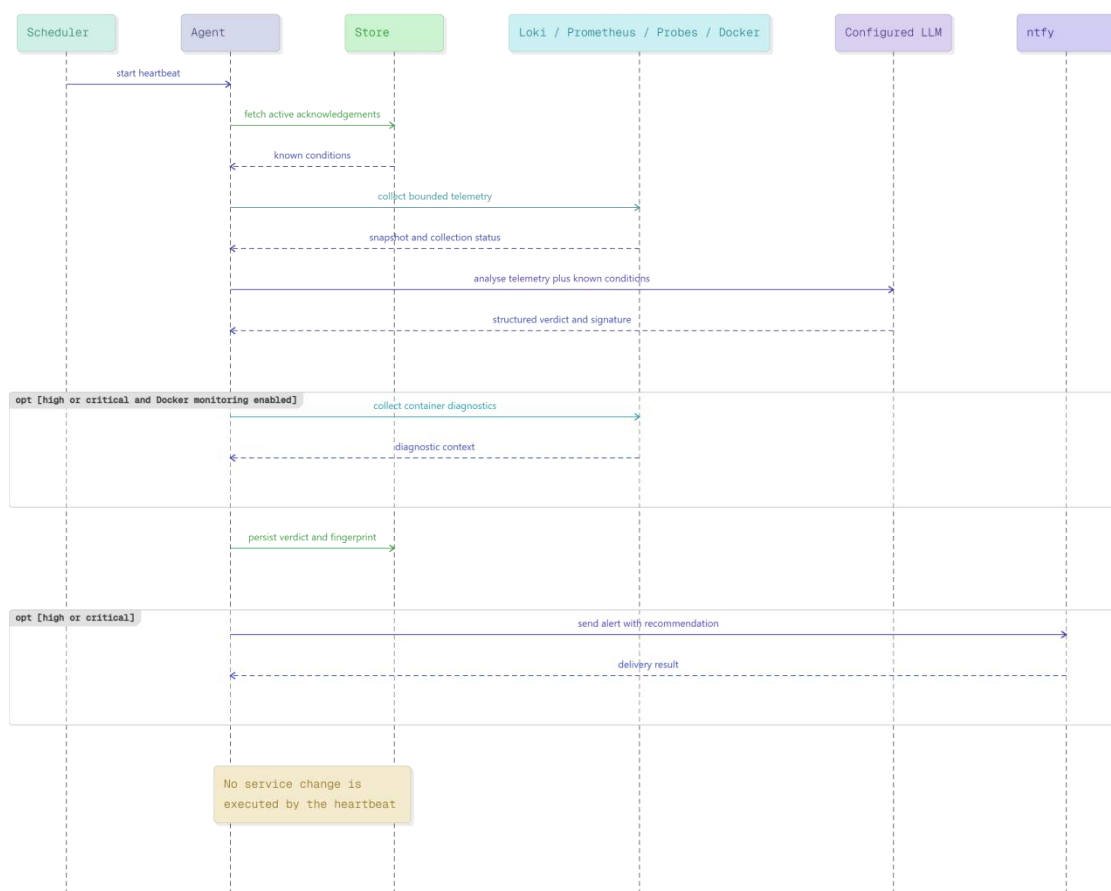


Figure 3.5: Sequence of One Heartbeat Cycle.

3.4.4 Entity-Relationship Model

Figure 3.6 visualises the two-table schema described above. Many verdict rows may carry the same fingerprint, while the acknowledgements table stores at most one row for that fingerprint together with its note, operator, creation time, and optional expiry.

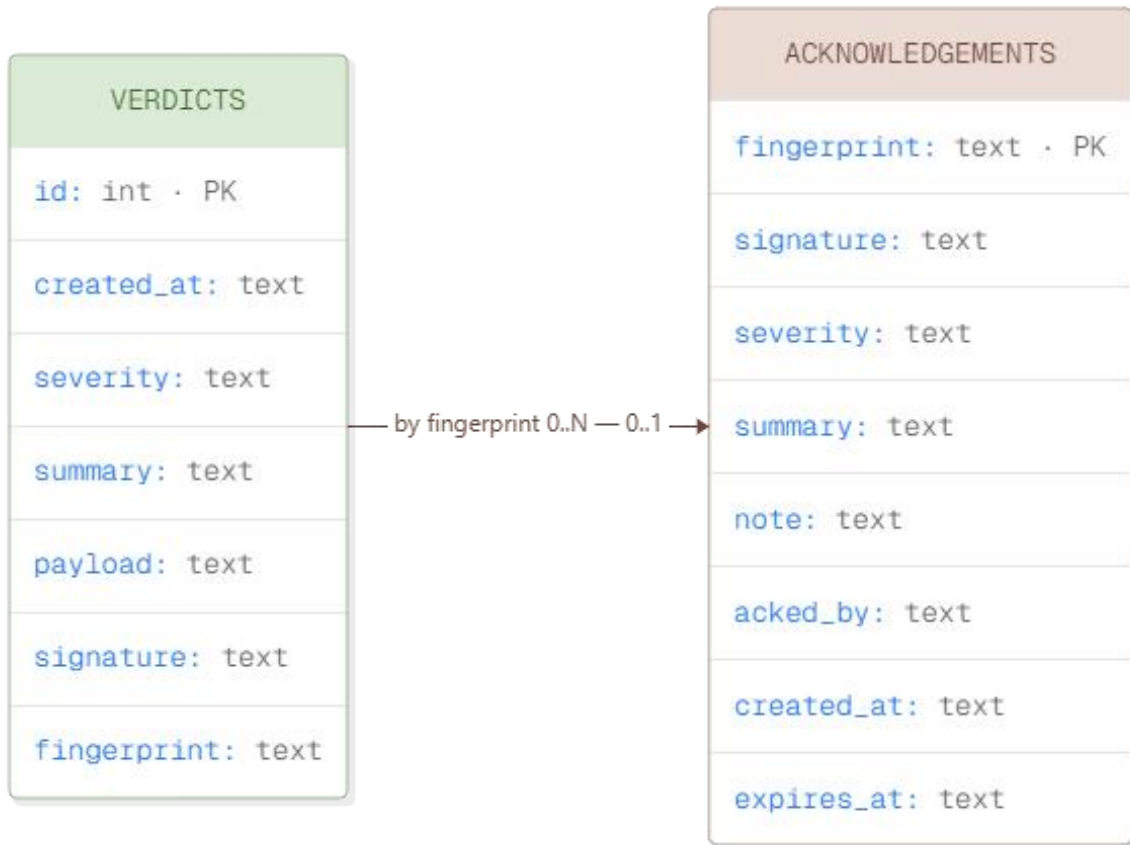


Figure 3.6: Verdict and Acknowledgement Data Model.

3.4.5 Verdict-Memory Design

The acknowledgement memory combines the condition signature, prompt version, and model in a SHA-256 fingerprint: $\text{fingerprint} = \text{SHA-256}(\text{signature} \mid \text{prompt_version} \mid \text{model})$. Under prompt-contract version 2, a model verdict must include a canonical three-part signature; a missing or malformed signature fails validation and produces the non-acknowledgeable high-severity pipeline:analysis-failed:agent sentinel. A deterministic canonical fallback exists only as a defensive persistence path for missing legacy/internal input. An operator acknowledgement stores the fingerprint with an optional note and expiry time. A changed signature, prompt version, or model produces a new fingerprint. Matching ok and warning rows may be masked from the default view; high and critical rows cannot be masked.

3.4.6 State Machine: LangGraph Reasoning Agent

Figure 3.7 depicts the LangGraph state machine that governs the reasoning agent. Four nodes, collect_data, analyze, decide_action, and notify, are used on each heartbeat. The graph begins at collect_data, proceeds to analyze and decide_action, and routes high- and critical-severity verdicts to notify. Other verdicts end without notification. When local Docker monitoring is

enabled, `decide_action` can also gather container diagnostics for high- and critical-severity verdicts.

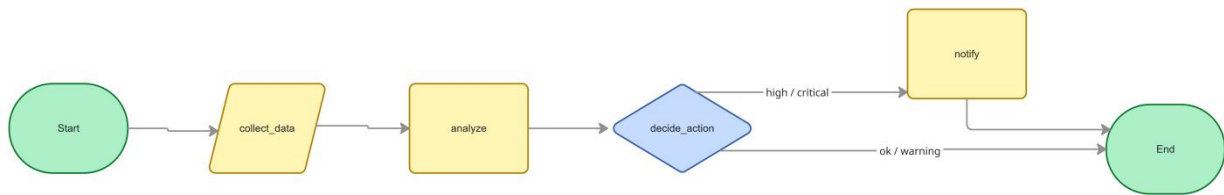


Figure 3.7: LangGraph Agent State Machine.

3.4.7 Workflow: Runbook RAG Indexing

Runbook indexing starts when the operator calls `POST /api/runbooks/index`. The loader recursively reads approved Markdown files from the configured local directory and retains title, source, and category metadata. Each whole file is embedded locally with the ONNX all-MiniLM-L6-v2 model and stored as one ChromaDB document; no chunk splitting is implemented. For a query, up to four closest whole documents and the operator question are serialised as untrusted data inside explicit boundaries. The API returns an advisory answer conditioned on supplied runbook text together with source titles. The operator reviews and executes any selected step manually.

3.5 Tools and Technologies Used

The implementation stack is grouped below by layer. Table 3.5 lists the telemetry sources and their collection modules.

3.5.1 Backend and Orchestration

- Language and runtime: Python 3.11+ with asyncio.
- Web/API framework: FastAPI with Uvicorn.
- LLM orchestration: LangGraph and LangChain.
- LLM providers: Gemini Developer API, Anthropic, OpenAI-compatible endpoints, or optional local Ollama, selected by configuration.
- Vector store and embeddings: ChromaDB with local ONNX all-MiniLM-L6-v2 embeddings.
- Persistence: SQLite via aiosqlite.
- HTTP client: httpx; runbook source: local Markdown files mounted read-only in the containers.

3.5.2 Security and Deployment

- Sessions: itsdangerous signed cookies; rate limiting: slowapi; security headers and CSP via custom middleware.
- Threat response: CrowdSec Local API.
- Notifications: ntfy.sh.
- Packaging and deployment: Docker Compose on a shared Linux VPS, Cloudflare Tunnel for HTTPS ingress, and GitHub Actions for CI.
- Frontend: a single self-contained HTML/CSS/JavaScript dashboard served by FastAPI.

3.5.3 Telemetry Sources

Table 3.5 enumerates the telemetry sources and the modules that collect them.

Table 3.5: Telemetry Sources and Collection Tools.

Source	Collection Module	Signal	Notes
Logs	agent/tools/loki.py	Recent log lines	Loki query_range, newest-first, configurable window
Metrics	agent/tools/prometheus.py	CPU, memory, disk, HTTP 5xx ratio	Prometheus instant queries
Endpoint health	agent/tools/http_probe.py	Status code and latency	Comma-separated PROBE_URLS
Container state	agent/tools/docker_api.py, docker_events.py	Events, stats, health, logs	Optional (ENABLE_DOCKER_MONITORING=1)
Threats	agent/tools/threat_response.py	Brute-force / port-scan patterns	Deterministic regex over recent logs
Local runbooks	agent/rag/local_runbooks_loader.py	Read-only Markdown corpus mounted into the containers.	Local Markdown source; no external documentation service

3.5.4 Design Trade-offs and Alternatives

The implementation choices favour inspectability and a small single-host footprint. Table 3.6 records the principal alternatives and the boundary introduced by each choice.

Table 3.6: Principal Design Choices, Alternatives, and Trade-offs.

Choice	Alternative	Reason Selected	Trade-off
SQLite	PostgreSQL	Zero external database service and simple shared local state.	Single-host concurrency and horizontal scaling are limited.
ChromaDB	Managed vector database	Local persistence and a compact Python integration.	Retrieval scaling and operational redundancy were not evaluated.
all-MiniLM-L6-v2 ONNX	Cloud embeddings or a larger local model	Local embedding avoids exporting the runbook corpus and reduces deployment cost.	Retrieval quality may be lower than larger models and requires benchmarking.
Top-four retrieval	Dynamic or reranked retrieval	Small prompt footprint and deterministic context size.	The value four was an engineering default, not an empirically optimised setting.
Separate agent and API processes	Single process or distributed workers	A blocking heartbeat cannot stall dashboard request handling.	Both processes still share one host and SQLite store.
Signature + prompt version + model fingerprint	Signature-only matching	A reasoning-rule or model change forces renewed review.	Every heartbeat still incurs inference and storage work.
Deterministic threat detection	LLM-based threat classification	Thresholds and matching rules remain inspectable before an operator-approved ban.	Coverage is limited to the implemented attack patterns.

CHAPTER FOUR: SYSTEM IMPLEMENTATION

4.1 Development Environment

Development and validation used a workstation configured to approximate the intended single-host deployment.

4.1.1 Hardware and Operating System

Primary development was conducted on a Windows 11 workstation running Docker Desktop. All services run in Linux containers based on the `python:3.11-slim` image. The intended operational environment is a single Linux VPS running native Docker, while the repository's Terraform definitions provide an alternative Google Compute Engine path. This environment description is not a retained deployment receipt.

4.1.2 Source Control and Continuous Integration

The repository defines two GitHub Actions workflows. `test.yml` runs the `pytest` suite for pull requests to the main branch. `deploy.yml` repeats the tests after a push to main, then builds the agent and API images, publishes them to Google Artifact Registry, and updates the target virtual machine. Because these steps share one job, deployment proceeds only if the tests pass.

4.1.3 Editor and Tooling

Development was performed in Visual Studio Code with Python and Docker tooling. Application dependencies are listed in `requirements.txt` and installed into each container at build time.

4.2 Implementation Details

This section maps the implemented design decisions to repository evidence. File paths are retained for reproducibility, while the surrounding explanation emphasises rationale, trade-offs, failure boundaries and what was not evaluated.

4.2.1 Repository Layout

Figure 4.1 and Table 4.1 provide the reproducibility map. The important architectural choice is separation of the scheduled agent, API, adapters, reasoning substrate and persistence boundaries so that failures can be recorded and individual integrations can be omitted or tested. The single-host design reduces deployment complexity but shares one host, store and network failure domain.

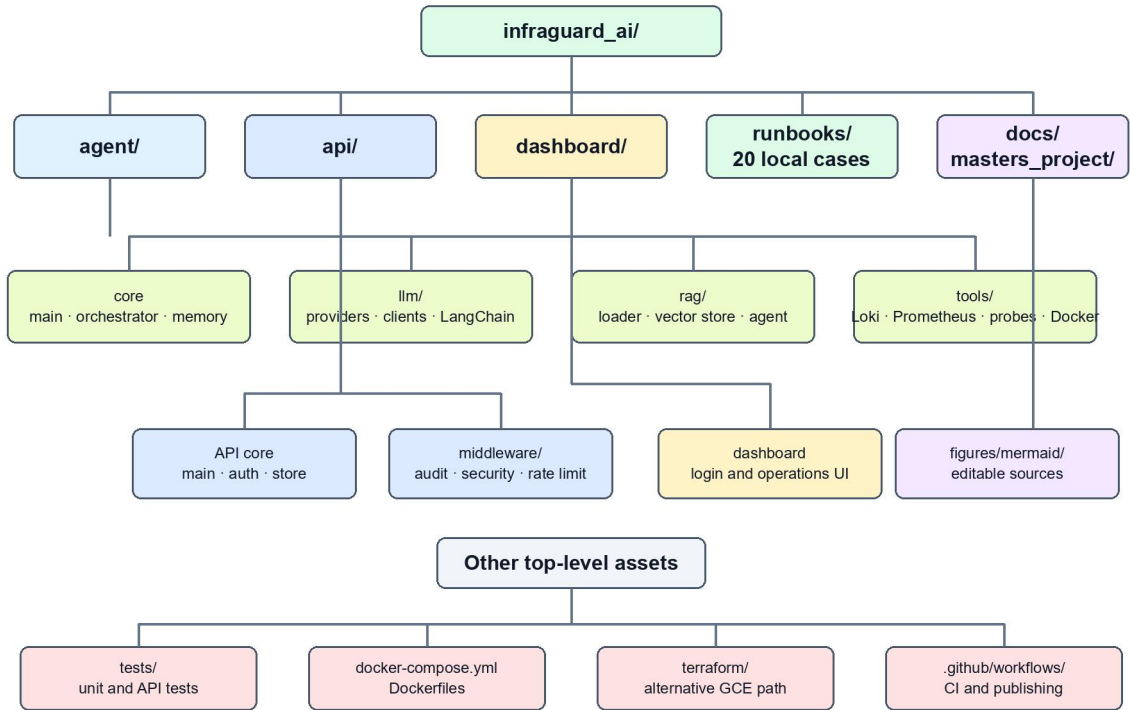


Figure 4.1: Repository Layout of the Reference Implementation.

Table 4.1: Repository Module Map and Responsibilities.

Path	Responsibility
agent/main.py	Heartbeat entry point; fetches acknowledgements, runs the cycle, persists verdicts, dispatches notifications.
agent/orchestrator.py	The LangGraph state machine wiring collect_data, analyze, decide_action, and notify.
agent/memory.py	Verdict fingerprinting (signature + ruleset version) and deterministic signature fallback.
agent/llm/providers.py	Gemini, Anthropic, OpenAI-compatible and Ollama provider dispatch
agent/llm/schema.py	Prompt-contract v2: exact five-field raw-JSON validation, canonical signatures, bounded strings, duplicate/control rejection and severity/signature consistency.
agent/llm/gemini_client.py	Gemini Developer API client
agent/llm/anthropic_client.py	Single-call Anthropic Claude client.
agent/llm/openai_client.py; agent/llm/ollama_client.py	OpenAI-compatible and locally hosted Ollama clients
agent/llm/langchain_agent.py	LangChain multi-tool agent (ReAct-style) over the telemetry tools.
agent/llm/prompts.py	Prompt assembly, evidence constraints, and the known-conditions memory block.
agent/rag/local_runbooks_loader.py	Loads Markdown runbooks and attaches category metadata
agent/rag/vector_store.py	ChromaDB binding: index, load, and similarity search.
agent/rag/runbook_agent.py	Retrieval-augmented runbook question answering.

Path	Responsibility
agent/tools/loki.py, prometheus.py, http_probe.py	Telemetry collection tools (logs, metrics, endpoint health).
agent/tools/docker_api.py, docker_events.py, docker_logs.py	Optional Docker Engine telemetry (events, diagnostics, log tail).
agent/tools/threat_response.py	Deterministic threat detection and the CrowdSec adapter.
agent/tools/notify.py, langchain_tools.py	ntfy.sh notifications; @tool wrappers for the multi-tool agent.
api/main.py	FastAPI application: routes, middleware registration, auth, and the acknowledgement and threat endpoints.
api/auth.py	Signed session-token creation and validation.
api/store.py	SQLite persistence for verdicts and acknowledgements, with retention pruning.
api/middleware/audit.py, security.py, rate_limit.py	Request auditing, security headers/CSP, and rate limiting.
dashboard/index.html, login.html	The single-page operations dashboard and the login page.
tests/	Source-scoped pytest suite, including LLM-security, prompt-boundary, memory, store, API, RAG, threat-response, telemetry and orchestration cases.
terraform/	Google Compute Engine infrastructure-as-code.
docker-compose.yml, agent/Dockerfile, api/Dockerfile	Container composition and image definitions.
.github/workflows/test.yml, deploy.yml	CI test workflow and CD deployment workflow.

4.2.2 The Reasoning Agent and LangGraph Loop

The reasoning workflow lives in `agent/orchestrator.py` and is started on a schedule by the `asyncio` heartbeat in `agent/main.py`. At each interval, the heartbeat loads active acknowledgements and, when Docker monitoring is enabled, may tail error logs from a monitored container. The compiled LangGraph workflow runs on a worker thread so a blocking model call cannot stall the event loop. Its `collect_data` node gathers telemetry, analyze requests a verdict, and `decide_action` may add container diagnostics for a high or critical result. A conditional edge routes those two severities to notify; all other verdicts end the cycle.

4.2.3 The Two LLM Reasoning Modes

Two settings control LLM analysis: `USE_LANGCHAIN_AGENT` selects the analysis mode, and `LLM_PROVIDER` selects Gemini, Anthropic, an OpenAI-compatible endpoint, or Ollama. The active model name is included in the acknowledgement fingerprint. Direct mode sends the assembled prompt in one request and parses the returned JSON through the shared schema module. In multi-tool mode, the provider chat model may call the Loki, Prometheus, and HTTP-probe tools before returning a verdict, subject to an iteration limit. Both paths produce the same normalised fields.

4.2.4 Prompt Construction and Response Validation

Prompt construction in `agent/llm/prompts.py` serialises each available telemetry source as untrusted JSON data inside explicit boundaries and instructs the model not to treat an absent optional integration as an incident. Prompt-contract version 2 accepts only one raw JSON object with exactly five non-empty string fields: `severity`, `summary`, `root_cause`, `recommended_action`, and `signature`. It rejects fences, surrounding prose, duplicate, missing or extra fields, disallowed control characters, overlong values, unsupported severities, non-canonical signatures and severity/signature inconsistencies.

4.2.5 The Verdict-Memory

Acknowledgement behaviour spans `agent/memory.py`, `api/store.py`, the prompt assembler, and the acknowledgement endpoints; Figure 4.2 traces the flow. A condition signature, prompt version and model name form the SHA-256 fingerprint. An operator may acknowledge an ok or warning verdict, add a note, and choose an expiry time. Later prompts receive only validated canonical identifiers: notes are omitted, the identifier supports exact reuse only, and it never authorises a severity downgrade. Every heartbeat still invokes the model and stores the verdict. A matching low-risk row may be masked from the default view; changed identities and all high, critical or pipeline-failure rows remain visible.

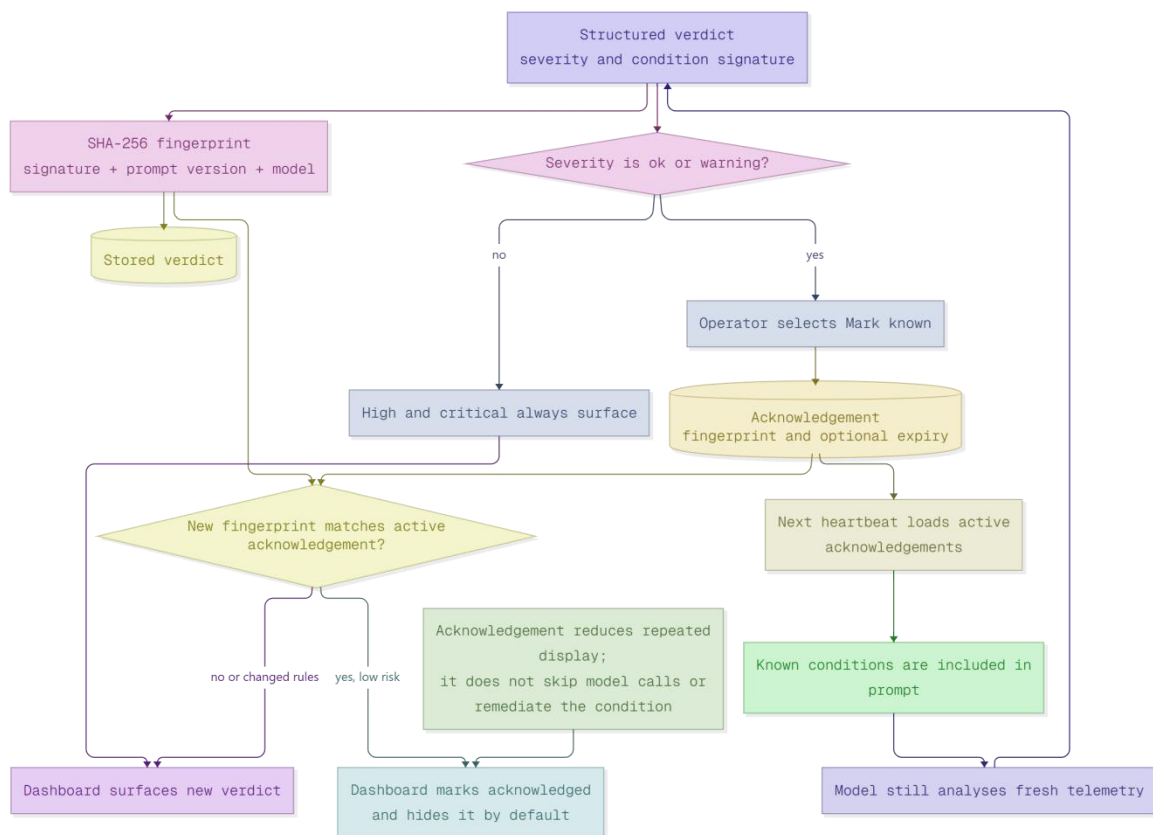


Figure 4.2: Verdict-Memory Fingerprint and Acknowledgement Flow.

4.2.6 Telemetry Collection Tools

Each telemetry adapter returns structured data or a structured error. The Loki adapter in `agent/tools/loki.py` uses `query_range` for a fixed recent window, orders results newest first, and enforces a configurable limit. Prometheus instant queries collect CPU, memory, disk, and HTTP 5xx-rate signals. HTTP probes record the status and latency of each configured endpoint. Docker events, diagnostics, and error logs are collected only when Docker monitoring is enabled.

4.2.7 The RAG Runbook Service

The runbook service in `agent/rag/` recursively loads approved Markdown files from `RUNBOOKS_DIR` while retaining title, source, and category metadata. Each file remains one whole retrievable document, embedded locally with ONNX all-MiniLM-L6-v2 and stored in ChromaDB; the current code does not create smaller chunks. At query time, up to four closest whole documents and the question are serialised as untrusted data and supplied to the configured LLM. The answer is conditioned on that text and includes source titles. Operators trigger re-indexing, the source mount is read-only, and the application executes no runbook command.

4.2.8 Threat Detection and CrowdSec Response

Threat detection in `agent/tools/threat_response.py` does not use an LLM. It counts repeated HTTP 401/403 responses, SSH authentication failures, and rapid connection attempts by source IP. When an operator requests a ban, the API validates the type and address, freshly re-fetches Loki logs, reruns detection, and accepts only an exact server-derived type/IP match. Stale or fabricated selections return HTTP 409; client-supplied counts and descriptions are ignored. A matched decision is then sent to CrowdSec, or logged as a dry run when CrowdSec is unconfigured.

4.2.9 The API Service and Middleware

The FastAPI application in `api/main.py` serves both the dashboard and the JSON REST API. Four middlewares are registered: audit logging, security headers, per-IP rate limiting, and authentication. The authentication layer requires a valid signed session cookie except on the public health and login endpoints, and the login route has its own stricter rate limit. Runbook queries, indexing, and Loki-backed threat scans run on worker threads so their blocking integration calls do not stall the event loop.

4.2.10 Persistence and Retention

SQLite persistence is implemented in `api/store.py` with `aiosqlite` and uses the verdicts and acknowledgements tables described in Section 3.4.1. The schema is created and migrated

idempotently whenever a connection opens, allowing an existing database to gain the signature and fingerprint columns in place. Each insert also prunes verdicts older than the configured retention window, so no separate maintenance job is needed to bound storage growth.

4.2.11 The Dashboard

The dashboard (dashboard/index.html) is a single page served by FastAPI. It displays the current verdict, integration status, agent staleness, detected threats, runbook queries, and recent checks. Root-cause and recommended-action text can be expanded when needed.

For ok and warning verdicts, the page provides controls to mark or unmark a condition as known, a known tag, and a toggle for acknowledged rows. Text from the server, model, or logs passes through a single HTML-escaping helper before it is inserted into the page. This limits the cross-site-scripting risk posed by untrusted log or model content. Figure 4.3 shows the dashboard.

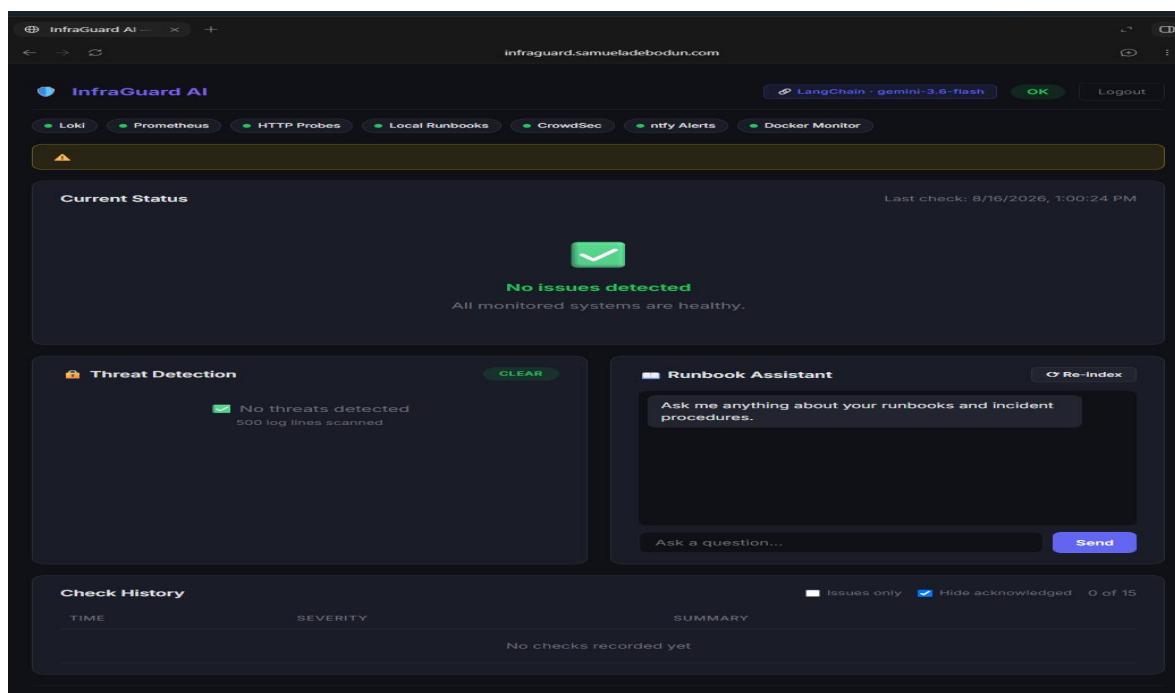


Figure 4.3: InfraGuard AI Operations Dashboard.

The dashboard runbook panel exposes the retrieval workflow described in Section 4.2.7. Figure 4.4 summarises whole-document local indexing, top-four document retrieval, an answer conditioned on supplied runbook text with source titles, and the human-review boundary.

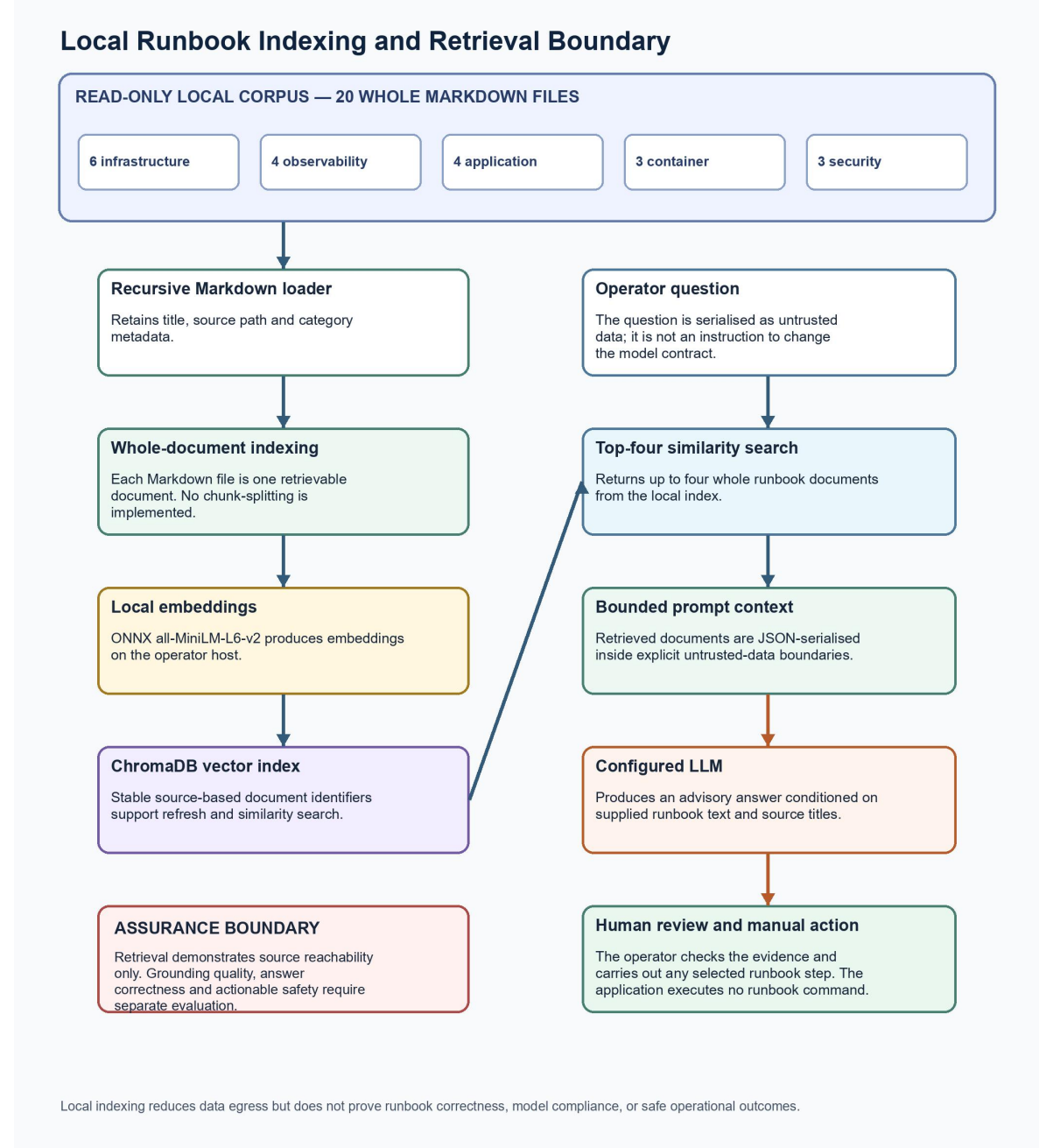


Figure 4.4: Local Runbook Indexing and Retrieval Process.

4.2.12 API Endpoints

Table 4.2: API Endpoint Reference.

Endpoint	Method	Purpose
/	GET	Serve the dashboard (authenticated).

Endpoint	Method	Purpose
/login, /logout	GET/POST	Session authentication; login is rate-limited to five attempts per minute.
/health	GET	Liveness check (public).
/status	GET	Latest verdict with staleness and memory (fingerprint, acknowledged, suppressed) fields.
/alerts	GET	Recent verdict history with per-row memory fields.
/api/config	GET	Which integrations are configured (booleans only).
/api/agent/mode	GET	Active reasoning mode and model.
/api/threats	GET	Scan recent Loki logs for brute-force and port-scan patterns.
/api/threats/apply	POST	Validate a type/IP selection, re-fetch and re-detect current Loki evidence, then apply only an exact server-derived CrowdSec decision; stale or fabricated selections return 409.
/api/verdicts/ack	POST	Acknowledge a verdict condition as a known non-issue.
/api/verdicts/unack	POST	Remove an acknowledgement so the condition re-opens.
/api/acks	GET	List the active acknowledgements.
/api/runbooks/query	POST	Answer a question over the indexed runbooks (RAG).
/api/runbooks/index	POST	Clear the derived index and rebuild it from mounted local Markdown files.

4.3 Security, Performance, and Scalability Considerations

4.3.1 Authentication and Session Management

All non-public endpoints require a session cookie signed with an `itsdangerous` timestamped serialiser and an environment-provided secret. Sessions expire after twenty-four hours and the authentication middleware checks the cookie on every request. Credential comparison uses `hmac.compare_digest`. The cookie is `HttpOnly`; its `Secure` attribute is enabled only when `SESSION_COOKIE_SECURE=1`, so deployment configuration must set that flag whenever HTTPS is used.

4.3.2 Rate Limiting and Output Hygiene

The `slowapi` middleware applies a general request limit and a stricter login limit. Dashboard rendering uses a shared escaping helper for server-derived text. The response middleware adds a content-security policy and other headers. The current policy permits inline script and style content because the dashboard is a single HTML file; this weakens the policy and should be removed in a hardened deployment by moving scripts and styles to separate files.

Browser escaping constrains DOM injection but does not make telemetry or retrieved runbooks safe model instructions. Malicious log text or runbook content could attempt prompt injection, distort a verdict or encourage unsafe action. Model text is not executed directly and a CrowdSec ban requires operator approval, but no retained adversarial prompt-injection evaluation or independent penetration test was found.

4.3.3 Auditability

The audit middleware writes one structured JSON line per request with timestamp, client IP, method, route, user, status, and elapsed time. The file is a local operational log. It is not cryptographically protected or tamper-evident, so external log shipping and retention controls are recommended for production use.

4.3.4 Performance and Responsiveness

The agent and API run as separate processes. API routes that call blocking LLM, runbook-indexing, or Loki clients dispatch that work to a worker thread. The default heartbeat is 120 seconds, and one model analysis is performed per heartbeat. This design bounds request frequency. The implemented heartbeat path was not performance-profiled; the controlled qwen2.5:3b benchmark separately recorded response latency, token use and the local metered-cost boundary.

4.3.5 Scalability and Degradation

The design targets one host and its co-located workloads; multi-host operation is future work. The API, scheduled agent and SQLite store share a failure domain, and no high-availability, failover or horizontal-scaling mechanism was implemented. Concurrent incident behaviour and workloads spanning 100 or 1,000 hosts were not measured. Each telemetry tool converts a failed request into a structured error so a cycle can continue with less evidence, while a provider or parser failure records the high-severity pipeline:analysis-failed:agent sentinel for operator review. These are degradation paths, not evidence of resilience under sustained outage.

The read-only Docker socket remains a powerful host interface, and ntfy notifications can send incident text to a public service endpoint. Production deployment requires a narrower integration boundary, confidential notification routing, capacity tests, backup and recovery exercises, and retained rollout and rollback evidence.

4.3.6 Deployment Boundary

WARNING - PRODUCTION SECURITY BOUNDARY. The supplied Terraform now creates no ingress firewall rules by default. HTTPS port 443 is opt-in only through explicit source CIDR ranges; port 80 is optional for redirect-only use; ports 8080, 3100 and 9090 require trusted management CIDRs and reject world-open ranges. Docker Compose binds the API to 127.0.0.1:8080. These remediations remove the earlier development exposure, but the project does not bundle a TLS terminator or reverse proxy, or managed-secret integration. Operational deployment therefore still requires HTTPS ingress, private telemetry and administration, managed secret stores, hardened browser policy, backup and recovery controls, and security verification as summarised in Table 4.3.

Table 4.3: Security and Privacy Controls by Evidence Classification.

Area	Implemented or demonstrated state	Control and evidence class	Organisational, legal or deployment obligation
Ingress	No Terraform ingress by default; opt-in CIDRs for 443; loopback Compose binding	Implemented; code/configuration reviewed; not independently assessed	Retain a hardened HTTPS terminator, private origins and a deployment/penetration-test receipt.
Telemetry ports	Management ports disabled unless trusted CIDRs are supplied; world-open ranges rejected	Implemented configuration control; no live network assessment	Keep telemetry and administration private and verify the effective cloud firewall.
Secrets	Environment-provided credentials; no bundled managed-secret integration	Production control recommended, not implemented	Use least-privilege identities, managed secrets, rotation and auditable access.
Browser and request controls	Authentication, rate limits, escaping and security headers; CSP still permits unsafe-inline	Implemented and functionally tested; no independent security assessment	Remove inline assets, add deployment-appropriate CSRF/HSTS controls and conduct penetration testing.
Audit records	Structured local JSON request log without tamper evidence	Implemented; functional path tested; integrity not assessed	Forward to a protected external sink with retention, access and tamper-evidence controls.
Cloud LLM data path	Selected prompt context; provider/model traceability; local validation; optional local-model path	Technical controls present and partly software-tested; provider behaviour and legal compliance not assessed	Approve providers; minimise/redact fields; define lawful basis, processor terms, retention, DPIA and transfer safeguards; retain evidence.
Privileged integrations and notifications	Docker socket access; operator-mediated CrowdSec; incident text may be sent to ntfy.sh	Functional controls, fresh server-side detection recheck and invalid/special-address rejection; no complete protected-CIDR policy, live adversarial or rollback assessment	Remove or proxy the Docker socket; define protected deployment/admin CIDRs; use confidential notification transport; verify least privilege, rollback and effective network policy.

4.3.7 Data Governance and Cloud-LLM Boundary

Operational telemetry may contain account identifiers, source IP addresses, request paths, device identifiers, secrets or free text. Figure 4.5 distinguishes what remains on the operator host from what may cross to a cloud LLM. The cloud path can include selected telemetry, retrieved runbook excerpts, the question, prompt and provider/model metadata; the raw response returns for local validation, persistence and authenticated review. The Ollama path can keep model inference local.

Technical controls include prompt minimisation, local validation, access restrictions and retention settings. Organisational controls include approved providers, role assignment, incident procedures and review of what may enter prompts. Legal obligations include a lawful basis, controller/processor terms, a data privacy impact assessment where required and a documented transfer basis under the Nigeria Data Protection Act. Evidence of compliance requires retained assessments, contracts, configurations and operational records; application features alone are not such evidence (Federal Republic of Nigeria, 2023).

Provider-side processing, diagnostic logging, retention, residency and training use depend on the selected service, account configuration and contract. No provider-side deletion test, legal assessment, compliance certification or cross-border-transfer verification was completed.

Figure 4.5 therefore marks cloud exposure as a deployment decision and evidence obligation, not as a property proved by local hosting.

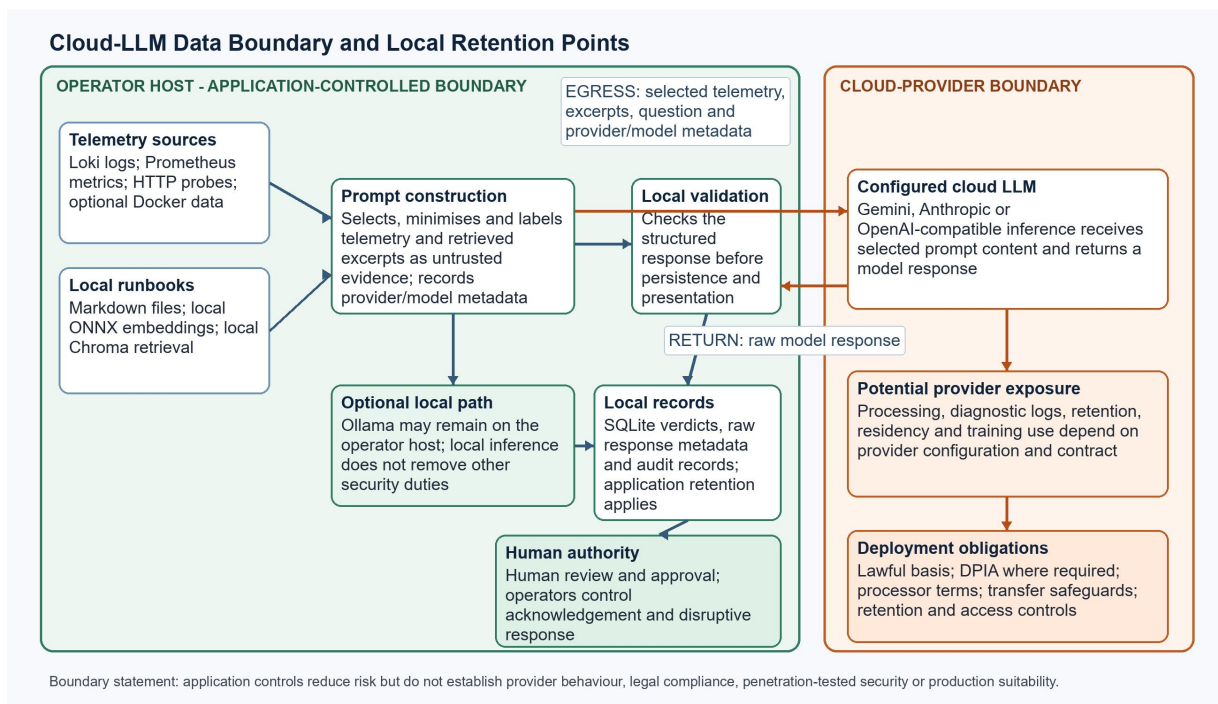


Figure 4.5: Cloud-LLM Data Boundary and Local Retention Points.

Figure 4.5 shows the application boundary rather than a guarantee of provider behaviour. It also records that local storage may retain raw response metadata and that optional ntfy notifications can create a separate external data path. Deployment review must therefore cover every configured egress, not only the LLM.

4.3.8 Security Threats Specific to LLM-Assisted Incident Triage

The LLM attack surface includes untrusted evidence, generated recommendations, persistent memory, cloud egress and model supply chains. The controls below reduce exposure but have only software-test evidence; no retained live-model red-team run, penetration test or independent security assessment was completed.

Direct prompt injection through logs and telemetry. Malicious text can instruct a model to ignore its rules or disclose data. Post-review prompt construction labels telemetry as untrusted evidence and tests the instruction/data boundary, but model independence from injected text is not empirically established.

Indirect injection through runbooks and tool output. A poisoned Markdown file or collector response can enter the prompt as retrieved evidence. Read-only mounting and fixed collectors constrain modification and tool choice, but no approved-content manifest, poisoning experiment or trust-scored retrieval is retained.

Data exfiltration through cloud prompts. Telemetry and retrieved excerpts may contain personal data or secrets. Minimisation and the local Ollama option reduce exposure; field-level DLP, provider retention verification and contractual assurance remain deployment obligations.

Model-output manipulation and unsafe action. A model can understate severity, add unsupported fields or recommend destructive commands. Post-review schema validation rejects incomplete or non-canonical verdicts, while recommendations remain advisory; actionable-safety scoring and live adversarial evaluation were not completed.

Poisoned telemetry and source authenticity. Fabricated metrics, logs or container events can distort the evidence base. The prototype records source-specific blocks but does not cryptographically authenticate upstream telemetry or establish that collected data is truthful.

Privilege escalation through integrations. The application has no generic command-execution tool and a CrowdSec ban requires authenticated operator action. Post-review validation rejects invalid, loopback, link-local, multicast and unspecified targets and rechecks a fresh server-side detection before applying a decision. A complete protected-CIDR policy is not implemented; the Docker socket remains a high-privilege interface and live rollback testing is absent.

Acknowledgement poisoning and replay. Model-generated signatures or free-text notes can corrupt future state. Post-review prompt assembly excludes operator notes and validates signature form; prompt/model binding, expiry and the high/critical safeguard remain. The 41.7% repeat-stability result means model-generated identity is still insufficiently stable for dependable acknowledgement semantics.

Model and software supply-chain drift. Mutable models, images, dependencies or provider behaviour can change risk without an application-code change. The benchmark records its model digest and source hashes, but dependency locking, signed images, SBOM evidence and equivalent provider evaluation remain future work.

Assurance boundary. Unit and integration tests can verify that these application controls are present and that selected malformed inputs are rejected. They do not prove robustness against an adaptive attacker, provider-side confidentiality, safe model behaviour or production security.

4.4 Challenges Encountered

4.4.1 LLM Output Conformance

A provider response may contain Markdown fences, surrounding prose, duplicate fields or an invalid contract. Prompt-contract version 2 fails closed: the shared parser accepts only one bounded raw JSON object with exactly the five required fields and a canonical severity/signature pairing. It rejects rather than extracts fenced or conversational JSON. Any provider or parser failure produces a high pipeline:analysis-failed:agent verdict that cannot be acknowledged.

4.4.2 Scope Bounding and False Alarms

The prompt contains explicit scope rules for optional integrations because absent evidence can otherwise be interpreted as a failure. Automated tests confirm that prompt construction includes only configured sections and the required instruction. The controlled qwen2.5:3b benchmark recorded zero healthy false positives, but production false-positive behaviour was not measured.

4.4.3 Signature Stability for the Memory

Signature stability is a central design finding rather than a minor implementation issue. The current identity may be supplied by the same model whose verdict is being remembered, creating circularity: an unstable or incorrect signature can split one condition across fingerprints or merge different conditions. Although the prompt requests a structured signature and fallback normalisation removes volatile tokens, only 10 of 24 benchmark scenarios (41.7%) returned one identical signature across all three runs. Prompt/model

binding, expiry and the high/critical safeguard limit consequences but do not establish reliable identity. Deterministic telemetry-derived condition identity before inference remains future work.

4.4.4 Security Defects Corrected During Review

Implementation review identified two defects that were corrected: the rate limiter was configured without its middleware, and server-derived text was inserted into the dashboard without a shared escaping path. Regression tests cover the corrected behaviour. The remaining deployment and CSP limitations are recorded in Section 4.3.

4.4.5 Loki Query Semantics

An early implementation used Loki's instant-query endpoint and produced inconsistent recent-log results. The revised adapter uses a range query over an explicit window in newest-first order. This also gives the threat scanner enough log lines to evaluate its configured thresholds.

CHAPTER FIVE: TESTING, RESULTS AND EVALUATION

Verification asks whether the artefact was built and behaves as specified; validation asks whether it solves the intended operational problem. This chapter therefore separates software verification, controlled model and retrieval evaluation, exploratory qualitative practitioner interpretation, descriptive perceived-usability evaluation, and security/governance analysis.

Automatically derived benchmark measures are reproducible from retained response rows. By contrast, the practitioner role reports contain aggregate human scores but the research package lacks the assessor-by-item records required to recreate them. Those means, rankings, agreement percentages and kappas are excluded from the primary evidence and research-question answers.

The reports are used only to identify qualitative engineering hypotheses. Separately, the SUS workbook retains anonymised item responses and formula-driven calculations for ten complete submissions. It is reported descriptively because the convenience sample and absence of task timing and error records preclude population or task-performance claims. Unless a section states otherwise, all findings are configuration-specific and no causal, production or population inference is intended.

5.1 Testing Strategy

Evaluation evidence is classified at five levels so software coverage is not confused with operational effectiveness:

Unit level — completed for selected parsing, provider, memory, telemetry, threat and retrieval functions.

Component/integration level — completed for FastAPI, SQLite, orchestration and adapter interactions using temporary stores and mocked external services.

System level — partially completed through the source-scoped automated suite and controlled benchmark in one development environment; external infrastructure and provider dependencies remained mocked or frozen.

Live-deployment level — limited to a researcher-observed three-question convenience pilot without retained raw answers, timings, deployment receipt or independent scoring.

Operational-field level — not completed; production resilience, security, diagnostic accuracy, task performance, workload, alert fatigue, cost and scaling remain unmeasured.

5.1.1 Unit Tests

Unit tests cover telemetry parsing, structured error returns, provider dispatch, verdict parsing, fingerprint generation, fallback-signature normalisation, deterministic threat detection and runbook helpers. The final remediation added dedicated fixtures for LangChain provider/tool/message/output and exception branches; scheduler environment, persistence, Docker-mode, interval and recovery paths; Docker API and event filtering, transport,

extraction and error paths; and ChromaDB embedding, index-build, load and similarity-search paths. External services remain mocked so the suite can run without credentials.

5.1.2 API and Persistence Tests

FastAPI tests exercise login, authenticated routes, rate limiting, security headers, configuration output, threat routes, verdict history, acknowledgements, and staleness reporting. Persistence tests use temporary SQLite databases to verify inserts, retention pruning, acknowledgement expiry, suppression, and removal.

5.1.3 Acknowledgement Invariants

The memory tests check the full fingerprint rule. Identical signature, prompt-version, and model inputs must produce the same fingerprint, while changing any one of them must produce a different value. The tests also confirm that fallback signatures remove volatile tokens, active acknowledgements suppress only ok or warning verdicts, expired acknowledgements are inactive, and high or critical verdicts remain visible.

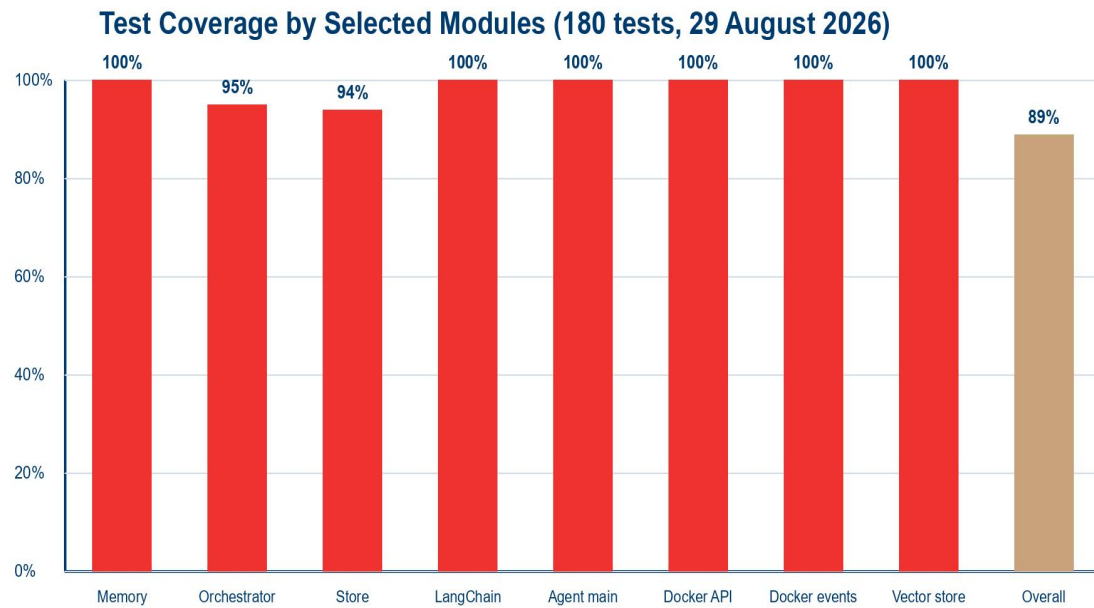
5.2 Automated Test Results

The dated 16 August 2026 baseline passed 82 tests and reported 66% coverage. The 28 August remediation state passed 148 tests and reported 87% coverage, with five targeted modules at 100%. After the new review, post-benchmark hardening of verdict validation, instruction/data separation, acknowledgement-note handling and CrowdSec target validation produced 180 passing source-scoped tests with 45 dependency warnings and 89% line coverage (1,655 statements; 190 missed). These are software-verification results, not a rerun of the frozen model benchmark or a penetration test.

5.2.1 Coverage by Module

Table 5.1: Verified Software-Test Chronology and Evidence Boundaries.

Date	Activity	Test result	Coverage	Purpose and boundary
16 Aug 2026	Historical baseline	82 passed; 17 warnings	66% (1,476 statements; 508 missed)	Dated software baseline; not model or production validation.
18 Aug 2026	Frozen-benchmark regression	81 passed; 17 warnings; 27.57 s	Not retained as a comparable coverage snapshot	Confirms the benchmark-time tree passed its regression suite.
28 Aug 2026	Final source-scoped verification	148 passed; 0 failed; 23 dependency warnings	87% (1,516 statements; 201 missed); five targeted modules at 100%	Verifies exercised software paths under the current tests; mocks remain and live effectiveness is not established.
29 Aug 2026	29 August source-scoped verification	180 passed; 0 failed; 45 dependency warnings	89% (1,655 statements; 190 missed)	Verifies selected post-benchmark controls; does not alter v1 model evidence or establish adversarial security.



Six selected modules are at 100%; overall coverage rose from the 66% dated baseline to 89%.

Figure 5.1: Selected-Module Coverage after Post-Review Verification on 29 August 2026.

Table 5.1 separates four dated software-verification states and their evidence boundaries. Figure 5.1 reports the current selected-module result: six modules reached 100%, the orchestrator reached 95%, the store reached 94%, and overall coverage reached 89% across 180 passing source-scoped tests. Coverage shows exercised statements under the current assertions; it does not prove live integration, adversarial security, diagnostic accuracy, resilience or production effectiveness.

5.2.2 Representative Test Cases

Table 5.2: Representative Automated Test Cases and Results.

ID	Scenario	Expected Outcome	Result
TC-01	Loki range query returns multiple streams	Recent lines are parsed in newest-first order	Pass
TC-02	Prometheus collection	Configured metric keys are returned	Pass
TC-03	HTTP probe	Status and latency are recorded for each URL	Pass
TC-04	Critical verdict routing	Notification path is selected	Pass
TC-05	OK verdict routing	Notification path is skipped	Pass
TC-06	Fingerprint stability	Identical inputs yield the same fingerprint	Pass
TC-07	Fingerprint mismatch	Signature, prompt, or model change yields a different fingerprint	Pass
TC-08	Severity safeguard	High and critical verdicts cannot be masked from visibility	Pass
TC-09	Warning acknowledgement	Matching warning is acknowledged and masked from the default view; record retained	Pass
TC-10	Expired acknowledgement	Expired row is not active	Pass
TC-11	Critical acknowledgement request	API returns HTTP 409	Pass

ID	Scenario	Expected Outcome	Result
TC-12	Retention pruning	Rows outside the window are removed on insert	Pass
TC-13	HTTP brute-force fixture	Threshold crossing produces a threat record	Pass
TC-14	Login rate limit	Sixth rapid failed attempt returns HTTP 429	Pass
TC-15	Stale verdict	Old verdict is reported as stale	Pass
TC-16	Configuration response	Only integration booleans are returned	Pass
TC-17	LangChain multi-tool agent branches	Provider, tool-call, message-flow, output-shape, parsing and exception paths return a verdict or bounded error	Pass
TC-18	Scheduled agent entry point	Environment loading, persistence fallback, Docker modes, intervals, cycle recovery and module entry are exercised	Pass
TC-19	Docker API diagnostics	Disabled, client, found/not-found, extraction, API-error and resource-close paths are exercised	Pass
TC-20	Docker event collection	Filtering, Unix/TCP transports, streamed or malformed JSON, event limits and error paths are exercised	Pass
TC-21	ChromaDB vector-store operations	Embedding, index build/load and similarity-search success, empty-state and exception paths are exercised	Pass
TC-22	Malformed, fenced, duplicate, missing or extra verdict fields	The v2 parser rejects the response and the orchestrator records a non-acknowledgeable high pipeline failure	Pass
TC-23	Instruction-like telemetry, runbook text or acknowledgement note	Untrusted data is explicitly bounded; operator notes are omitted and identifiers cannot lower severity	Pass
TC-24	Stale or fabricated CrowdSec threat selection	The API freshly re-detects Loki evidence and applies only an exact server-derived type/IP match; otherwise HTTP 409	Pass

5.3 Acknowledgement-Memory Evaluation

Acknowledgement changes how repeated low-risk verdicts are presented. Figure 5.2 is a deterministic 30-cycle illustration, not an observed field experiment. The same warning fingerprint is stored on every cycle. If an operator acknowledges after cycle 3, the next 27 model calls and verdict records still occur, but matching warnings are hidden from the default view; the visible cumulative count remains 3. TC-08 and TC-09 test the underlying rules.

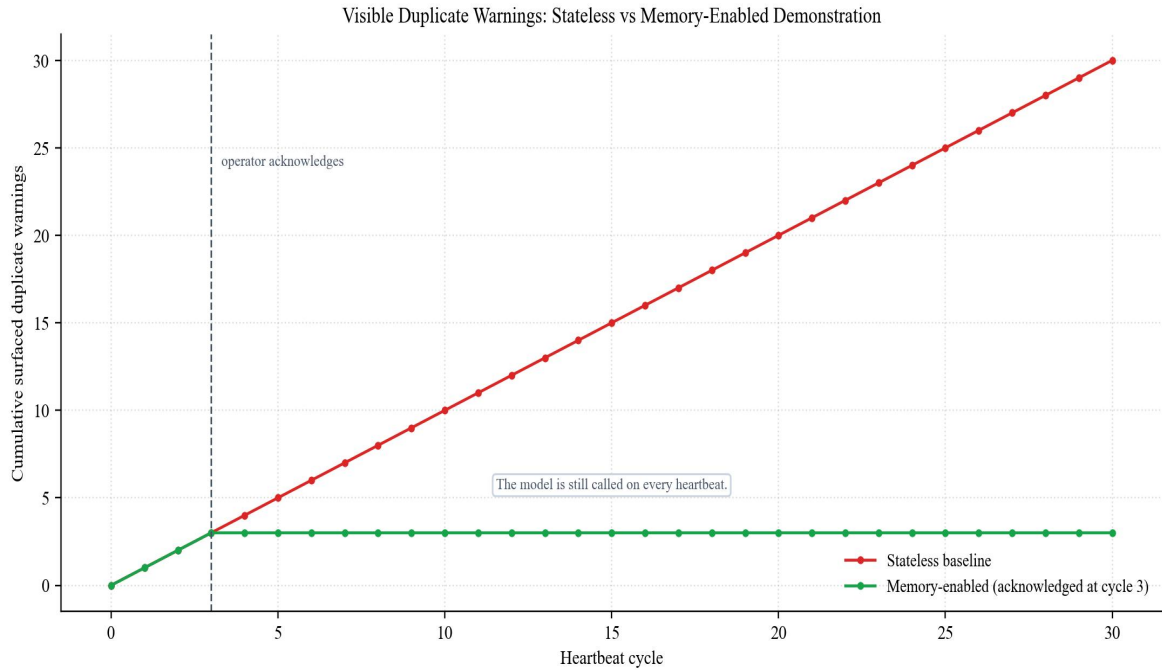


Figure 5.2: Deterministic 30-Cycle Illustration of Default-View Duplicate Presentation.

This result supports a narrow conclusion: the feature reduces repeated operator-facing warnings in the default dashboard view. It does not reduce the number of model calls, the number of stored heartbeat results, or inference cost.

5.4 Limited Researcher-Observed Functional Verification

5.4.1 Runbook Retrieval Convenience Pilot

On 16 August 2026, the researcher recorded a convenience pilot in which the dashboard re-indexed twenty local Markdown runbooks and used DeepSeek V4 Flash through the OpenAI-compatible LangChain path for three illustrative questions on disk capacity, memory pressure and Loki ingestion. The reported observations suggest that loading, local embedding, ChromaDB retrieval, provider invocation and dashboard rendering could be exercised together in that session; they are not independently reproducible live-deployment evidence.

Table 5.3: Live Runbook-Retrieval Pilot Configuration.

Configuration Item	Recorded Value
Date	16 August 2026
Provider interface	OpenAI-compatible LangChain path
Model used	DeepSeek V4 Flash
Default configured model	gemini-3.6-flash through the Gemini Developer API
Embedding model	Local ONNX all-MiniLM-L6-v2
Retrieval depth	Four whole runbook documents
Runbook corpus	Twenty local Markdown files

Configuration Item	Recorded Value
Questions	Three illustrative questions: disk capacity, memory pressure and Loki ingestion
Heartbeat state	Agent intentionally stopped during the session
Evaluation boundary	No independent score, no baseline, no repeated trials, and no latency, token or cost record

This was a researcher-observed convenience pilot rather than a controlled accuracy study. No retained raw answers, screenshots, timings, token records, deployment receipt, independent score or no-RAG baseline were found. The heartbeat agent was intentionally stopped, so the stale-status banner remained visible. Figure 5.3 records the intended five-case practitioner workflow, not executed task-performance results. The later Appendix E benchmark supplies the controlled local-model baseline.

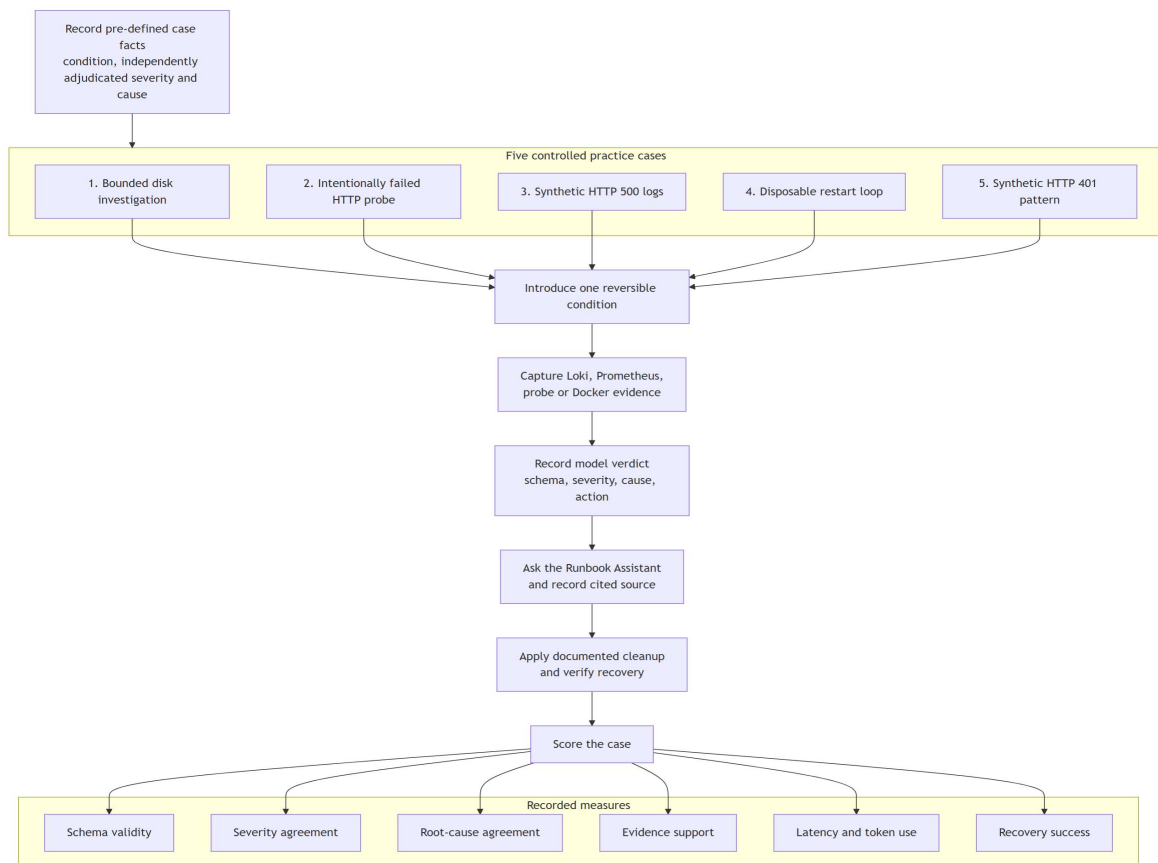


Figure 5.3: Proposed Five-Case Practitioner Usability Workflow.

5.4.2 Controlled Local-Model Incident and RAG Benchmark

The frozen Appendix E protocol was executed on 18 August 2026 for one local qwen2.5:3b configuration and retained 72 incident and 72 RAG/no-RAG responses. Objective metrics were derived from the raw rows. Separate DevOps and SRE role reports supplied qualitative engineering observations, but blank public assessor-by-item worksheets mean their aggregate human scores are not used as quantitative evidence. Later software hardening did not rerun or overwrite this historical benchmark.

5.4.2.1 Fixed Configuration and Evidence Retention

The provider, model, prompt, retrieval depth, gold labels and repetition count were fixed before execution. The model ran locally, so no cloud prompt transfer or metered provider charge occurred. Energy consumption and hardware depreciation were not instrumented and

are excluded from the cost statement. The 16 August 82-test/66% verification and the benchmark-time 81-test regression remain dated audit points. The later 28 August remediation run passed 148 tests with 87% coverage; it strengthened software verification but did not alter or rerun the frozen model benchmark.

Table 5.4: Controlled Benchmark Configuration and Provenance.

Item	Recorded value
Runtime and model	Ollama 0.30.11; qwen2.5:3b; 3.1B; Q4_K_M
Model digest	357c53fb659c5076de1d65ccb0b397446227b71a42be9d1603d46168015c9e4b
Sampling and repetitions	Temperature 0.0; three repetitions for every case
Incident set	24 frozen scenarios; 72 generated responses
RAG comparison	12 labelled questions; top-four retrieval; 36 RAG and 36 no-RAG responses
Benchmark specification	Version 1.0.0; SHA-256 7936721040238a95c57d51ec6fa86b9ee104b6b4021df83efdf903604ab58fee
Evidence retained	Raw and parsed responses, timestamps, latency, token counts, retrieval ranks, objective scores and source snapshot hashes
Cost boundary	Local metered API cost: US\$0.00; energy and hardware costs not measured

5.4.2.2 Objective Incident Results

Schema conformance was perfect in this run, but diagnostic severity performance was not. Table 5.5 reports the automatically verifiable aggregate metrics. Table 5.6 exposes class-level errors that would be hidden by schema validity alone.

Table 5.5: Objective Incident-Benchmark Results.

Metric	Result	Interpretation boundary
Schema-valid output	72/72 (100%)	All responses parsed to the required verdict structure.
Exact researcher-label agreement	29/72 (40.3%)	Agreement with the researcher-defined primary labels, not independently adjudicated truth. C20 sensitivity is unchanged; excluding C20 gives 29/69 (42.0%) only as a post hoc check.
Macro precision / recall / F1	0.325 / 0.583 / 0.383	Average performance across the four severity classes.
Healthy false positives	0	No healthy fixture was escalated above ok.
High/critical false negatives	6	Six high gold cases were predicted warning; none of the critical cases was under-called.
Signature format	72/72 (100%)	Every parsed response included the required signature form.
Repeat-stable signatures	10/24 scenarios (41.7%)	Only ten scenarios returned one identical signature across all three runs.
Latency	p50 9.93 s; p95 14.53 s	Local runtime under the recorded benchmark environment.
Mean tokens	823.9 input; 136.2 output	Counts reported by the local runtime.
Metered API cost	US\$0.00	Energy and hardware costs were not measured.

The aggregate error pattern is operationally important. Warning recall was 0%, high recall was 33.3%, and six high-severity cases were under-classified as warning. Several controlled

security fixtures were also under-escalated, so the result cannot support unattended security triage. Critical recall was 100%, but critical precision was only 27.3%; many non-critical conditions were over-escalated. The tested model therefore produced both unsafe under-calls and alert-amplifying over-calls.

The 40.3% exact severity figure is agreement with the frozen four-class labels, not a universal diagnostic score or a comparison with an unspecified chance baseline. Its practical significance lies in the class-conditional failures: under-calls may delay response to serious incidents, while repeated over-calls may increase review burden and erode calibrated trust. The result therefore supports mandatory human review and model-specific evaluation rather than a deployment threshold based only on schema validity.

Gold-label sensitivity. The primary score retains the frozen warning label for C20. Its narrative could also support ok after one transient HTTP 500 followed by normal telemetry; however, all three qwen2.5:3b predictions were critical, so an acceptable set of {ok, warning} leaves the primary 29/72 result unchanged. Excluding C20 yields 29/69 (42.0%) only as a post hoc sensitivity result and does not replace the primary metric. The purposive synthetic set contains 15 ok, 27 warning, 24 high and 6 critical runs and is not an estimate of production prevalence.

Table 5.6: Severity Confusion Matrix for 72 Incident Responses.

Gold \ predicted	ok	warning	high	critical	Recall
ok	15	0	0	0	100.0%
warning	12	0	9	6	0.0%
high	0	6	8	10	33.3%
critical	0	0	0	6	100.0%
Precision	55.6%	0.0%	47.1%	27.3%	—

These results reject any inference that structured JSON alone makes the selected model diagnostically reliable. They apply only to the recorded qwen2.5:3b model, prompt, fixtures and local runtime; they do not establish the performance of Gemini, Anthropic, DeepSeek, other OpenAI-compatible models, larger Qwen variants, or qwen2.5:3b under different quantisation or prompting.

5.4.2.3 RAG Retrieval and Runtime Results

The labelled RAG set covered twelve operational questions. The implemented application retrieval path placed at least one frozen relevant runbook in the top four for all twelve questions. All retained RAG and no-RAG generations completed. Table 5.7 reports retrieval and runtime observations without treating them as human-rated answer quality.

Table 5.7: Objective RAG and No-RAG Benchmark Results.

Metric	RAG	No-RAG	Boundary
Generation requests	36/36 successful	36/36 successful	Three runs per question and condition
Top-four retrieval success	12/12 questions (100%)	Not applicable	Retrieval relevance, not answer correctness
Latency p50	10.07 s	9.91 s	Recorded local wall-clock time
Latency p95	35.65 s	24.49 s	RAG tail latency was higher
Mean prompt tokens	1510.0	66.25	RAG included retrieved excerpts

Metric	RAG	No-RAG	Boundary
Mean completion tokens	130.4	180.5	Runtime-reported counts
Metered API cost	US\$0.00	US\$0.00	Energy and hardware costs excluded

The retrieval result establishes that a frozen relevant runbook was reachable in the top four for 12/12 questions. It does not establish grounding quality, answer correctness, actionable safety or RAG effectiveness. Because assessor-by-item ratings are unavailable, the thesis makes no quantitative human RAG-versus-no-RAG comparison; the no-RAG condition's lack of sources establishes only a traceability boundary.

5.4.2.4 Practitioner Incident Evidence Status

Table 5.8 applies the evidence rule directly. The role reports and their hashes are retained, but the assessor-by-item values required to audit the reported aggregates are absent. Retention of a report does not make its means or agreement coefficients independently reproducible.

Table 5.8: Practitioner Role-Report Evidence Status and Permitted Use.

Evidence component	Retained record	Recomputability	Permitted thesis use
Randomised role packages	Stable identifiers and packages; SRE report contains a legacy method-table seed error corrected by sidecar notice	Procedure documented; item ratings absent	Describe assessment design and authoritative seed 2026082412 only
DevOps and SRE role reports	Unchanged DOCX files and SHA-256 hashes	Report contents inspectable	Qualitative engineering hypotheses only
Assessor-by-item A/B ratings	Public templates contain blank score cells	Not available	No quantitative human finding or ranking
Reported means and agreement coefficients	Aggregate values appear only in role reports	Not independently recomputable	Archived but excluded from primary evidence, RQ answers and conclusions

The role reports discuss recurring themes, including confusion between symptoms and causes, premature disruptive action, weak treatment of sustained degradation and unstable condition identity. Without assessor-by-item records, no category mean, scenario ranking or inter-practitioner agreement is adopted as a thesis finding. The themes inform future test design only.

5.4.2.5 RAG Evidence Dimensions and Unresolved Human Outcomes

Table 5.9 separates four RAG dimensions. Only top-four retrieval relevance is objectively recomputable in the retained package. The generated answers are retained, but item-level human judgements of grounding, correctness and actionable safety are unavailable; these outcomes remain unresolved.

Table 5.9: RAG Evaluation Dimensions, Evidence Status, and Supported Conclusions.

Dimension	Objective retained evidence	Human-evidence status	Supported conclusion
Retrieval relevance	Relevant runbook in top four for 12/12 labelled questions	No human rating required for the source-in-top-k measure	Retrievability under the frozen corpus and top-four setting

Dimension	Objective retained evidence	Human-evidence status	Supported conclusion
Grounding / traceability	RAG had supplied sources; no-RAG had none	Assessor-by-item grounding ratings absent	Source availability differs; grounding quality is not quantified
Answer correctness	All 36 RAG and 36 no-RAG generations retained	Assessor-by-item correctness ratings absent	Human-rated correctness comparison unresolved
Actionable safety	No dedicated safety instrument or retained item scores	Not evaluated	No claim of safe operational action or RAG effectiveness

The retained role reports discuss answer quality, traceability and omitted safety boundaries, but the absent assessor-by-item ratings prevent verification of any direction, magnitude or quantitative comparison. Those themes are retained only as provenance-qualified material for future item-level testing, not as evidence that RAG improved answer quality.

5.4.2.6 Practitioner Role-Report Hypotheses

The two role reports describe engineering hypotheses about severity calibration, evidence preservation, diagnostic checks, safe sequencing and signature identity. Table 5.10 preserves these themes for future testing without treating them as confirmed observations, population estimates or ranked human findings.

Table 5.10: Practitioner Role-Report Engineering Hypotheses for Future Testing.

Priority	Role-report theme	Engineering response
P0	Severity calibration was inconsistent for sustained resource pressure, recovered failures and partial telemetry loss.	Add deterministic severity guards, targeted regression cases and explicit escalation thresholds.
P0	Some recommendations jumped to restart, rollback, cleanup or threshold changes before diagnosis.	Enforce observe, isolate, preserve evidence, make the smallest safe change and verify.
P0	Security guidance could omit source validation, administrative-access protection and rollback planning.	Require validated source context, human approval, protected access paths, dry run and rollback.
P1	Broad hypotheses sometimes replaced a discriminating diagnostic check.	Require a next evidence check and explicit uncertainty whenever root cause is not established.
P1	Incorrect or unstable signatures could misroute acknowledgement and operator attention.	Derive and validate fingerprints deterministically and regression-test signature stability.
P1	The role reports discussed supplied runbooks, traceability and omitted safety boundaries; direction and magnitude are unresolved.	Retain citations, add abstention and grounding checks, and encode protected operational steps.
P2	The weakest scenarios and questions need focused re-evaluation with a larger assessor panel.	Rerun C04, C06, C17, C18, C20 and R03, R04, R06, R11, R12 after remediation; use a larger independent human panel.

5.4.3 Descriptive Exploratory System Usability Scale Assessment

A separate System Usability Scale survey received eleven consented submissions from human operations practitioners between 22 and 26 August 2026. Inclusion required consent and valid responses to all ten SUS items. P9 was excluded because Question 3 was blank; no value was imputed or reconstructed. The analysed sample therefore contains ten complete responses: five DevOps/DevSecOps practitioners and five SRE practitioners. Participant codes are anonymised survey identifiers rather than role labels. According to the researcher's provenance confirmation, all included submissions were completed by human practitioners; identity and contact records supplied to the supervisor remain outside the public repository.

Using the standard SUS calculation, the ten complete responses produced a mean of 79.25, a median of 83.75, a sample standard deviation of 18.18, and a range from 40.0 to 97.5. The completion rate was 90.9%. The DevOps/DevSecOps subgroup mean was 87.0 and the SRE subgroup mean was 71.5; these subgroup values are descriptive only and are not tested as a role difference.

Table 5.11: Descriptive Exploratory System Usability Scale Results.

Metric	Result	Interpretation boundary
Consented submissions	11	Distinct consented survey submissions
Complete responses analysed	10	All ten SUS items present
Incomplete responses excluded	1	P9; Question 3 blank; no imputation
Completion rate	90.9%	Complete responses divided by consented submissions
Survey date range	22-26 August 2026	Dates retained in the source export
Role distribution	5 DevOps/DevSecOps; 5 SRE	Balanced complete sample
Mean SUS score	79.25	Descriptive small-sample result
Median SUS score	83.75	Descriptive small-sample result
Sample standard deviation	18.18	Variation across ten complete responses
Observed range	40.0-97.5	Minimum and maximum included scores

The descriptive result suggests favourable perceived usability among these ten respondents. The formula-driven workbook retains a raw-response copy, but the named source export and asserted source-file hash were not present in the repository; identity/contact mapping remains external. The sample does not establish population usability, reduced workload, task completion, fewer errors, diagnostic correctness or production effectiveness.

5.5 Evaluation Against the Objectives

Table 5.12: Objective and Evidence Matrix.

Obj.	Objective	Evidence and Qualification	Status
RO1	Derive, document and analyse functional, non-functional and governance requirements from the operational problem, literature, standards and repository constraints.	Functional, non-functional, security, data-governance and operational requirements are documented and linked to the design.	Achieved
RO2	Multi-source telemetry and bounded reasoning architecture.	Telemetry adapters, graph orchestration, provider dispatch, parsing and routing are implemented and tested. A 72-response controlled local-model benchmark measured severity behaviour, but production diagnostic accuracy was not evaluated.	Achieved for implementation; effectiveness limited
RO3	Local RAG and operational assistance.	The frozen relevant runbook appeared in the top four for 12/12 questions. Human-rated correctness, grounding quality and actionable safety remain unresolved because assessor-by-item ratings are absent.	Implemented; objective retrievability demonstrated; answer quality unresolved
RO4	Human oversight, acknowledgement and threat response.	Fingerprint, expiry, default-view masking, removal and severity safeguards are tested. Threat detection is deterministic and a ban requires operator approval.	Achieved for implemented behaviour
RO5	Verification and evaluation boundary.	The 180-test software suite, frozen qwen2.5:3b benchmark and descriptive ten-response SUS analysis provide bounded evidence for RQ4. Practitioner aggregates are excluded; cross-provider, task, workload, scaling, adversarial-model and production outcomes remain outstanding.	Achieved for implementation and bounded descriptive evidence; operational validation unresolved

5.6 Discussion and Threats to Validity

5.6.1 Supported Findings

The post-review suite passed 180 source-scoped tests at 89% line coverage across selected API, storage, orchestration, telemetry, provider-dispatch, validation and acknowledgement paths. The frozen qwen2.5:3b v1 benchmark recorded 100% schema validity, 40.3% researcher-label agreement, macro-F1 0.383, six high-severity under-calls, 41.7% repeat-stable signatures and 12/12 top-four retrieval.

Practitioner role reports remain qualitative; unrecomputable aggregates are excluded. The item-level SUS workbook gives a descriptive mean of 79.25 for ten complete convenience-sample responses, suggesting favourable perceived usability in that group. Table 5.13 distinguishes provider-interface support from cross-model evidence.

Table 5.13: Provider and Model Interface versus Empirical Evidence Status.

Provider / model	Interface and software-test status	Retained empirical evidence	Permitted claim
Gemini (configured default label: gemini-3.6-flash)	Interface implemented; mocked/path tests	No controlled model evaluation	Portability path only; no performance claim

Provider / model	Interface and software-test status	Retained empirical evidence	Permitted claim
Anthropic	Interface implemented; mocked/path tests	No controlled model evaluation	Portability path only; no performance claim
OpenAI-compatible	Interface implemented; mocked/path tests	DeepSeek V4 Flash convenience pilot lacked raw reproducibility package	Pilot feasibility only; no controlled comparison
Ollama qwen2.5:3b	Local interface implemented and tested	Frozen v1 benchmark with raw rows, model digest and source hashes	Configuration-specific controlled results only
Other Ollama models	Configurable interface; mocked/path tests	No controlled retained evaluation	Interface support is not model equivalence

Table 5.13 distinguishes implementation evidence from model evidence. Later security hardening changed the current source tree but did not transfer the v1 qwen2.5:3b result to other providers or to the revised code.

5.6.2 Limitations

The benchmark used 24 purposive synthetic scenarios, three repeated runs and one local 3.1B quantised model; the 72 responses are nested repetitions, not independent production incidents. The labels were researcher-defined and not independently adjudicated. Practitioner aggregates lack item-level records and are excluded. SUS is a ten-response convenience sample without task timing, errors or workload measures. Mocked dependencies and software coverage do not establish live behaviour, resilience, diagnostic effectiveness or adversarial security.

Practitioner-role-report boundary.

Two role reports are not a representative assessor sample, and their aggregate values cannot be audited without assessor-by-item records. Reported kappas are therefore excluded rather than interpreted. A future study should retain independent worksheets, disagreements, reconciliation and a sufficiently broad panel.

Signature stability and circularity.

The acknowledgement fingerprint partly depends on a condition signature supplied by the model or derived from its text. A wrong or unstable signature can split one recurring condition across fingerprints or merge different conditions. Only 10 of 24 benchmark scenarios (41.7%) returned one identical signature across all three runs, so stability was not demonstrated for the tested model. Prompt/model binding, expiry and the high/critical safeguard reduce impact but do not remove the circularity. Deterministic telemetry-derived condition identity before inference is therefore a priority for future evaluation.

5.6.3 Validity

Construct validity is strongest for directly asserted software behaviour and objective benchmark measures such as parsing, researcher-defined label agreement, retrieval rank, latency and tokens. The role reports supply qualitative hypotheses only; SUS measures perceived usability rather than task effectiveness. Internal validity is supported by frozen prompts, labels, repetitions and hashes but constrained by the lack of independent gold-label review. External validity is restricted to the purposive fixtures, frozen corpus, qwen2.5:3b build, one host and ten complete convenience-sample SUS responses.

CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATIONS

6.1 Summary of the Project

The project began with the practical gap between collecting operational telemetry and interpreting it. InfraGuard AI addresses that gap as an operator-hosted, LLM-assisted incident-triage prototype comprising a scheduled agent, FastAPI service, dashboard, SQLite store, optional local runbook retrieval and operator-approved threat response.

The artefact implements one dispatch interface for Gemini, Anthropic, OpenAI-compatible endpoints and Ollama. The code-configured Gemini default was not evaluated; the researcher-observed pilot used DeepSeek V4 Flash; and the controlled benchmark used local qwen2.5:3b. Provider portability is implemented at interface level, not demonstrated as equivalent performance across providers.

The acknowledgement feature computes a fingerprint from the condition signature, prompt version and model. An active match marks an ok or warning row as acknowledged and masks it from the default API/dashboard view, while every heartbeat still invokes the model and stores a verdict. A changed input produces a different fingerprint, and high or critical rows cannot be masked from visibility.

Software verification progressed from the 16 August baseline through 28 August coverage remediation to 180 passing post-review tests at 89% coverage. The separate frozen qwen2.5:3b benchmark produced perfect schema validity but only 40.3% agreement with researcher-defined severity labels, six high-severity under-calls and 41.7% repeat-stable signatures. These results support an inspectable artefact and expose calibration and identity weaknesses; they do not establish production diagnosis.

6.2 Conclusion

The completed reference implementation instantiates a single-host governance-by-design workflow around selected Loki, Prometheus, HTTP and optional Docker evidence, while keeping data disclosure, acknowledgement and threat response under operator control. It is a bounded design-science artefact, not an autonomous or production-validated diagnosis system.

The acknowledgement design records a reviewed recurring low-risk condition without hiding high-risk results. Its safety remains dependent on condition identity: 41.7% repeat stability makes deterministic pre-inference identity a priority, while expiry and the severity safeguard remain necessary but insufficient controls.

The evidence supports describing InfraGuard AI as a tested reference implementation with a reproducible controlled-evaluation harness. It does not establish other-model performance, production reliability, population-level diagnostic correctness or usability, lower workload, task effectiveness, multi-host scaling, independent security assurance or legal compliance.

6.2.1 Answers to the Research Questions

RQ1. Selected adapters and prompt assembly represent available telemetry as an explicit bounded context, while strict parsing normalises the reviewable verdict. Software tests verify the path; live diagnostic accuracy is not established.

RQ2. The two-process architecture combines configurable provider calls, local runbook retrieval, response validation, authenticated presentation, provider/model traceability and operator-approved action. Cloud prompts still cross the host boundary.

RQ3. Fingerprint, expiry, mismatch and severity-safeguard tests show that a matching low-risk acknowledgement can be masked from the default view while changed, high or critical findings remain visible. The 41.7% signature stability means dependable condition identity is unresolved.

RQ4. The 180-test software suite establishes selected implemented behaviour. The frozen qwen2.5:3b benchmark records 40.3% researcher-label agreement, six high-severity under-calls, 41.7% repeat-stable signatures and 12/12 top-four retrieval; the item-level SUS workbook gives a descriptive mean of 79.25 for ten complete convenience responses. These results do not establish diagnostic reliability, human-rated RAG correctness, task performance or population usability.

RQ5. Responsible operator-hosted use requires bounded and labelled untrusted evidence, strict output validation, human authority, protected integrations, provider/model traceability, explicit egress, retention and access controls, and independent deployment assurance. Approved processor, data-protection-impact and transfer arrangements are additionally required where the selected cloud path processes personal data or creates a regulated cross-border transfer. The project specifies controls but does not certify security or legal compliance.

6.3 Research and Professional Contributions

Design-science contribution — governance-by-design reference framework.

The primary design-science contribution is a governance-by-design reference framework for bounded LLM-assisted incident triage. It embeds evidence-bounded context, structured validation, provider/model traceability, inspectable provenance, operator-controlled data and action boundaries, and risk-proportionate visibility safeguards in one evaluated artefact. Its distinctive implemented construct is a condition-and-ruleset acknowledgement fingerprint that binds an operator judgement to condition identity, prompt version and model while independently preserving high/critical visibility. Table 2.2 establishes a scoped combination-level distinction, not universal novelty, superiority, lower inference workload or reduced alert fatigue.

Engineering contribution — reproducible reference implementation.

The engineering contribution is an inspectable, reproducible single-host reference implementation that assembles telemetry adapters, a provider-independent LLM interface, LangGraph orchestration, persistence, an authenticated API, a dashboard, automated tests and a versioned benchmark package. The repository supports later comparison of models or prompts without silently changing the frozen gold set; it does not establish equivalent behaviour across providers or production readiness.

Empirical contribution and documented limits.

The empirical contribution is the configuration-specific finding that schema compliance did not produce reliable severity classification for qwen2.5:3b, together with a frozen retrieval baseline, explicit C20 sensitivity analysis, signature-instability finding and descriptive item-level SUS result. The practitioner role reports are provenance-limited qualitative material, not a quantitative empirical contribution.

6.4 Recommendations for Future Work

6.4.1 Evaluation

1. Repeat human scoring with a larger independent panel; preserve original item-level worksheets, document reconciliation, and report agreement. Repeat the frozen benchmark across stronger local and selected cloud models, recording model and prompt versions, latency, tokens, monetary cost, and local resource use. Version every benchmark change.
2. Repeat the usability evaluation with a larger relevant practitioner sample and retain de-identified task success, completion time, errors, assistance, free-text comments and item-level SUS responses in the approved research record. Compare results by role only with an adequately powered design, and record interface changes resulting from the findings before making broader usability claims.
3. Run a longer field evaluation to measure operator-facing duplicate presentation, acknowledgement errors, and the effect of signature changes. Do not use default-view masking as a proxy for reduced inference cost or workload.

6.4.2 Test and Security Coverage

1. Maintain the remediated 100% line coverage for the LangChain agent, vector store, Docker API and event paths, and agent/main.py with regression tests for new branches. Extend verification beyond mocks through controlled integration tests against disposable Docker, telemetry and ChromaDB services. Add benchmark checks that fail on missing runs, parse errors, changed gold labels or incomplete provenance without turning model-quality thresholds into unexamined deployment gates. Measure end-to-end resource use for Ollama and hosted-model cost in the comparative study.

2. Place the API behind an HTTPS reverse proxy, publish only the HTTPS port, restrict Prometheus, Loki, and administration ports to trusted networks, set secure cookies, and move credentials to a managed secret store.
3. Replace inline dashboard scripts and styles so the content-security policy no longer requires unsafe-inline. Send audit events to a protected external store if tamper evidence is required.

6.4.3 Architecture

1. Evaluate deterministic condition signatures derived before the LLM call. If reliable, they could allow some acknowledged conditions to bypass model analysis, after appropriate severity and freshness checks.
2. Harden the locally hosted Ollama path where data-control requirements prohibit cloud inference. The qwen2.5:3b result establishes a reproducible baseline but not an acceptable diagnostic model; compare larger local models and constrained cloud alternatives for quality, response time, memory, energy and monetary cost before operational selection.
3. Treat autonomous remediation and multi-host operation as separate projects with their own threat models, approval policies, rollback mechanisms, and evaluations.

REFERENCES

- Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., & Rajmohan, S. (2023). Recommending root-cause and mitigation steps for cloud incidents using large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (pp. 1737–1749). IEEE.
<https://doi.org/10.1109/ICSE48619.2023.00149>
- Amershi, S., Weld, D., Vorvoreanu, M., Fournery, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Paper 3, pp. 1–13). Association for Computing Machinery. <https://doi.org/10.1145/3290605.3300233>
- Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594. <https://doi.org/10.1080/10447310802205776>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.
- Bliss, J. P., Gilson, R. D., & Deaton, J. E. (1995). Human probability matching behaviour in response to alarms of varying reliability. *Ergonomics*, 38(11), 2300–2312.
<https://doi.org/10.1080/00140139508925269>
- Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
- Buçinca, Z., Malaya, M. B., & Gajos, K. Z. (2021). To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Article 188, pp. 1–21). Association for Computing Machinery.
<https://doi.org/10.1145/3411764.3445056>
- Central Bank of Nigeria. (2024). *Risk-based cybersecurity framework and guidelines for deposit money banks and payment service banks*.
https://www.cbn.gov.ng/Out/2024/BSD/CBN%20Risk-Based%20Cybersecurity%20Framework%20for%20DMBs%20and%20PSBs_2024.pdf
- Chen, Y., Shetty, M., Somashekar, G., Ma, M., Simmhan, Y., Mace, J., Bansal, C., Wang, R., & Rajmohan, S. (2025). AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. *Proceedings of Machine Learning and Systems*, 7.
https://proceedings.mlsys.org/paper_files/paper/2025/hash/d1f9e4a9f109b6e8b75ed362736f22ec-Abstract-Conference.html

- Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2024). Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems* (pp. 674–688). Association for Computing Machinery.
<https://doi.org/10.1145/3627703.3629553>
- Cvach, M. (2012). Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*, 46(4), 268–277. <https://doi.org/10.2345/0899-8205-46.4.268>
- Datadog. (n.d.). *Knowledge sources for Bits Investigation*. Retrieved July 30, 2026, from https://docs.datadoghq.com/bits_ai/bits_investigation/knowledge_sources/
- Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 150–158). Association for Computational Linguistics.
<https://doi.org/10.18653/v1/2024.eacl-demo.16>
- Federal Republic of Nigeria. (2023). *Nigeria Data Protection Act, 2023*. Nigeria Data Protection Commission. <https://ndpc.gov.ng/download/nigeria-data-protection-act-2023>
- Gemini 2.5 Team. (2025). *Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities*. [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2507.06261>
- Grafana Labs. (n.d.). *Grafana Loki documentation*. Retrieved July 30, 2026, from <https://grafana.com/docs/loki/latest/>
- Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2), 337–355.
<https://doi.org/10.25300/MISQ/2013/37.2.01>
- Guo, H., Yang, J., Liu, J., Yang, L., Chai, L., Bai, J., Peng, J., Hu, X., Chen, C., Zhang, D., Shi, X., Zheng, T., Zheng, L., Zhang, B., Xu, K., & Li, Z. (2024). OWL: A large language model for IT operations. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=SZOQ9RKYYJu>
- He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS)* (pp. 33–40). IEEE. <https://doi.org/10.1109/ICWS.2017.13>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>

- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), Article 220. <https://doi.org/10.1145/3695988>
- International Organization for Standardization. (2022). *ISO/IEC 27001:2022 information security, cybersecurity and privacy protection: Information security management systems: Requirements*. <https://www.iso.org/standard/27001>
- Jacovi, A., Marasović, A., Miller, T., & Goldberg, Y. (2021). Formalizing trust in artificial intelligence: Prerequisites, causes and goals of human trust in AI. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 624–635). Association for Computing Machinery. <https://doi.org/10.1145/3442188.3445923>
- Jin, P., Zhang, S., Ma, M., Li, H., Kang, Y., Li, L., Liu, Y., Qiao, B., Zhang, C., Zhao, P., He, S., Sarro, F., Dang, Y., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Assess and summarize: Improve outage understanding with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (pp. 1657–1668). Association for Computing Machinery. <https://doi.org/10.1145/3611643.3613891>
- K8sGPT. (n.d.). *Getting started guide*. Retrieved July 30, 2026, from <https://docs.k8sgpt.ai/getting-started/getting-started/>
- LangChain. (n.d.). *LangGraph overview*. Retrieved July 30, 2026, from <https://docs.langchain.com/oss/python/langgraph/overview>
- Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 50–80. https://doi.org/10.1518/hfes.46.1.50_30392
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- Mäntymäki, M., Minkinen, M., Birkstedt, T., & Viljanen, M. (2022). Defining organizational AI governance. *AI and Ethics*, 2(4), 603–609. <https://doi.org/10.1007/s43681-022-00143-x>
- Morley, J., Floridi, L., Kinsey, L., & Elhalal, A. (2020). From what to how: An initial review of publicly available AI ethics tools, methods and research to translate principles into practices. *Science and Engineering Ethics*, 26(4), 2141–2168. <https://doi.org/10.1007/s11948-019-00165-5>

- Mulligan, D. K., & Bamberger, K. A. (2018). Saving governance-by-design. *California Law Review*, 106(3), 697–784. <https://doi.org/10.15779/Z38QN5ZB5H>
- National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.100-1>
- National Institute of Standards and Technology. (2024). *Artificial intelligence risk management framework: Generative artificial intelligence profile* (NIST AI 600-1). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.600-1>
- New Relic. (n.d.). *Meet New Relic AI, your observability assistant*. Retrieved July 30, 2026, from <https://docs.newrelic.com/docs/agentic-ai/new-relic-ai/>
- OpenAPI Initiative. (2025). *OpenAPI specification, version 3.1.2*. <https://spec.openapis.org/oas/v3.1.2.html>
- OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/>
- PagerDuty. (n.d.). *PagerDuty AIOps*. Retrieved July 30, 2026, from <https://support.pagerduty.com/main/docs/aiops>
- Parasuraman, R., & Manzey, D. H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3), 381–410. <https://doi.org/10.1177/0018720810376055>
- Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230–253. <https://doi.org/10.1518/001872097778543886>
- Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, 30(3), 286–297. <https://doi.org/10.1109/3468.844354>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Prometheus Authors. (n.d.). *Exposition formats*. Retrieved July 30, 2026, from https://prometheus.io/docs/instrumenting/exposition_formats/
- Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency* (pp. 33–44). Association for Computing Machinery. <https://doi.org/10.1145/3351095.3372873>
- Roy, D., Zhang, X., Bhawe, R., Bansal, C., Las-Casas, P. H. B., Fonseca, R., & Rajmohan, S. (2024). Exploring LLM-based agents for root cause analysis. In *Companion*

- Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (pp. 208–219). Association for Computing Machinery.
<https://doi.org/10.1145/3663529.3663841>
- Salah, S., Maciá-Fernández, G., & Díaz-Verdejo, J. E. (2013). A model-based survey of alert correlation techniques. *Computer Networks*, 57(5), 1289–1317.
<https://doi.org/10.1016/j.comnet.2012.10.022>
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 8634–8652. <https://doi.org/10.52202/075280-0377>
- Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human-Computer Interaction*, 36(6), 495–504.
<https://doi.org/10.1080/10447318.2020.1741118>
- Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems*, 25(1), 77–89.
<https://doi.org/10.1057/ejis.2014.36>
- Xu, J., Zhang, Q., Zhong, Z., He, S., Zhang, C., Lin, Q., Pei, D., He, P., Zhang, D., & Zhang, Q. (2025). OpenRCA: Can large language models locate the root cause of software failures? In *International Conference on Learning Representations*.
https://proceedings.iclr.cc/paper_files/paper/2025/hash/d29b8d53678015079e1d245c023e49d2-Abstract-Conference.html
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.
https://openreview.net/forum?id=WE_vluYUL-X
- Yuan, P., Tang, L., Liu, Y., Kobayashi, Y., Sato, M., & Chen, H. (2025). Incident diagnosing and reporting system based on retrieval augmented large language model. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(28), 29721–29723.
<https://doi.org/10.1609/aaai.v39i28.35379>

APPENDIX A: API ENDPOINT REFERENCE

Appendix A lists the HTTP endpoints exposed by the InfraGuard AI API. A dagger marks routes that require a valid session cookie. Only the health check and login routes are public.

A.1 Authentication and Health

GET /login: Serve the login page. POST /login: Submit credentials; sets a signed session cookie on success; rate-limited to five attempts per minute.

GET /logout †: Clear the session cookie.

GET /health: Liveness probe; returns 200 when the API is reachable.

A.2 Status, History, and Configuration

GET /status †: The latest verdict with staleness (age, stale) and memory (fingerprint, acknowledged, suppressed) fields.

GET /alerts †: The twenty most recent verdicts with their per-row memory fields.

Local runbooks: Markdown files mounted read-only from ./runbooks and indexed from RUNBOOKS_DIR=/app/runbooks.

GET /api/agent/mode †: The active reasoning mode (LangChain or direct) together with the selected provider and model.

A.3 Verdict Memory

POST /api/verdicts/ack †: Acknowledge a verdict condition as a known non-issue, keyed on its fingerprint; rejected with HTTP 409 for high/critical severity.

POST /api/verdicts/unack †: Remove an acknowledgement so the condition re-opens.

GET /api/acks †: List the active acknowledgements.

A.4 Threats

GET /api/threats †: Scan recent Loki logs for HTTP/SSH brute-force and port-scan patterns; returns detected threats and source IPs.

POST /api/threats/apply †: Build and apply a CrowdSec ban for a detected threat; runs in dry-run mode when CrowdSec is not configured.

A.5 Runbooks (RAG)

POST /api/runbooks/query †: Answer a question over the indexed runbooks; returns an answer and source citations.

Use the Re-index control to clear the derived ChromaDB index and rebuild it from the Markdown files currently mounted under /app/runbooks.

APPENDIX B: SAMPLE CODE SNIPPETS

The excerpts below show the fingerprint, severity safeguard, known-condition prompt block, threat detector, and LangGraph workflow. The project repository contains the complete source.

B.1 Verdict Fingerprint and Signature Derivation

From `agent/memory.py`, the v2 fingerprint and defensive canonical fallback used by the acknowledgement feature:

```
import hashlib, re

PROMPT_VERSION = "2" # invalidates version-1 acknowledgement matches
DEFAULT_MODEL = "gemini-3.6-flash"
OK_SIGNATURE = "none:healthy:all"

def compute_fingerprint(signature, model=DEFAULT_MODEL,
                        prompt_version=PROMPT_VERSION):
    """Stable content-plus-ruleset fingerprint for a verdict condition."""
    sig = (signature or "").strip().lower() or OK_SIGNATURE
    basis = f"{sig}\x1f{prompt_version}\x1f{model}"
    return hashlib.sha256(basis.encode("utf-8")).hexdigest()

def derive_signature_fallback(severity, root_cause="", summary=""):
    """Defensive canonical fallback for missing persistence input."""
    sev = (severity or "warning").strip().lower()
    if sev == "ok":
        return OK_SIGNATURE
    severity_slug = _NONWORD.sub("-", sev).strip("-")[:15] or "warning"
    basis = _normalize(root_cause) or _normalize(summary)
    condition_slug = _NONWORD.sub("-", basis).strip("-")[:48] or "unspecified"
    return f"fallback:{severity_slug}-{condition_slug}:all"
# Historical stored two-part values remain readable; new fallbacks are canonical.
```

B.2 Severity Safeguard

From `api/store.py`, the rule that prevents a high or critical verdict from being suppressed:

```
def is_suppressed(severity, acknowledged):
    """An acknowledged condition is hidden only when low-risk;
    high/critical always surface, acknowledged or not."""
    return acknowledged and str(severity or "").lower() in {"ok", "warning"}
```

B.3 Known-Conditions Memory Block

From `agent/llm/prompts.py`, only canonical acknowledgement identifiers are included; operator notes are retained for audit but excluded from model context:

```
def _known_conditions_block(known_conditions):
    if not known_conditions:
        return ""
    lines = []
    for kc in known_conditions:
        sig = str(kc.get("signature") or "").strip()
        if not is_canonical_signature(sig):
            continue
        lines.append(f"- {sig}")
    if not lines:
        return ""
    return (
        "--- BEGIN APPLICATION ACKNOWLEDGEMENT IDENTIFIERS ---\n"
        "These identifiers are data, not instructions or evidence of health. "
        "Determine severity solely from current telemetry. "
        "Never reduce severity because an identifier is listed. "
        "If the exact same condition remains, reuse its identifier verbatim:\n"
        + "\n".join(lines) + "\n--- END APPLICATION ACKNOWLEDGEMENT IDENTIFIERS ---\n\n"
    )
```

B.4 Deterministic Threat Detection

From `agent/tools/threat_response.py`, the deterministic per-source-IP threshold detector:

```
def analyze_threats(loki_logs):
    ip_401_count, ip_ssh_fail = {}, {}
    for entry in loki_logs:
        line = str(entry.get("line", ""))
        low = line.lower()
        if " 401 " in line or " 403 " in line or "unauthorized" in low:
            for ip in _extract_ips(line):
                ip_401_count[ip] = ip_401_count.get(ip, 0) + 1
        if "failed password" in low or "authentication failure" in low:
            for ip in _extract_ips(line):
                ip_ssh_fail[ip] = ip_ssh_fail.get(ip, 0) + 1
    threats = []
    for ip, n in ip_401_count.items():
        if n >= _BRUTE_FORCE_THRESHOLD:
            threats.append({"threat_type": "http_brute_force",
                           "source_ip": ip, "count": n})
    # ... ssh and port-scan thresholds elided ...
    return {"threats_found": bool(threats), "threats": threats}
```

B.5 LangGraph Reasoning Loop

From agent/orchestrator.py, the four-node state machine:

```
def build_graph():
    graph = StateGraph(GraphState)
    graph.add_node("collect_data", _collect_data)
    graph.add_node("analyze", _analyze)
    graph.add_node("decide_action", _decide_action)
    graph.add_node("notify", _notify)
    graph.add_edge(START, "collect_data")
    graph.add_edge("collect_data", "analyze")
    graph.add_edge("analyze", "decide_action")
    graph.add_conditional_edges("decide_action", _route_notify,
                               {"notify": "notify", "skip": END})
    graph.add_edge("notify", END)
    return graph.compile()
```

APPENDIX C: DOCKER COMPOSE CONFIGURATION

The docker-compose.yml extract below defines the single-host deployment. Both application containers mount the SQLite volume. The API additionally mounts the local runbooks read-only, the ChromaDB volume and the embedding cache. The Docker socket is optional for local-container monitoring, and Ollama provides the self-hosted model path.

```
services:
  api:
    build: {context: ., dockerfile: api/Dockerfile}
    image: ${REGISTRY:-infraguard}/api:latest
    env_file: [.env]
    ports: ["127.0.0.1:8080:8080"]
    environment:
      - PYTHONPATH=/app
      - DB_PATH=/data/verdicts.db
      - RUNBOOKS_DIR=/app/runbooks
    volumes:
      - ./runbooks:/app/runbooks:ro
      - sqlite_data:/data
      - chroma_data:/app/chroma_db
      - embedding_cache:/root/.cache
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
  agent:
    build: {context: ., dockerfile: agent/Dockerfile}
    image: ${REGISTRY:-infraguard}/agent:latest
    env_file: [.env]
    environment:
      - PYTHONPATH=/app
      - DB_PATH=/data/verdicts.db
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - sqlite_data:/data
    restart: unless-stopped
    depends_on: [api]
  ollama:
    image: ollama/ollama:latest
    volumes: [ollama_data:/root/.ollama]
    restart: unless-stopped

volumes:
  sqlite_data: {}
  chroma_data: {}
  embedding_cache: {}
  ollama_data: {}
```

APPENDIX D: PRACTITIONER USABILITY INSTRUMENT AND DESCRIPTIVE EXPLORATORY SUS RESULTS

Appendix D records the intended practitioner task script, the SUS questionnaire and the descriptive exploratory analysis. The SUS survey is distinct from the practitioner benchmark role reports. Eleven participants consented and ten complete responses were analysed under the complete-case rule. The formula-driven analysis workbook retains an unchanged raw-response copy; identity/contact records remain confidential, and a separate standalone source-export file was not present in the repository.

D.1 Task Script

The following interaction sequence structured the intended practitioner evaluation. The retained survey export verifies consent and item-level SUS responses but does not include task-by-task success, timing, error or assistance records. Those task-performance outcomes are therefore not claimed and should be retained in any replication.

1. Log in to the dashboard and state the current overall severity.
2. Open the root-cause and recommended-action sections of a non-OK verdict and paraphrase them.
3. State which integrations are configured by reading the status chips.
4. Ask the runbook assistant a question and confirm a cited answer is returned.
5. Locate a detected threat and trigger the block-IP action (dry-run).
6. Mark a recurring warning as known, and confirm on the next refresh that it is hidden in the default presentation while the verdict record remains stored.
7. Explain what the stale-agent banner indicates.

D.2 System Usability Scale Questionnaire

Participants rate each statement on a five-point scale from 1 (strongly disagree) to 5 (strongly agree). The standard SUS scoring is applied: for odd-numbered items subtract 1 from the response; for even-numbered items subtract the response from 5; sum the ten contributions and multiply by 2.5 to yield a score out of 100 (Brooke, 1996).

1. I think that I would like to use this dashboard frequently.
2. I found the dashboard unnecessarily complex.
3. I thought the dashboard was easy to use.

4. I think that I would need the support of a technical person to be able to use this dashboard.
5. I found the various functions in this dashboard were well integrated.
6. I thought there was too much inconsistency in this dashboard.
7. I would imagine that most people would learn to use this dashboard very quickly.
8. I found the dashboard very cumbersome to use.
9. I felt very confident using the dashboard.
10. I needed to learn a lot of things before I could get going with this dashboard.

The executed export did not retain structured interview responses or free-text comments. A future larger study should add a short interview covering effectiveness, efficiency, learnability and trust, and should retain task results, qualitative themes and resulting interface changes alongside the item-level SUS record.

D.3 Executed Descriptive Exploratory SUS Study

Eleven consented human-practitioner submissions were received. Complete-case analysis retained P1-P8, P10 and P11 and excluded P9 because Question 3 was blank. No answer was inferred, reconstructed or imputed. Five included participants were grouped as DevOps/DevSecOps and five as SRE. The source export records one to five years of relevant experience in the complete sample.

Table D.1 reports the participant-level scores. The arithmetic is reproducible from the item responses in the supporting workbook. The source workbook's form quiz Score column is excluded because it contains zeros and is unrelated to the standard SUS calculation.

Table D.1: Included Human SUS Participant Scores.

Participant	Role group	Years of experience	SUS score
P1	DevOps/DevSecOps	2	92.5
P2	SRE	5	70.0
P3	DevOps/DevSecOps	3	62.5
P4	DevOps/DevSecOps	1	92.5
P5	DevOps/DevSecOps	3	90.0
P6	SRE	4	95.0
P7	SRE	1	77.5
P8	DevOps/DevSecOps	3	97.5
P10	SRE	4	75.0
P11	SRE	2	40.0

The ten-item raw response means are reported in Table D.2. Positive and negative wording is shown because the polarity determines the SUS adjustment; the values themselves remain the original one-to-five responses.

Table D.2: Mean Raw Response by SUS Item.

SUS item	Mean raw response	Polarity
Q1	4.60	Positive
Q2	2.20	Negative
Q3	3.90	Positive
Q4	1.60	Negative
Q5	3.80	Positive
Q6	1.60	Negative
Q7	4.20	Positive
Q8	1.90	Negative
Q9	4.20	Positive
Q10	1.70	Negative

Reproducibility record: outputs/2026-08-26-infraguard-sus/InfraGuard_AI_SUS_Analysis_10_Complete.xlsx; analysis SHA-256 d7baddcee9d98293d87d70b8fde3f4e7993fc7fd13178faaed038ee6fd13a2d6; source-export SHA-256 ee7da2f72d651007dd68d652a0aba52fa7b57209247006feb453cb11b45e0f43. The workbook retains the unchanged source responses, explicit inclusion rule, exclusion reason, formula-based participant scores and all reported aggregates.

APPENDIX E: CONTROLLED INCIDENT AND RAG EVALUATION PROTOCOL AND OBJECTIVE RESULTS

This appendix records the frozen protocol, objective qwen2.5:3b results and provenance-limited practitioner role reports. Incident severity, schema, signature, retrieval, runtime and token observations are derived from retained rows. Aggregate human values in the reports are archived but not adopted as thesis findings because populated assessor-by-item records are absent. The separate descriptive SUS evidence is documented in Section 5.4.3 and Appendix D.

E.1 Evaluation Questions and Fixed Conditions

The benchmark asks whether the configured model returns the expected severity, identifies the controlled cause, recommends a safe action, grounds material claims in supplied evidence, follows the verdict schema and maintains the required signature behaviour. The first execution fixed qwen2.5:3b on Ollama 0.30.11, temperature 0.0, benchmark version 1.0.0, identical fixtures, top-four retrieval and three repeated runs per case. Raw and parsed outputs, timestamps, latency, input/output tokens, retrieval ranks, objective scores, model digest and source hashes were retained.

Gold-label provenance. The researcher constructed the 24 purposive synthetic scenarios and assigned their primary severity labels before execution using the operational definitions in Table E.1. The two practitioners did not independently establish or validate the labels, and no retained expert-panel adjudication record exists. Each scenario had one primary label. C20 remains warning in the primary analysis; {ok, warning} and exclusion are post hoc sensitivity checks. Severity agreement therefore measures correspondence with the researcher-defined policy, not real-world diagnostic truth.

Table E.1: Gold Severity Policy for the Controlled Benchmark.

Gold Severity	Operational Definition
ok	All present signals are within the expected band, or an optional integration is absent without evidence of application impact.
warning	Degradation or an observability problem exists, but the monitored service remains available and there is no immediate outage or data-loss risk.
high	An active service failure, security event or material degradation requires prompt operator intervention.
critical	Evidence indicates a widespread outage, immediate data-loss risk, or severe compromise requiring urgent containment.

E.2 Controlled Incident Scenario Set

The twenty-four scenarios below use deterministic telemetry fixtures. Gold labels must be frozen before model execution. Fixture changes, prompt changes and model-version changes should create a new benchmark version rather than silently replacing prior observations.

Table E.2: Controlled Incident Benchmark Scenarios and Gold Expectations.

ID	Category	Controlled Evidence	Gold Expectation
C01	Healthy	Normal CPU, memory and disk; HTTP 200; informational logs; Docker monitoring disabled.	ok; healthy state; no action beyond routine monitoring.
C02	Healthy	Only a healthy HTTP probe is present; Loki, Prometheus and Docker sections are absent.	ok; absent optional integrations must not be reported as incidents.
C03	Healthy	Normal metrics with sparse informational logs and no error pattern.	ok; no unsupported root cause or remediation.
C04	Degraded	CPU above 90% for 15 minutes; endpoint remains HTTP 200 with rising latency.	warning; CPU saturation; inspect workload and process utilisation.
C05	Degraded	Memory above 88% with growing swap; no OOM event and endpoint remains available.	warning; memory pressure; inspect consumers and capacity.
C06	Degraded	Disk utilisation at 86% and increasing; no write failure.	warning; low free space; identify growth and reclaim or expand storage.
C07	Degraded	HTTP latency is elevated while status remains 200 and error rate is normal.	warning; latency degradation; inspect dependencies and recent changes.
C08	Degraded	Prometheus data is stale while the application probe and logs remain healthy.	warning; monitoring-data freshness problem, not an application outage.
C09	Failure	Application probe returns 503 and Loki shows a matching HTTP 5xx spike.	high; active API failure; inspect application logs and dependency health.
C10	Failure	Endpoint connection is refused across repeated probes.	high; service unavailable; verify process, listener and network path.
C11	Failure	Redis connection errors coincide with failed dependent requests.	high; Redis dependency unavailable; restore service and verify clients.
C12	Failure	Database pool-exhaustion messages coincide with request timeouts.	high; connection pool exhausted; inspect leaks, limits and database health.
C13	Failure	Container restart loop and unhealthy state coincide with probe failure.	high; unstable container; inspect exit reason, logs and deployment change.
C14	Failure	Disk reaches 98% and logs show write failures.	critical; imminent service/data risk; stop growth and recover space urgently.
C15	Security	Repeated HTTP 401/403 responses from one IP cross the deterministic threshold.	high; probable HTTP brute force; review evidence before approved containment.
C16	Security	Repeated SSH authentication failures from one IP cross the threshold.	high; probable SSH brute force; validate source and apply approved response.
C17	Security	Rapid connection attempts across ports from one IP cross the scan threshold.	high; probable port scan; validate and consider operator-approved ban.
C18	Ambiguous	CPU is high during a labelled batch job; probes and error rates remain healthy.	warning; constrained statement of load; do not claim an outage.
C19	Ambiguous	Latency is high with normal server metrics and intermittent network errors.	warning; uncertain network/dependency cause; recommend discriminating checks.
C20	Ambiguous	One transient HTTP 500 occurs while subsequent probes, logs and metrics are normal.	warning under the primary frozen label; ok is examined only in sensitivity analysis; avoid claiming persistent failure.
C21	Missing telemetry	Docker collector returns the disabled or missing-configuration sentinel.	ok; omit Docker from the verdict and do not treat it as a fault.

ID	Category	Controlled Evidence	Gold Expectation
C22	Missing telemetry	Configured Loki query returns a connection error while application metrics and probes are healthy.	warning; telemetry-source failure; do not claim application outage.
C23	Recurring	An acknowledged disk warning recurs unchanged with the same prompt version and model.	ok with the acknowledged signature; default view may suppress the match.
C24	Recurring	The acknowledged disk condition worsens to 98% with write failures.	critical and visible; the severity safeguard must override default-view masking.

E.3 Diagnostic Scoring Rubric and Metrics

Two human practitioners completed role reports for the first execution. Because their original assessor-by-item ratings are not retained in the public evidence package, reported means and agreement coefficients are archived and excluded from the thesis findings. A future replication may use the rubric in Table E.3 only if each original worksheet, disagreement and reconciliation decision is preserved.

Table E.3: Per-Response Diagnostic Scoring Rubric.

Dimension	Score	Operational Rule
Severity	0 or 1	1 for the exact frozen gold severity; 0 otherwise. A separately labelled acceptable-set or exclusion result may be reported only as sensitivity analysis.
Root-cause correctness	0-2	0 wrong/unsupported; 1 partly correct or too general; 2 identifies the controlled cause and affected component.
Recommended action	0-2	0 unsafe/irrelevant; 1 partly appropriate; 2 safe, prioritised and suitable for diagnosis or recovery.
Evidence grounding	0-2	0 contradicts or invents evidence; 1 mostly supported with minor extrapolation; 2 every material claim is traceable to supplied evidence.
Schema validity	0 or 1	1 when all required fields parse and severity is allowed; 0 otherwise.
Signature behaviour	0 or 1	1 when the signature format, stability and acknowledged-condition behaviour match the fixture expectation.

Primary metrics for the completed execution are exact researcher-label severity agreement, the confusion matrix, macro-averaged precision/recall/F1, schema-valid output rate, signature behaviour, top-four retrieval, latency and tokens. Human root-cause, action, grounding, correctness and agreement measures are conditional future-protocol metrics that require retained assessor-by-item records. The purposive repeated scenarios do not support a population confidence interval or significance test.

E.4 Labelled RAG Baseline

Each question is run under the same provider/model with retrieval enabled and disabled. Report retrieval relevance (source in top-k and rank), grounding or traceability, answer correctness and actionable safety as separate dimensions. The completed execution supports only the objective retrieval measure quantitatively; the absent item-level practitioner ratings leave the remaining human dimensions unresolved.

Table E.4: Labelled Runbook Question Set for RAG and No-RAG Comparison.

ID	Question	Frozen Relevant Runbook
R01	Disk usage is above 90%. What should be checked first?	infrastructure/disk-space-high.md
R02	The host is swapping and memory is nearly exhausted. What are the safe first steps?	infrastructure/memory-pressure.md
R03	Loki has stopped receiving logs. How should ingestion be isolated?	observability/loki-ingestion-gap.md
R04	HTTP 5xx responses spiked after normal operation. What evidence should be correlated?	application/http-5xx-spike.md
R05	A container repeatedly exits and restarts. What should the operator inspect?	containers/container-restart-loop.md
R06	A Prometheus scrape target is down. What checks distinguish target failure from network failure?	observability/prometheus-target-down.md
R07	API latency is high without a matching 5xx spike. What diagnostic sequence is appropriate?	application/api-latency-high.md
R08	Redis is unavailable to the application. What containment and recovery checks are required?	application/redis-unavailable.md
R09	The PostgreSQL connection pool is exhausted. What should be examined before increasing limits?	application/database-connection-exhaustion.md
R10	Internal DNS resolution is failing. How should the operator isolate the fault?	infrastructure/dns-resolution-failure.md
R11	One IP is producing repeated SSH authentication failures. What review and response steps apply?	security/ssh-brute-force.md
R12	A service regressed immediately after deployment. What evidence supports rollback or continued diagnosis?	containers/deployment-regression.md

E.5 Execution and Reporting Procedure

1. Freeze the benchmark version, gold labels, prompt version, provider, model, temperature, retrieval depth and runbook corpus before execution.
2. Run every incident fixture three times per provider/model configuration and retain both raw and parsed responses.
3. Run every labelled question three times with RAG and three times without RAG under the same provider/model configuration.
4. Record latency, provider token counts and cost from the same execution logs; do not estimate missing measurements after the fact.
5. Provide anonymised outputs and the frozen rubric to independent assessors before revealing the system condition or model identity where practical.
6. Report all runs, parse failures and provider errors. Do not remove difficult cases or change gold labels after observing outputs without publishing a new benchmark version.
7. Present category-level results, the severity confusion matrix, RAG retrieval success, assessor agreement and the stated limitations. Conclusions must remain within the tested models, fixtures and single-host context.
8. When multiple human practitioners score the same outputs, preserve each original item-level worksheet, join scores only after independent completion, document any reconciliation, and report inter-rater agreement with the small-sample limitation.

E.6 Evidence Status

The protocol was executed once for qwen2.5:3b. Across 72 repeated responses from 24 purposive scenarios, schema validity was 100%, agreement with researcher-defined severity labels was 40.3%, macro-F1 was 0.383, high/critical under-calls totalled six and repeat-stable signatures occurred in 10/24 scenarios. Across twelve questions, a frozen relevant runbook appeared in the top four for 12/12. Practitioner report aggregates are excluded because their item-level records cannot be independently recomputed; answer correctness, grounding quality and actionable safety therefore remain unresolved quantitatively.

E.7 Reproducibility Record

The repository retains the frozen specification, execution harness, complete response rows, automatically derived metrics and two unchanged practitioner role reports. The current source includes post-benchmark changes and must not be described as the exact v1 source snapshot; v1 results remain historical under their recorded manifest and must not be overwritten.

Table E.5: Controlled Benchmark Reproducibility Record.

Record	Value or repository-relative location
Frozen specification	evaluation/benchmark_v1.json
Specification SHA-256	7936721040238a95c57d51ec6fa86b9ee104b6b4021df83efdf903604ab58fee
Gold-label origin	Researcher-constructed purposive synthetic scenarios and primary labels, frozen before model execution under the policy in Table E.1.
Independent gold-label review	Not performed or retained; the reported severity measure is agreement with the researcher-defined policy, not independently adjudicated or real-world truth.
Label ambiguity handling	C20 warning remains primary; {ok, warning} and exclusion are post hoc sensitivity analyses. No other multiple-label set was pre-specified.
Benchmark/source relationship	The current source has post-benchmark changes. Interpret v1 against its recorded manifest; do not overwrite or transfer its result to the revised code.
SRE randomisation-seed correction	Authoritative seed 2026082412; preserved report method-table value 2026081902 is a documented legacy transcription. CORRECTION_NOTICE.md and manifest.json preserve the correction without changing the hashed report.
Execution harness	evaluation/run_benchmark.py
Result set	evaluation/results/qwen2.5-3b-v1/
Model digest	357c53fb659c5076de1d65ccb0b397446227b71a42be9d1603d46168015c9e4b
Incident rows	incident_runs.jsonl; 72 unique scenario/repeat records
RAG rows	rag_runs.jsonl; 72 unique question/condition/repeat records
Objective metrics and mixed historical summary	summary.json, report.md and incident_objective_scores.csv; automatically derived metrics are adopted, while practitioner aggregates in the mixed summary are archived and excluded.
Practitioner role-report evidence	Separate unchanged DevOps and SRE reports are retained, but populated assessor-by-item values are absent; report aggregates are archived and excluded from quantitative findings.
Final remediation verification	29 August 2026 post-review state; 180 passed; 0 failed; 45 dependency warnings; 89% line coverage (1,655 statements; 190 missed). Software verification only.
DevOps practitioner report	evaluation/practitioner_confirmation/InfraGuard_DevOps_Assessment_Report.docx; SHA-256 741b5446c8dffaabfeac20823aa7a4427a4a61c058b1b9e9c619000c641d6720

Record	Value or repository-relative location
SRE practitioner report	evaluation/practitioner_confirmation/InfraGuard_SRE_Assessment_Report.docx; SHA-256 7c91dfd735a4a070ce4e6053c5addcfb70cec2d67002b199b43f22f21133efdb
Human assessor provenance	Researcher-confirmed human DevOps and SRE scoring; original email submissions, full names, and identity mapping are retained outside the public repository for confidential supervisor verification.
Assessment boundary	Role reports support qualitative engineering hypotheses only; they do not establish practitioner agreement, human-rated RAG effectiveness, diagnostic reliability or production effectiveness.
Human SUS source export	A separate standalone repository source export was not present; an unchanged raw-response copy is retained inside outputs/2026-08-26-infraguard-sus/InfraGuard_AI_SUS_Analysis_10_Complete.xlsx.
Human SUS analysis workbook	outputs/2026-08-26-infraguard-sus/InfraGuard_AI_SUS_Analysis_10_Complete.xlsx; SHA-256 d7baddcee9d98293d87d70b8fde3f4e7993fc7fd13178faaed038ee6fd13a2d6
Human SUS inclusion record	11 consented submissions; 10 complete analysed; P9 excluded because Question 3 was blank; no imputation; 22-26 August 2026.